

Universidade Federal do ABC



**DESENVOLVIMENTO DE UMA FERRAMENTA
COMPUTACIONAL DE DIMENSIONAMENTO E DE AJUSTES DE
SPRAY PARA INJETORES CENTRÍFUGOS E CÁLCULOS DE
INJEÇÃO PARA UM PROPULSOR DE PEQUENO PORTE A
ETANOL E OXIGÊNIO LÍQUIDO**

ALEXANDRE COSTA GOULART

São Bernardo do Campo - SP

2023

ALEXANDRE COSTA GOULART

**DESENVOLVIMENTO DE UMA FERRAMENTA
COMPUTACIONAL DE DIMENSIONAMENTO E AJUSTES DE
SPRAY PARA INJETORES CENTRÍFUGOS E CÁLCULOS DE
INJEÇÃO PARA UM PROPULSOR DE PEQUENO PORTE A
ETANOL E OXIGÊNIO LÍQUIDO**

Trabalho de graduação, apresentado à
Universidade Federal do ABC, como requisito
para a obtenção do título de Bacharel em
Engenharia Aeroespacial.

Orientador: Alexandre Alves

Coorientador: Wladimyr Mattos C. Dourado

São Bernardo do Campo - SP

2023

Sistema de Bibliotecas da Universidade Federal do ABC
Elaborada pelo Sistema de Geração de Ficha Catalográfica da UFABC
com os dados fornecidos pelo(a) autor(a).

Costa Goulart, Alexandre

DESENVOLVIMENTO DE UMA FERRAMENTA
COMPUTACIONAL DE DIMENSIONAMENTO E DE
AJUSTES DE SPRAY PARA INJETORES CENTRÍFUGOS E
CÁLCULOS DE INJEÇÃO PARA UM PROPULSOR DE
PEQUENO PORTE A ETANOL E OXIGÊNIO LÍQUIDO /
Alexandre Costa Goulart. — 2023.

132 fls. : il.

Orientador: Alexandre Alves

Coorientador: Wladimir Mattos da Costa Dourado

Trabalho de Conclusão de Curso — Universidade Federal do
ABC, Bacharelado em Engenharia Aeroespacial, São
Bernardo do Campo, 2023.

1. Motor Foguete. 2. Propulsão Líquida. 3. Injetores
Centrífugos. 4. Injetores Bipropelente. 5. Atomização. I. Alves,
Alexandre. II. Mattos da Costa Dourado, Wladimir. III.
Bacharelado em Engenharia Aeroespacial, 2023. IV. Título.



CENTRO DE ENGENHARIA, MODELAGEM E CIÊNCIAS
SOCIAIS APLICADAS

ENGENHARIA AEROESPACIAL

Aluno: Alexandre Costa Goulart

RA: 11003314

Título do TG: Desenvolvimento de uma ferramenta computacional de dimensionamento e de ajustes de spray para injetores centrífugos e cálculos de injeção para um propulsor de pequeno porte a etanol e oxigênio líquido.

Avaliador Orientador: Dr. Alexandre Alves

Instituição: Universidade Federal do ABC - UFABC

Conceito atribuído ao trabalho: A

Avaliador: Dr. Annibal Hetem Junior

Instituição: Universidade Federal do ABC - UFABC

Conceito atribuído ao trabalho: A

Avaliador: Dra. Thaís Maia Araújo

Instituição: Universidade Federal do ABC - UFABC

Conceito atribuído ao trabalho: A

Data: 15/08/2023

Conceito Final: A

A handwritten signature in black ink, appearing to be 'Dr. Alexandre Alves', written over a horizontal line.

Assinatura do Orientador

ALEXANDRE COSTA GOULART

**DESENVOLVIMENTO DE UMA FERRAMENTA
COMPUTACIONAL DE DIMENSIONAMENTO E AJUSTES DE
SPRAY PARA INJETORES CENTRÍFUGOS E CÁLCULOS DE
INJEÇÃO PARA UM PROPULSOR DE PEQUENO PORTE A
ETANOL E OXIGÊNIO LÍQUIDO**

Trabalho de graduação, apresentado à
Universidade Federal do ABC, como requisito
para a obtenção do título de Bacharel em
Engenharia Aeroespacial.

BANCA EXAMINADORA

Prof. Dr. Alexandre Alves
Universidade Federal do ABC

Prof. Dr. Annibal Hetem Junior
Universidade Federal do ABC

Profa. Dra. Thaís Maia Araújo
Universidade Federal do ABC

São Bernardo do Campo - SP

2023

AGRADECIMENTOS

Primeiramente gostaria de agradecer meu Orientador Prof. Dr. Alexandre Alves e meu Coorientador Prof. Dr. Wladimyr Dourado pela orientação deste trabalho, inspiração e ensino de tantos temas que colaboraram tanto para minha graduação quanto para minha formação pessoal.

Também faço um agradecimento a mais alguns pesquisadores da área de propulsão espacial que há anos também fazem a diferença no setor e na UFABC. São eles José Nivaldo Hinckel, Fábio Mota, Loreto Pizzuti, Thaís Maia, Mariana Conti e Maurício Gontijo.

Muitos amigos também tornaram essa jornada possível, mas destaco a participação dos grandes companheiros Gabriel Yamato, Bruno Costa, João Pedro Vilas Boas e Calvin Trubiene.

Também quero agradecer meus familiares por todo o carinho e lições de vida que foram tão preciosos durante toda minha trajetória de vida, além da minha companheira Isabelle Ianaguivara Kisanuki que torna este percurso tão mais leve e agradável.

RESUMO

O desenvolvimento de motores foguete é fundamental para o progresso do setor espacial, cujo momento apresenta intenso crescimento e inovação. Sendo o projeto dos injetores um dos pilares centrais do desenvolvimento de propulsores, são expostos métodos de validação experimental e é desenvolvida uma ferramenta computacional de dimensionamento de injetores centrífugos monopropelente e bipropelente, e é proposto um par de injetores para um propulsor de pesquisa. São utilizados os injetores dos motores RD0110 e RD0109 para validar os resultados dos algoritmos de dimensionamento e de ajustes, resultando num erro aceitável para o qual são sugeridos trabalhos futuros para delinear precisamente sua origem, realizar melhorias no método de cálculo e efetuar validações experimentais.

Palavras-chave: Motor-Foguete, Propulsão Líquida, Etanol, Oxigênio Líquido, Injetores Centrífugos, Injetores Bipropelente, Atomização.

ABSTRACT

The development of Rocket engines is a fundamental subject for the progress of the space sector, which is now in a critical moment of growth and innovation. Being injector design one of the main pillars of the development of space propulsion, in this work a computational tool for the design of monopropellant and bipropellant injectors is created and a pair of injectors is calculated based on a proposed research thruster using Ethanol and Liquid Oxygen. The injectors of the RD0110 and RD0109 engines are used as reference for validation of the results. Acceptable results are obtained, and discrepancies are discussed additionally to suggestion of future works to improve the current development.

Keywords: Rocket Engine, Liquid Propulsion, Ethanol, Liquid Oxygen, Swirl Injectors, Bipropellant Injectors, Atomization.

Lista de Siglas e Símbolos

MMD	: Mediana do Diâmetro Mássico
SMD	: Diâmetro Médio de Sauter
LOx	: Oxigênio Líquido
LN2	: Nitrogênio Líquido
RG-1	: Querosene específica para aplicação em motores foguete soviéticos
ST1	: 1º Estágio do injetor bipropelente (bocal de menor diâmetro)
ST2	: 2º Estágio do injetor bipropelente (bocal de maior diâmetro)

Geometria dos injetores

R_n	: Raio do bocal de saída do injetor
L_n	: Comprimento do bocal de saída do injetor
R_s	: Raio da câmara de vórtice
D_s	: Diâmetro da câmara de vórtice
L_s	: Comprimento da câmara de vórtice
R_{in}	: Raio da posição do canal tangencial
r_{in}	: Raio do canal tangencial
l_{in}	: Comprimento do canal tangencial
n_{in}	: Número de canais tangenciais
t_w	: Espessura da parede entre estágios em um injetor bipropelente
h_1	: Comprimento do recesso do 1º estágio de um injetor bipropelente
θ_c	: Ângulo de convergência da câmara de vórtice para o bocal do injetor

Propriedades dos injetores

- $\dot{m}$: Vazão mássica do injetor
- ΔP : Diferencial de pressão aplicado no injetor
- α : Ângulo do eixo longitudinal do injetor ao cone do spray
- α_{in} : Ângulo de entrada do canal tangencial
- μ : Coeficiente de descarga do injetor
- φ : Coeficiente de preenchimento do injetor
- r_{mn} : Raio da camada líquida no interior do bocal do injetor
- A : Parâmetro geométrico característico do injetor
- λ : Coeficiente de atrito
- ξ_{in} : Coeficiente de perda hidráulica nos canais tangenciais
- ξ : Coeficiente de perda hidráulica total do injetor
- K : Coeficiente de perda de momento angular
- λ_i : Razão de velocidades característica para bocal de diâmetro ampliado

Subscritos:

- i : Valor calculado para fluido ideal, desconsiderando a viscosidade
- r : Valor calculado considerando as perdas viscosas

Propriedades dos Fluidos

- σ_p : Tensão Superficial do propelente
- μ_p : Viscosidade Dinâmica do propelente
- ν_p : viscosidade cinemática do propelente
- ρ_p : Massa Específica do propelente
- ρ_g : Massa Específica do gás circundante

ÍNDICE

1	Introdução	1
1.1	Motivação e Justificativa	3
1.2	Objetivos	5
1.3	Estruturação do trabalho	5
2	Revisão Bibliográfica	7
2.1	Dimensionamento de Injetores Centrífugos Monopropelente	8
2.2	Atomização e Tamanho de Gota	11
2.3	Configuração e Mistura de Injetores Bipropelente	14
3	Metodologia	18
3.1	Cálculos de Injetores Centrífugos	23
3.1.1	Cálculo do Injetor Ideal	23
3.1.2	Cálculo do Injetor Real	24
3.1.3	Cálculo do Injetor modulado – Saída Expandida	26
3.1.4	Cálculo do Injetor modulado – Saída em Corte	27
3.1.5	Algoritmos de dimensionamento	28
4	Resultados e Discussão	30
4.1	Resultados do Algoritmo de Dimensionamento M1	31
4.1.1	Resultados do M1 para o RD0110	31
4.1.2	Resultados do M1 para o RD0109	35
4.1.3	Resultados do M1 para o Propulsor Proposto – Injetor Central	38
4.1.4	Dados do M2 para o Propulsor Proposto – Injetor Periférico	41
4.2	Resultados do Algoritmo de Ajustes M2	43

4.2.1	Resultados do M2 para o RD0110	43
4.2.2	Resultados do M2 para o RD0109	45
4.2.3	Resultados do M2 para o Propulsor Proposto – Injetor Central	46
4.2.4	Resultados do M2 para o Propulsor Proposto – Injetor Periférico.....	47
4.3	Valores Calculados de Tamanho de Gota.....	48
5	Conclusões e Trabalhos Futuros.....	50
6	Referências.....	53
7	Anexo A – Códigos do Método M1	57
7.1	M1_main.py	57
7.2	M1_FileReader.py.....	58
7.3	M1_Method.py	63
7.4	M1 Input File – RD0110: RD0110.txt.....	86
7.5	M1 Input File – RD0109: RD0109.txt.....	89
7.6	M1 Input File – Injetor Proposto Central: B01_C.txt.....	92
7.7	M1 Input File – Injetor Proposto Periférico: B01_W.txt	95
8	Anexo B – Códigos do Método M2	98
8.1	M2_main.py	98
8.2	M2_FileReader.py.....	99
8.3	M2_Method.py	101
8.4	M2 Input File – RD0110	109
8.4.1	Estágio 1 – Oxidante: RD0110_Ox.txt.....	109
8.4.2	Estágio 2 – Combustível: RD0110_F.txt.....	110
8.5	M2 Input File – RD0109	112

8.5.1	Estágio 1 – Oxidante: RD0109_Ox.txt.....	112
8.5.2	Estágio 2 – Combustível: RD0109_F.txt.....	113
8.6	M2 Input File – Injetor Proposto Central	115
8.6.1	Estágio 1 – Oxidante: B01_C_Ox.txt.....	115
8.6.2	Estágio 2 – Combustível: B01_C_F.txt.....	116
8.7	M2 Input File – Injetor Proposto Periférico	118
8.7.1	Estágio 1 – Oxidante: B01_W_Ox.txt	118
8.7.2	Estágio 2 – Combustível: B01_W_F.txt	119

1 INTRODUÇÃO

O desenvolvimento de propulsores se mostra como um dos principais desafios nas ciências espaciais. Possuindo estreitas restrições operacionais e de performance, muitas vezes sendo necessários propulsores mais complexos, como os propulsores líquidos. Em geral estes apresentam um maior desempenho, traduzido em um Impulso Específico superior em comparação com propulsores sólidos ou híbridos. Além disso, apresentam uma maior versatilidade operacional, podendo ser desligado e religado em voo ou órbita e atualmente podendo até mesmo ser reutilizados, operando em múltiplas missões.

É previsto que o mercado espacial como um todo triplique em valor de mercado até 2040 (CNBC, 2017), impactando positivamente no número de lançamentos ao espaço. Neste cenário em que notícias e relatórios de mercado (BRYCE TECH, 2022) indicam amplo crescimento para o setor, proliferam-se as *startups* que seguem um modelo denominado *New Space*. Nesta categoria é típica a implementação de tecnologias disruptivas, o uso de *smallsats* e a promoção de lançamentos orbitais com menor custo e menor *lead time* como formas de alcançar novas fatias de mercado que se abrem e são vistas com bons olhos pelos investidores privados (MARTIN, 2015). Em geral continua havendo grande participação pública quanto aos investimentos realizados no setor, no entanto essa considerável parcela de capital privado introduzida em novos empreendimentos se mostra como uma característica especialmente importante do *new space*.

Tanto os veículos lançadores quanto a grande maioria das espaçonaves requerem o uso de propulsores, o que neste contexto certamente traz grande interesse econômico ao desenvolvimento e pesquisa destes dispositivos. Além disso, os propulsores são pontos centrais no esforço de redução de custos e implementação de tecnologias disruptivas especialmente no que tange ao seu projeto e fabricação. Um exemplo sólido disso é a popularização dos processos de manufatura aditiva em metal de ligas de alto desempenho para aplicação em câmaras de combustão (GRADL, PROTZ, *et al.*, 2019), cabeçotes e

componentes diversos, que podem representar vantagens tanto do ponto de vista na redução do número de peças e complexidade na fabricação quanto em novas possibilidades de desenho e dimensionamento (KAJON, LIUZZI, *et al.*, 2019).

No cenário nacional de propulsão espacial temos exemplos de propulsores líquidos como o L5, o L15 e o L75, entregando respectivamente 5, 15 e 75 kN de empuxo. No caso do último, este está sendo desenvolvido em conjunto com a Agência Espacial Alemã (DLR). Ainda com todos estes desenvolvimentos, esta categoria de motorização ainda é um grande desafio, pois em veículos lançadores é usual a utilização de propulsores com empuxo na ordem das centenas aos milhares de kN que, muitas vezes, utilizam ciclos termodinâmicos mais eficientes e mais complexos.

No panorama atual, evidenciado pelos temas recorrentes em simpósios, congressos e revistas científicas renomadas, prevalecem os temas como o de reutilização de veículos lançadores, que geram uma redução de custo significativa para o lançamento à órbita, mas que ainda são palco de ampla discussão devido a questões como custos e possível ociosidade das infraestruturas responsáveis pela manufatura dos veículos (CALABUIG e WILKEN, 2022), (CALLSEN, STAPPERT e SIPPEL, 2022). Ainda assim, têm-se como exemplo os desenvolvimentos recentes atrelados à Agência Espacial Europeia como o motor Prometheus que tem como objetivo não apenas a reusabilidade, mas também a redução drástica de custos de produção por meio do uso de novas tecnologias de manufatura aditiva (SOLLER, BEHR, *et al.*, 2017). Também está em evidência o estudo dos “propelentes verdes” ou *green propellants* (TOKUDOME, YAGISHITA, *et al.*, 2013), que são estocáveis (em reservatórios de satélites, por exemplo) e possuem menor toxicidade e impacto ambiental que os, amplamente utilizados, propelentes hipergólicos baseados em Hidrazina.

Com o exposto, se mostra como imperativo realizar pesquisa nestes quesitos e para tal, uma das principais ferramentas é o desenvolvimento de motor foguete, que mesmo em escala reduzida, é capaz de explorar conceitos de injetores, pares propelentes e câmaras de

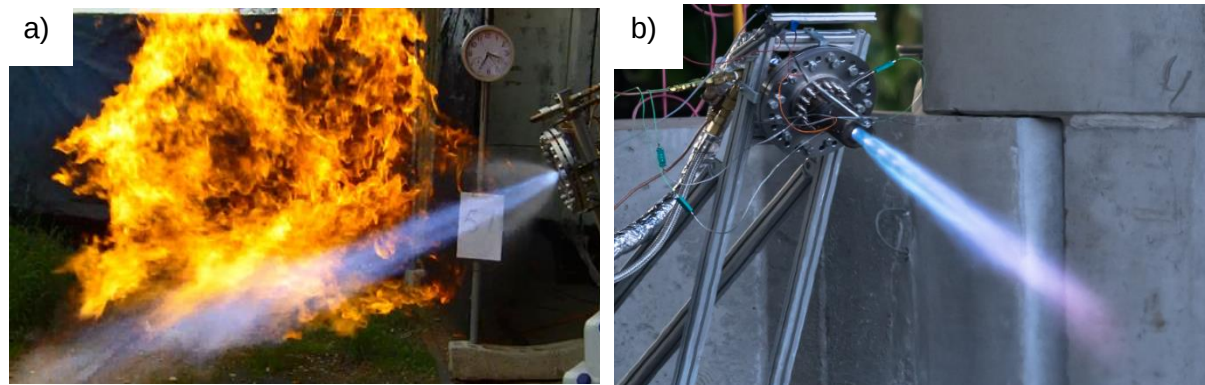
combustão tanto em viés didático para demonstrações e instruções que preparem pessoal capacitado na área, como para pesquisa no estado da arte, seguindo os temas citados.

1.1 Motivação e Justificativa

Além das motivações provenientes das tendências e necessidades do Mercado espacial no mundo, como exposto na seção anterior, o desenvolvimento de uma bancada de testes de motores foguete e de um propulsor segue uma tendência evidente em universidades de renome no mundo, sendo estas muitas vezes atreladas às respectivas agências espaciais de seus países. Tal inclinação segue desenvolvimentos atuais da propulsão espacial como descrito pela subdivisão de temas da 8ª *European Conference for Aeronautics and Space Sciences* (8th EUCASS), por exemplo, “*Conference on Reusable Systems for Space Access*” (EUCASS2019) e na 7ª EUCASS com uma categoria de publicação dedicada aos *green propellants* (EUCASS2017), que incluem a utilização do Etanol como combustível e oxidantes como Óxido Nitroso ou Peróxido de Hidrogênio (INDIANA, BOUST, *et al.*, 2017). A Technische Universität Dresden (TU Dresden) desenvolveu com estudantes o projeto “SMART Rocket Project” (SIDER, KLEEBUSCH, *et al.*, 2017), utilizando o mesmo par propelente pretendido para o trabalho em questão, Etanol e Oxigênio líquido e com um empuxo de 500 N. Na Figura 1 são apresentados um teste a quente da placa de injeção e um teste do propulsor completo.

A agência espacial francesa, CNES, desenvolveu com estudantes o propulsor Perseus (DELPY e OSWALD, 2017), utilizando o mesmo par propelente pretendido para o trabalho em questão, Etanol e Oxigênio líquido, e com um empuxo de 5 kN. Na Figura 2 é apresentado um teste a frio da placa de injeção apresentando o fluxo de combustível, o Fluxo de um substituto do oxidante e o fluxo de ambos.

Figura 1 - a) teste à quente da placa de injeção constituída de um único injetor centrífugo bipropelente; b) teste da câmara de empuxo de 500 N.



Fonte: (SIDER, KLEEBUSCH, *et al.*, 2017)

Figura 2 - Testes à frio da placa de injeção do motor Perseus.



Fonte: (DELPY e OSWALD, 2017)

Outros exemplos são:

- UCSD (University of California, San Diego) (UCSD-SEDS, 2016): Desenvolvimento da bancada Colossus e dos motores de propulsão líquida Tri-D, Ignus I, Callan e Ignus II; todos construídos com manufatura aditiva metálica.
- SDSU (San Diego State University) (SDSU ROCKET PROJECT): Desenvolvimento de um motor de propelente líquido à Metano líquido e oxigênio líquido para a competição FAR/MARS
- Cal Poly Pomona (California Polytechnic State University, Pomona) (BETADY, ORTIZ, *et al.*, 2018); (JOHNSON): Desenvolvimento de uma bancada de testes de motores a propelente líquido para o desenvolvimento de um motor para a competição FAR/MARS e posteriormente desenvolver seu próprio veículo suborbital.
- TU Delft (Delft University of Technology) (DARE - TU DELFT): Desenvolvendo o motor DCX-1 à etanol e oxigênio líquido e a bancada para os respectivos testes.

- DanStar, constituída de alunos da *Technical University of Denmark* (DTU) e da *Maskinmesterskolen København* (MSK) (DANSTAR - TECHNICAL UNIVERSITY OF DENMARK): Desenvolvimento de uma bancada de testes de motores foguete de propelente líquido e um motor de demonstração de 300 N à Isopropanol e Óxido Nitroso.

1.2 Objetivos

O primeiro objetivo deste trabalho é desenvolver um software que auxilie no projeto de injetores centrífugos para propulsores foguete à propelente líquido. Para tal, implementado a metodologia de dimensionamento desenvolvida por Bazarov (BAZAROV, YANG e PURI, 2004) e validando-os com dados experimentais apresentados por Alves (ALVES, 2008) e Rubinsky (RUBINSKY, 1994), respectivamente dos injetores bipropelente dos motores RD0109 e RD0110.

O segundo objetivo é propor os injetores para um propulsor a Oxigênio Líquido e Etanol com aplicação didática e de pesquisa utilizando o software desenvolvido no seu dimensionamento.

1.3 Estruturação do trabalho

Na Introdução é apresentada uma contextualização das condições e dos desenvolvimentos sendo realizados no setor e na área específica de propulsão espacial, seguido da motivação do trabalho, focado na criação de infraestrutura de ensino e pesquisa para a universidade, e finalmente, são apresentados os objetivos.

Na Revisão Bibliográfica são apresentados os conceitos fundamentais sobre injetores, injetores para aplicação em propulsores espaciais e, mais especificamente configuração de injetores bipropelente. Também são apresentados trabalhos que demonstram o estado da arte dos estudos sendo realizados com injetores centrífugos e são discutidos os efeitos e parâmetros que podem afetar seu desempenho.

Na Metodologia são descritos os parâmetros de embasamento do desenvolvimento dos injetores e dos códigos de dimensionamento, além de descrever o funcionamento e os objetivos de cada algoritmo

Nos Resultados e Discussão são comparados os resultados obtidos utilizando os algoritmos desenvolvidos com dados disponíveis para os injetores do propulsor RD-0109 e RD-0110. Além disso, são apresentados os valores obtidos para os injetores propostos no trabalho.

Nas Conclusões e Trabalhos Futuros são relatados e discutidos acertos, divergências e efeitos que podem influenciar os resultados, além de descrever possíveis melhorias e trabalhos futuros no sentido de melhorar os resultados obtidos e ampliá-los para o contexto de controle de fabricação numa linha de produção.

2 REVISÃO BIBLIOGRÁFICA

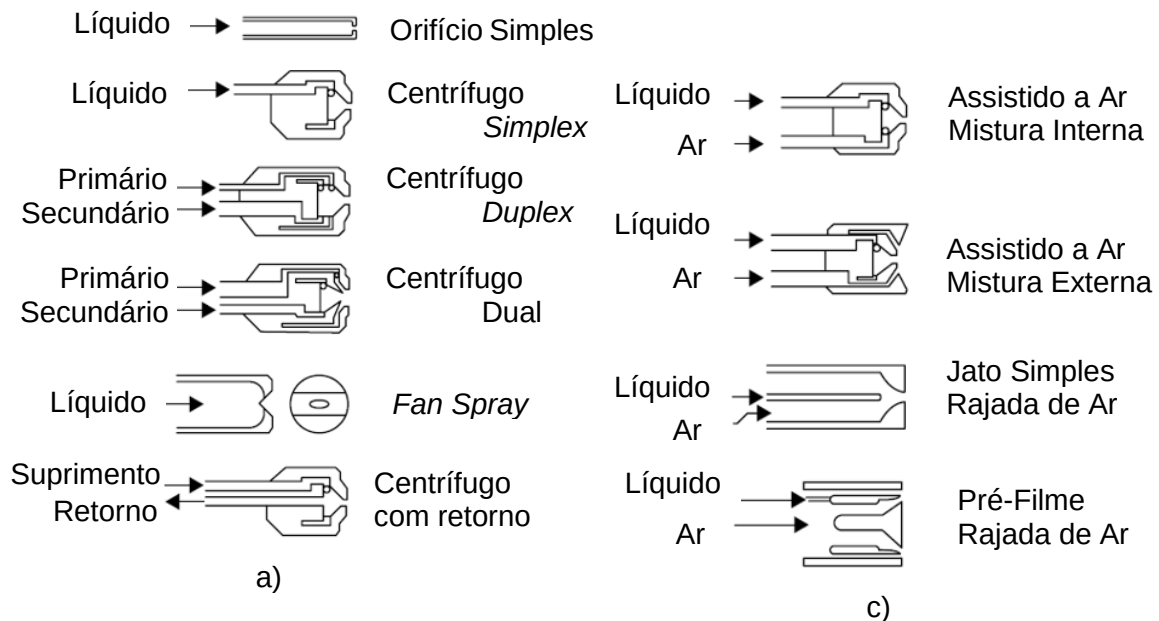
Existe uma ampla gama de Injetores os quais desempenham diversas funções tanto na indústria quanto no dia a dia. Nas aplicações atreladas aos processos de combustão em geral têm-se o papel central da injeção de combustível, de oxidante, ou de uma mistura de ambos, sendo central na garantia de performance e funcionamento adequado de tais dispositivos.

De acordo com Sutton (SUTTON e BIBLARZ, 2017), para propulsores espaciais à propelente líquido, a função dos injetores consiste em introduzir de forma regulada e na devida proporção, os propelentes no interior da câmara de combustão. Simultaneamente, quando em estado líquido, devem transformar a corrente contínua destes fluidos em pequenas gotículas, ou seja, promover sua atomização.

No processo de atomização, segundo Lefebvre (LEFEBVRE e MCDONELL, 2017), os jatos ou filmes de fluido são desintegrados pela energia cinética do fluido que os compõe, pela exposição a gases em alta velocidade ou por energia mecânica aplicada externamente. O mecanismo de desintegração é particular para cada tipo de injetor, dos quais alguns exemplos são expostos na Figura 3.

Em especial para o uso em propulsores espaciais, (SUTTON e BIBLARZ, 2017) apresenta-se alguns injetores específicos como os de jatos colidentes, que combinam dois ou mais injetores de orifício simples posicionados de forma a seus sprays colidirem, e os do tipo Pintle, que, de acordo com (VASQUES e HAIDN, 2017), consistem em uma combinação de injetores que criem filmes ou jatos de forma que um dos propelentes seja ejetado longitudinalmente e o outro radialmente de forma a interceptarem-se. Ambos são apresentados na Figura 4.

Figura 3 – Tipos de Injetores categorizados como a) Atomizadores por pressão; b) Atomizadores rotativos; c) Atomizadores que utilizam dois fluidos distintos.



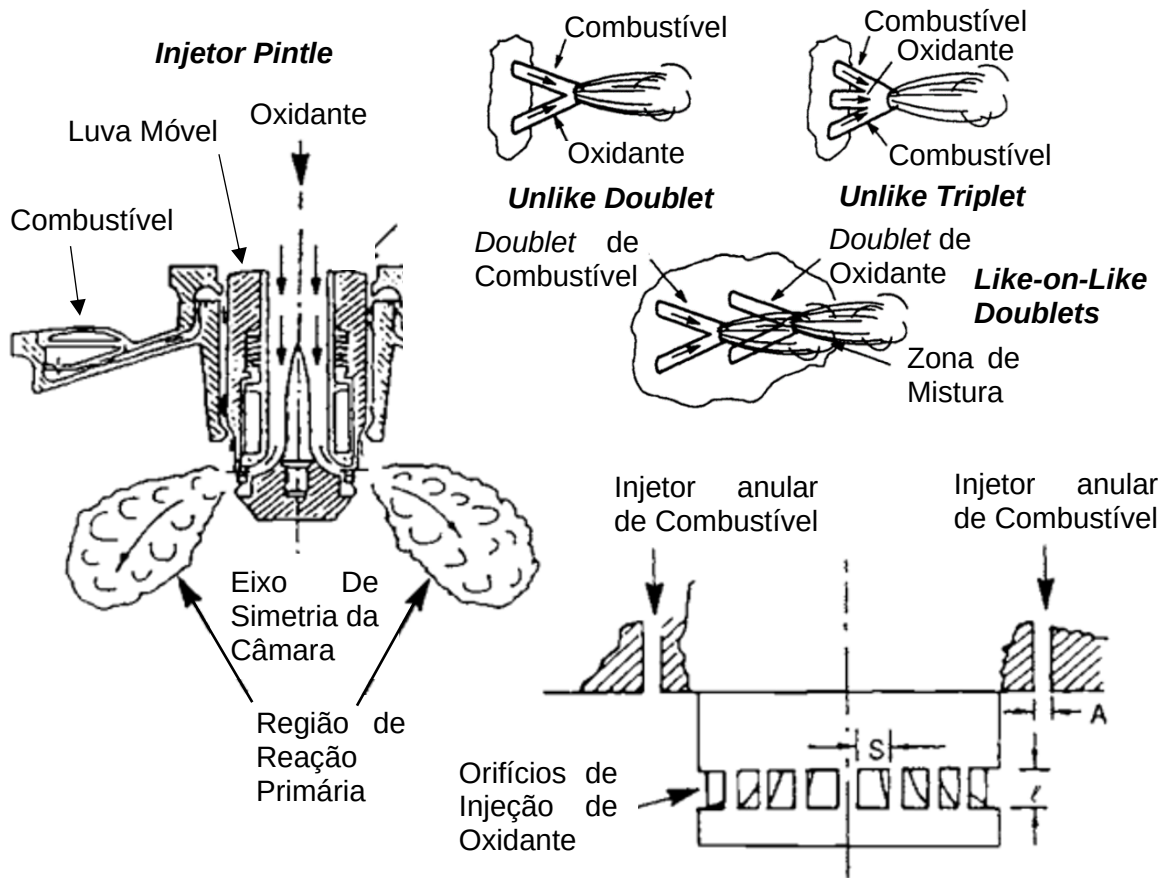
Fonte: (LEFEBVRE e MCDONELL, 2017)

Neste trabalho serão estudados os injetores centrífugos, os quais, de acordo com Bayvel (BAYVEL e ORZECOWSKI, 1993), apresentam diversas vantagens como possuir um design simples, apresentar alta confiabilidade, prover boa qualidade de atomização e necessitar de pequenos diferenciais de pressão na sua operação. Como exposto na Figura 4 esses podem ser classificados como Simplex, Duplex e Dual (ou de dois orifícios).

2.1 Dimensionamento de Injetores Centrífugos Monopropelente

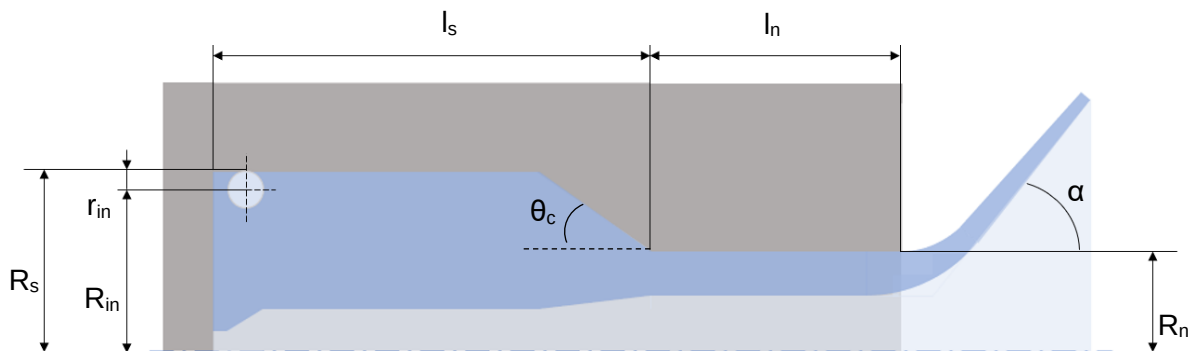
A modelagem das propriedades de injetores centrífugos tem suas origens nas propostas de cálculo desenvolvidas por Abramovich e que foram ampliadas, por exemplo, com as propostas de Kliachko. Outras propostas surgiram, porém, este formato proposto teve grande adoção por conta da sua relativa simplicidade em comparação com outros métodos. Tal base teórica foi adotada nos trabalhos do Bazarov (BAZAROV, YANG e PURI, 2004) e Kessaev (ALVES, 2008), que embasaram os cálculos desenvolvidos neste trabalho. Na Figura 5 é apresentado o diagrama de um injetor centrífugo com as principais medidas identificadas.

Figura 4 - Exemplos de injetores do tipo jatos impingentes e do tipo pintle.



Fonte: (HUZEL e HUANG, 1992)

Figura 5 – Diagrama do interior de um injetor centrífugo e suas principais medidas.



Fonte: Elaborado pelo autor.

No trabalho de Kavkhin (KHAVKIN, 2004) são discutidas as diversas modelagens conhecidas para injetores centrífugos e suas vantagens, desvantagens, além da sua aderência aos resultados experimentais desenvolvidos por ele e outros autores. Há um destaque especial à discussão sobre o “Princípio de Consumo Máximo” que completa a teoria de Abramovich, mas carece de interpretação física que a justifique com clareza dado que este origina-se de teorias ultrapassadas de cálculo de vertedouros largos.

Apesar de outras propostas que não dependem de tal princípio serem descritas, trabalhos de Kliachko e Zukovsky (BAZAROV, YANG e PURI, 2004), (KHAVKIN, 2004) justificam o uso deste princípio e demonstram que sua modelagem não acarreta consequências práticas nos resultados obtidos.

São postuladas duas variáveis que auxiliam no projeto de injetores centrífugos, que são o parâmetro geométrico característico A (Equação (2.1)), que correlaciona as principais dimensões do injetor:

$$A = \frac{R_{in} \cdot R_n}{n \cdot r_{in}^2} \quad (2.1)$$

E, o coeficiente de preenchimento φ (Equação (2.2)), que traz informação sobre a condição do fluido no interior do injetor:

$$\varphi = \frac{\text{Área não ocupada por fluido no bocal}}{\text{Área de seção do bocal}} = \frac{\pi \cdot (R_n^2 - r_{mn}^2)}{\pi \cdot R_n^2} = 1 - \frac{r_{mn}^2}{R_n^2} \quad (2.2)$$

Utilizando a geometria do injetor, os parâmetros básicos de operação e do fluido de trabalho, as relações de conservação de massa e momento angular é possível correlacionar o coeficiente de preenchimento e o parâmetro geométrico característico ao coeficiente de descarga do injetor (Equação (2.3)). Aplicando-se o princípio de consumo máximo, têm-se

que para cada valor de parâmetro geométrico escolhido, o coeficiente de preenchimento correspondente será o que provê o maior coeficiente de descarga:

$$\mu = \left(\frac{A^2}{1 - \varphi} + \frac{1}{\varphi^2} \right)^{-\frac{1}{2}} \quad (2.3)$$

Esta modelagem contempla um injetor com fluido ideal, sem viscosidade. Para o cálculo do valor efetivo da vazão com fluido real, são utilizados fatores de correção dos efeitos viscosos e um processo iterativo que converge ao valor almejado.

Outro fator que amplia a modelagem desenvolvida por Bazarov (BAZAROV, YANG e PURI, 2004), é a observação do efeito do ângulo de convergência escolhido, o qual foi estudado por Xue (XUE, JOG e JENG, 2002). Foi identificado que com um acréscimo do ângulo de convergência há um aumento da espessura do filme no bocal e do coeficiente de descarga, enquanto o ângulo do cone de spray decresce. No exemplo estudado houve um acréscimo de 30% no coeficiente de descarga ao variar θ_c de 45° a 90°.

A partir da análise de dinâmica de fluidos apresentadas por Xue (XUE, JOG e JENG, 2002), identifica-se que, com o aumento do ângulo de convergência, há um aumento da perda de momento angular, conseqüentemente reduzindo a velocidade tangencial, o ângulo do cone de spray e aumentando a vazão.

Como demonstrado, efeitos secundários como este podem impactar significativamente os valores obtidos, portanto, a modelagem utilizada neste trabalho deve ser considerada de forma criteriosa, já que não é capaz de contemplar todas as variáveis de projeto possíveis.

2.2 Atomização e Tamanho de Gota

A determinação do tamanho médio de gota e distribuição de tamanhos de gota são algumas das principais características que atestam uma performance adequada do injetor,

tendo relação direta com os processos de vaporização e mistura e, portanto, tem grande influência no processo de combustão (LEFEBVRE, 1983).

O processo de atomização da grande maioria dos injetores resulta numa distribuição ampla de tamanhos de gota, sendo necessário determinar padrões de comparação e análise. Um dos mais utilizados é o *Sauter Mean Diameter* (SMD), representado pelo diâmetro da gota que tenha a mesma razão de volume por área de superfície quanto o spray como um todo. Portanto, temos por definição o SMD representado como na Equação (2.4).

$$SMD \equiv \frac{\sum nD^3}{\sum nD^2} \quad (2.4)$$

Portanto, para determinar este valor a partir da geometria e parâmetros de operação de um injetor, podemos usar diversas relações expostas por Lefebvre (LEFEBVRE e MCDONELL, 2017), cujos métodos selecionados são expostos na Tabela 1.

Tabela 1 – Métodos de cálculo de SMD para injetores centrífugos.

Método	Equação
Radcliffe	$SMD(Radcliffe) = 7.3 \cdot \sigma_p^{0.6} \cdot \left(\frac{\mu_p}{\rho_p}\right)^{0.2} \cdot \dot{m}_p^{0.25} \cdot \Delta P_p^{-0.4}$
Jasuja	$SMD(Jasuja) = 4.4 \cdot \sigma_p^{0.6} \cdot \left(\frac{\mu_p}{\rho_p}\right)^{0.16} \cdot \dot{m}_p^{0.22} \cdot \Delta P_p^{-0.43}$
Lefebvre	$SMD(Lefebvre) = 2.25 \cdot \sigma_p^{2.25} \cdot \left(\frac{\mu_p \cdot \dot{m}_p}{\Delta P_p \cdot \rho_g}\right)^{0.25}$
Wang & Lefebvre	$SMD(W. \& L.) = 4.52 \cdot \left(\frac{\sigma_p \cdot \mu_p^2}{\rho_g \cdot \Delta P_p^2}\right)^{0.25} \cdot (t \cdot \cos \alpha)^{0.25} + 0.39 \cdot \left(\frac{\sigma_p \cdot \rho_p}{\rho_g \cdot \Delta P_p}\right)^{0.25} \cdot (t \cdot \cos \alpha)^{0.75}$

Fonte: (LEFEBVRE e MCDONELL, 2017)

Outro padrão importante é o Mass Mean Diameter (MMD) que é representado pelo diâmetro da gota para o qual 50% do volume total está em gotas de menor diâmetro e representado na Equação (2.5).

$$\text{MMD} \equiv \left(\frac{\sum nD^3}{\sum n} \right)^{0.33} \quad (2.5)$$

Em propulsores espaciais foi demonstrado que o valor SMD tem grande influência sobre a determinação de parâmetros fundamentais da câmara de empuxo como o comprimento característico (L^*) (GONTIJO, FISCHER e COSTA, 2020).

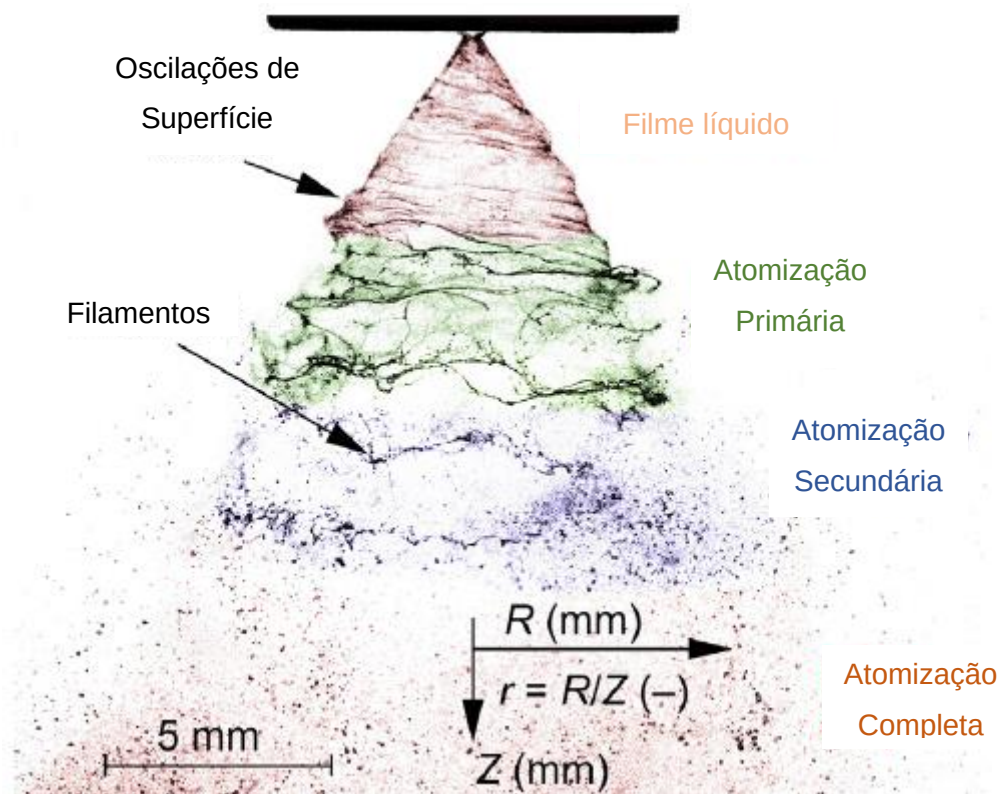
O comprimento característico possui relação com o comprimento de atomização, o qual foi estudado por autores como Moon (MOON, BAE e ABO-SERIE, 2009) e Jedelsky (JEDELSKY, MALY, *et al.*, 2018). Conceitualmente, a partir do comprimento de atomização, encontra-se a região onde espera-se que o spray apresentará aproximadamente a distribuição de tamanho de gotas previsto. Portanto, as relações que levam em conta a vaporização, mistura e posterior reação química, devem levar em conta tal dimensão.

Moon estudou a anatomia do spray de injetores centrífugos por meio de uma bancada de micro imageamento. A partir dos dados obtidos, validou-se o modelo de filme líquido desenvolvido para estimar o comprimento de atomização. Para o caso estudado, o comprimento de atomização foi extremamente curto, com 0.1 mm de distância da saída do bocal. A curta dimensão se alinha com o fato de haver-se optado pela pressão de injeção de 5 Mpa, extremamente elevada quando comparada injetores centrífugos de aplicação espacial.

No estudo desenvolvido por Jedelsky, a estrutura do spray e suas etapas de atomização no espaço são descritas detalhadamente (Figura 6). Tendo estudado o spray em pressões menos elevadas, de 0.5 a 1.5 MPa, fica evidente o impacto no comprimento de atomização, o qual gira em torno de 3 a 10 mm. Portanto, mesmo possuindo valores

adequados de distribuição de tamanho de gota, as interações dos sprays dentro da câmara de combustão podem ser drasticamente diferentes a depender dos parâmetros de operação.

Figura 6 – Estrutura do spray de um injetor centrífugo



Fonte: adaptado de (JEDELSKY, MALY, *et al.*, 2018)

2.3 Configuração e Mistura de Injetores Bipropelente

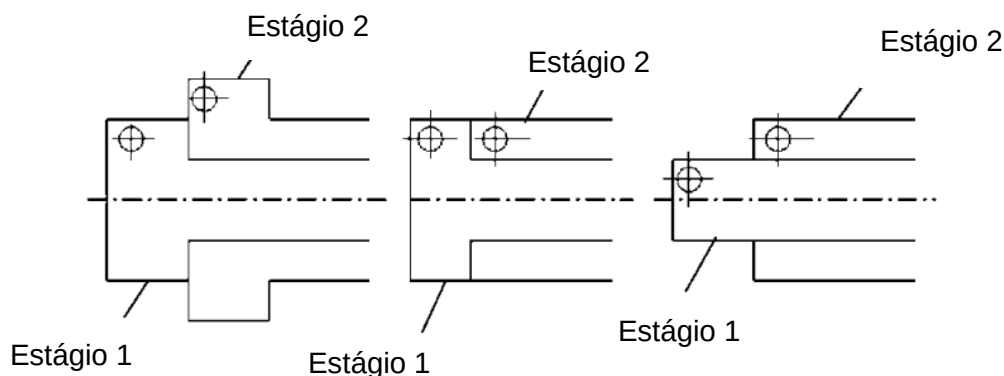
Os injetores centrífugos bipropelente podem ser de dois tipos de mistura:

- Mistura Externa: o par propelente se intercepta no exterior do injetor, de forma que o escoamento no interior de um do estágio não afeta o escoamento no outro estágio. Portanto, nesta configuração ambos os estágios permanecem hidraulicamente independentes.
- Mistura Interna: o par propelente se intercepta no interior do injetor. Uma configuração comum é a do Oxidante no 1º estágio interceptando o Combustível no bocal do 2º estágio promovendo uma mistura aprimorada. Além do impacto na modelagem da

vazão para este tipo de injetor, também é importante a atenção ao tempo de estadia na mistura no injetor que, quando excessivo, pode causar danos (ALVES, 2008).

Na Figura 7 são demonstradas algumas das configurações que podem comportar estes dois tipos de mistura.

Figura 7 - Tipos de Injetores Centrífugos bipropelentes com mistura externa, sendo em a) 1º e 2º estágios hidraulicamente independentes; b) bocal do 1º estágio submerso no 2º estágio; c) config. de dimensão reduzida com os estágios abertos.



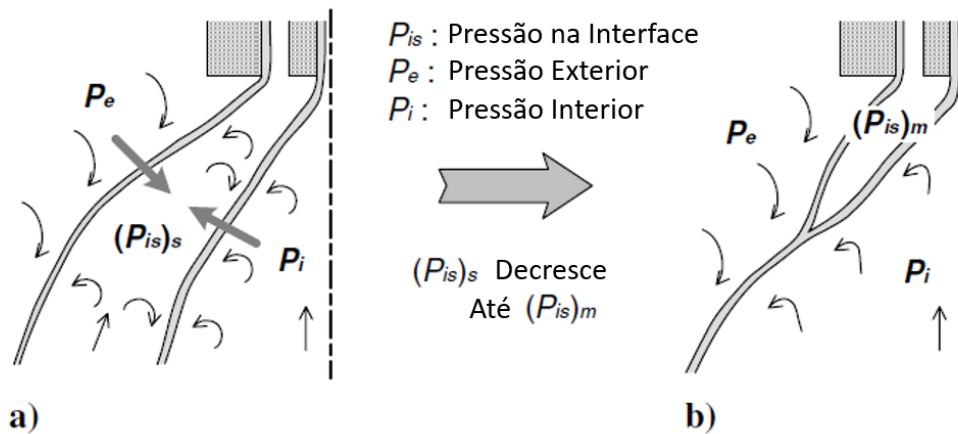
Fonte: (BAZAROV, YANG e PURI, 2004)

É sugerida por Bazarov (BAZAROV, YANG e PURI, 2004) uma diferença de ângulo de spray entre o primeiro e segundo estágios de 10 a 15 graus, de forma a garantir sua interceptação e mistura efetiva. Outras abordagens também são possíveis, como descrito por Kim (KIM, HAN, *et al.*, 2007) e representado na Figura 8, onde os dois sprays se encontram devido às características aerodinâmicas da interação entre eles.

No caso de injetores de mistura interna, a mistura não se dá apenas pelo balanço de momento entre os dois filmes líquidos, também é necessário levar em conta as condições de impingência ou colisão do spray interno no bocal do spray interno (KIM, HAN, *et al.*, 2007). O impacto gerado no interior do injetor gera oscilações com amplitude maior que a amplitude natural das oscilações das camadas líquidas, dominando o processo de atomização e favorecendo-o. No entanto, caso a amplitude seja alta o suficiente, há uma onda de impacto

que se propaga para o exterior do injetor e prejudica o processo de atomização, alongando seu comprimento.

Figura 8 - Esquemático do processo de união da a) Etapa Separada, até b) Etapa Unida devido à redução da pressão na interface.

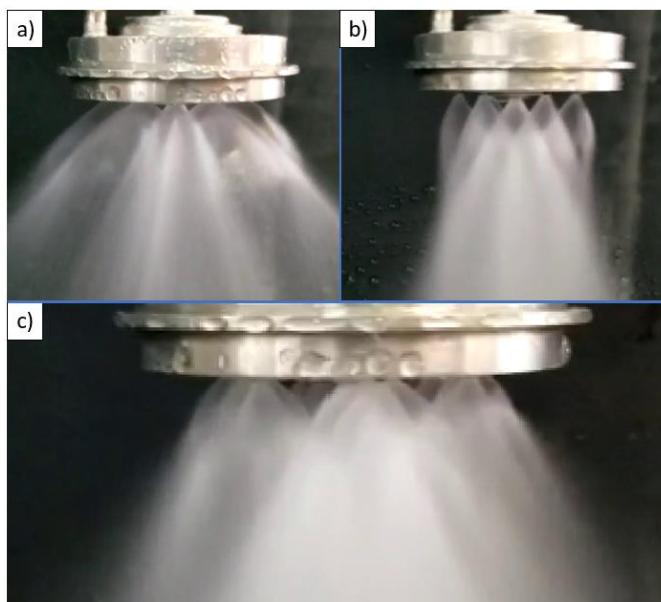


Fonte: adaptado de Kim (KIM, HAN, *et al.*, 2007).

Também são descritas por Bazarov (BAZAROV, YANG e PURI, 2004) outras formas de modificar a geometria do spray, porém, sem alterar sua vazão, por exemplo adicionando na saída do bocal uma seção inclinada ou um aumento de diâmetro da saída após o bocal original (Figura 21). Ambas as configurações promovem uma diminuição do ângulo de spray, o que pode ser usado para garantir ou facilitar a interceptação dos sprays num injetor de mistura externa.

Outra alteração da geometria da saída do injetor é possível de forma a alterar a geometria do spray e, conseqüentemente, a distribuição deste no espaço (GOULART, HINCKEL e BONTEMPO, 2020). Para a aplicação em injetores centrífugos bipropelente, esta alteração pode proporcionar um padrão alternado da razão de mistura conforme desejado no entorno do injetor. Na Figura 9 é apresentado um exemplo no qual os injetores de oxidante na periferia da placa de injeção possuem corte voltado para o centro do motor, fazendo com que haja menos oxidante direcionado para as paredes da câmara.

Figura 9 – Placa de injeção dotada de injetores com saída em corte; a) Inj. de combustível (convencionais); b) Inj. de Oxidante (corte voltado para o centro); c) Todos.



Fonte: (GOULART, HINCKEL e BONTEMPO, 2020)

3 METODOLOGIA

Para iniciar o dimensionamento dos injetores propostos, precisa-se partir dos parâmetros básicos do propulsor e da placa de injeção. Para tal, determina-se inicialmente os requisitos de alto nível do propulsor.

Possuir Relevância para o desenvolvimento nacional de propulsão:

- Uso do mesmo par propelente de outros desenvolvimentos nacionais como L5, L15 e L75 (LOx/Etanol)

Possuir especificação compatível com aplicações comerciais

- Empuxo mínimo capaz de atender os cenários a seguir:
 - Propulsor de estágio de inserção orbital de satélites e microsatélites.
 - Propulsor de *kickstage* para inserção, ajuste ou alteração da órbita de satélites e microsatélites.
- Pressão de câmara compatível com alimentação por tanques pressurizados

Possuir especificação compatível com uma infraestrutura de testes em bancada simples e de baixo custo:

- Empuxo máximo permissível que não necessite válvulas de grande porte.
- Pressão de câmara compatível com pressão máxima dos reservatórios de 70 bar.
- Pressão típica de propulsores pressure-fed (minimizar massa de tanques e pressurizante). Exemplos: L5; Aestus; FF-400N; FTBL
- Com a similaridade com o motor L5, é esperado que o motor proposto compartilhe suas características térmicas e de film-cooling. Portanto, sendo possível realizar ajustes para garantir um design similarmente confiável e durável.

É proposto um propulsor de 5 kN de empuxo, o qual se enquadraria no contexto de um propulsor para estágio superior de um veículo lançador de microsatélites ou *kickstage* para inserção ou alteração de órbita de pequenos satélites.

Também são considerados aspectos importantes a simplicidade do projeto e da operação com o propulsor proposto, dado que este pode servir também como plataforma didática para a formação de proficiência em propulsão líquida em instituições de formação em engenharia.

Tal contextualização da proposta do propulsor direciona e estabelece os requisitos do projeto conceitual deste, pautando as especificações expostas na Tabela 2. Para propulsores existem maneiras diferentes de determinar os parâmetros de operação, em especial levando em conta o sistema de alimentação e, quando presente, os sistemas de resfriamento regenerativo. No contexto do propulsor proposto, pauta-se um dimensionamento simplificado para um propulsor de bancada.

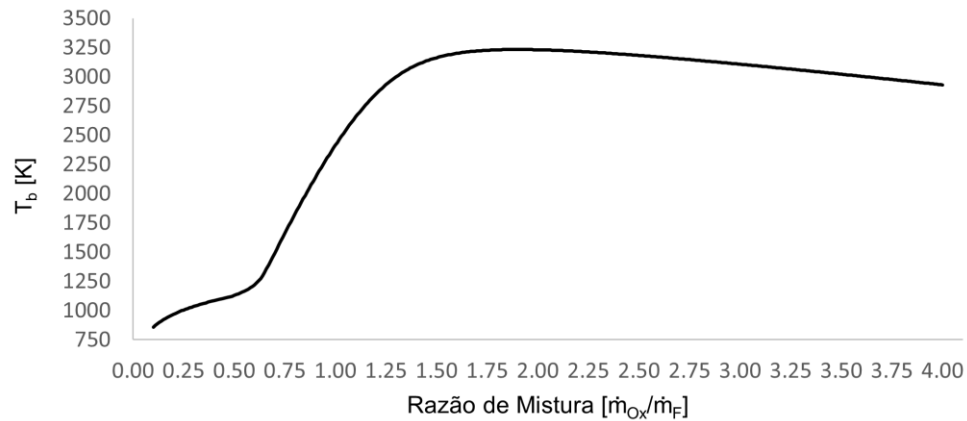
Tabela 2 - Parâmetros escolhidos para o propulsor proposto e que servirão de base para o dimensionamento dos injetores.

Parâmetros Básicos do Propulsor Proposto	
Empuxo	5000 N
Pressão de câmara	15 bar
Oxidante	Oxigênio Líquido
Combustível	Etanol 95%

Fonte: elaborado pelo autor.

Para as condições de operação estabelecidas, obtém-se a curva de Temperatura de chama apresentada na Figura 10, calculada utilizando o software Rocket Propulsion Analysis (PONOMARENKO, 2023). Identifica-se o valor de temperatura máxima na razão de mistura de aproximadamente 1.90.

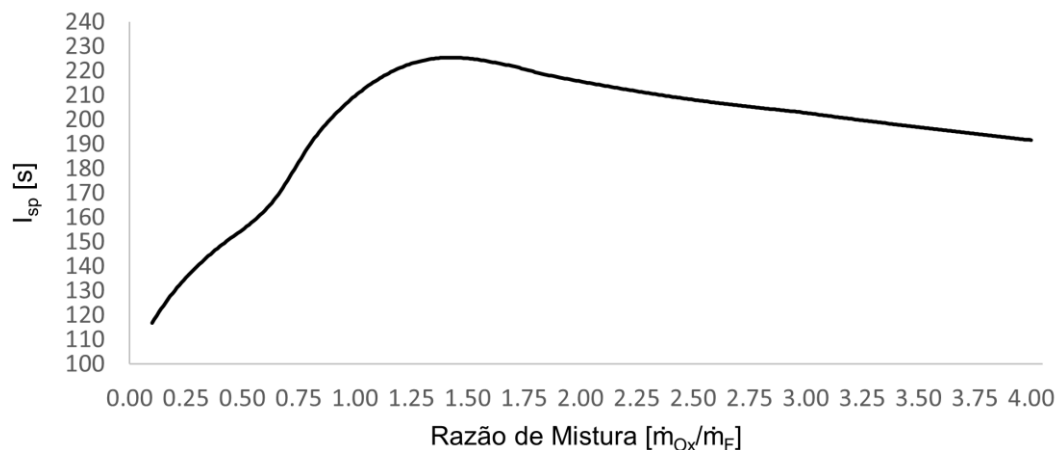
Figura 10 - Temperatura dos produtos de combustão em função da Razão de Mistura para as condições escolhidas de câmara, com pressão de câmara a 15 bar.



Fonte: elaborado pelo autor.

Como uma das ferramentas para determinar a razão de mistura média do motor, plota-se o impulso específico em função da razão de mistura na Figura 11, indicando o seu valor máximo em aproximadamente 1.40 e 225 s.

Figura 11 – Impulso Específico em função da Razão de Mistura para as condições escolhidas de câmara, com pressão de câmara a 15 bar.



Fonte: elaborado pelo autor.

Utiliza-se um fator de correção a partir de testes realizados com um propulsor semelhante da Universidade Técnica de Dresden (SIEDER, KLEEBUSCH, *et al.*, 2017). Utiliza-se o mesmo tipo de injetor, par propelente e a mesma ferramenta de análise e o valor

obtido experimental foi cerca de 9.5% abaixo do simulado, portanto, aplicando o mesmo padrão, obtemos um valor de I_{sp} de aprox. 204 s ao invés dos 225 s originalmente calculados.

Os injetores propostos serão divididos em duas categorias, Centrais e Periféricos, referente a seu posicionamento na placa de injeção,

Figura 12. Os injetores centrais apresentam razão de mistura mais pobre, para compensar a mistura rica dos injetores periféricos que realizam a proteção da parede da câmara.

Também há um maior proveito térmico da razão de mistura do oxidante, pois este apresentará temperatura mais elevada, já que para um maior impulso específico deseja-se uma maior temperatura e uma menor massa molar média. Os parâmetros escolhidos para os injetores centrais são expostos na Tabela 3.

Tabela 3 – Parâmetros dos injetores centrais propostos.

Parâmetros dos Injetores Centrais	
MR	2.00
Modificação	Bocal de diâmetro ampliado
Propelente ST1	LOx
ΔP_{ST1}	6.00 Bar
$\dot{m}_{ST1}$	190.3 g/s
$2\alpha_{ST1}$	120 Graus
Recesso ST1	2.00 mm
t_w	1.00 mm
Propelente ST2	Etanol 95%
ΔP_{ST2}	4.00 Bar
$\dot{m}_{ST2}$	95.1 g/s
$2\alpha_{ST2}$	105 graus

Fonte: elaborado pelo autor.

Os parâmetros para o dimensionamento dos injetores periféricos são expostos na Tabela 4. Neste fica evidente a diferença nos ângulos de spray de forma ao spray dos dois

estágios não se interceptarem, ressaltando a função principal de proteção da parede da câmara com excesso de combustível.

Tabela 4 - Parâmetros dos injetores periféricos propostos.

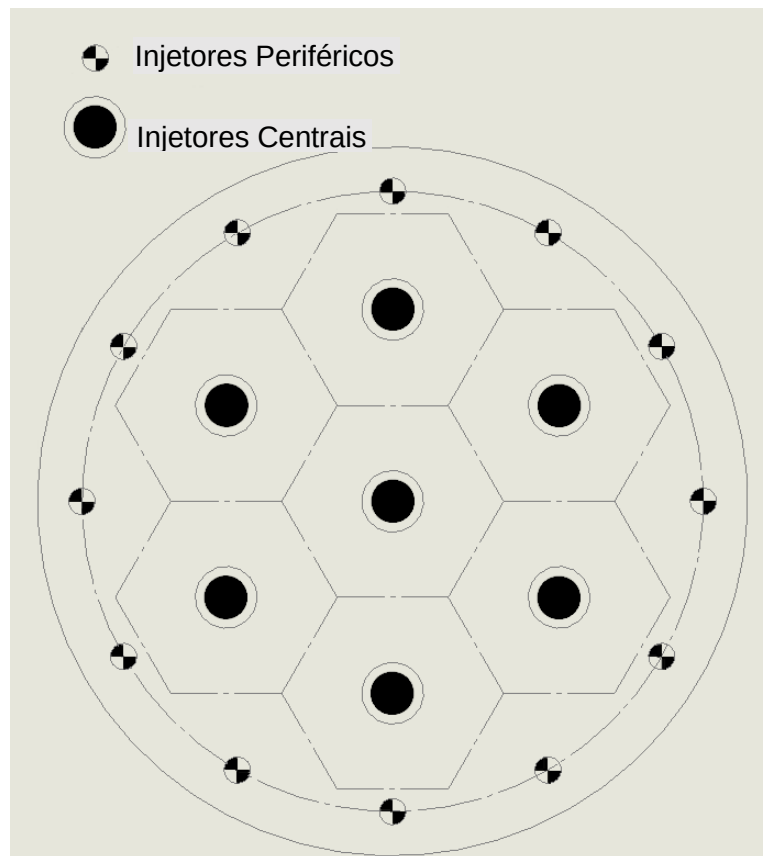
Parâmetros dos Injetores Periféricos	
MR	1.00
Modificação	Bocal com corte inclinado
Propelente ST1	LOx
ΔP_{ST1}	6.00 Bar
$\dot{m}_{ST1}$	20.8 g/s
$2\alpha_{ST1}$	95 Graus
Recesso ST1	1.00 mm
t_w	1.00 mm
Propelente ST2	Etanol 95%
ΔP_{ST2}	4.00 Bar
$\dot{m}_{ST2}$	20.8 g/s
$2\alpha_{ST2}$	120 graus

Fonte: elaborado pelo autor.

Na Tabela 4 nota-se a baixa vazão dos injetores periféricos, o que pode ser explicado pela sua distribuição na placa de injeção, exposta na

Figura 12. Para garantir uma cobertura completa e homogênea da parede da câmara de combustão foram utilizados o máximo de elementos injetores possíveis de acordo com o limite inferior da vazão mássica habitualmente utilizado, citado por (KHAVKIN, 2004), de aproximadamente 20 g/s. Alguns aspectos podem contribuir para esse limite inferior quando se tratando de combustíveis convencionais; com a menor dimensão do injetor, efeitos viscosos tendem a aumentar a vazão com uma mais elevada perda de velocidade tangencial e a compensação deste efeitos, em geral, implica em dimensões excessivamente pequenas para os métodos e tolerâncias de fabricação disponíveis.

Figura 12 - Placa de Injeção Proposta para definição dos parâmetros básicos dos injetores, com exibindo a divisão entre injetores “Centrais” e “Periféricos”.



Fonte: Elaborado pelo Autor.

3.1 Cálculos de Injetores Centrífugos

O cálculo de injetores centrífugos gira em torno de algumas propriedades fundamentais que, a partir destas, todos os outros parâmetros são calculados. São eles o Coeficiente de Preenchimento, ϕ , definido como a fração da área transversal livre no centro do bocal, pela área transversal total do bocal e o Parâmetro Geométrico do injetor, A , o qual é descrito na Tabela 5 para o caso ideal e na

Tabela 6 para o caso viscoso.

3.1.1 Cálculo do Injetor Ideal

No equacionamento do injetor ideal, não é considerada a viscosidade do fluido. Portanto, não são contabilizadas as perdas na entrada e no interior dos canais tangenciais e

a perda de momento angular no interior do injetor. Na Tabela 5 são descritas as principais equações utilizadas nos algoritmos de dimensionamento desenvolvidos na seção em que é calculado o injetor ideal.

Tabela 5 - Equações para o Cálculo do Injetor Ideal.

Símbolo	Variável	Equação
A_i	Parâmetro Geométrico	$A_i = \frac{R_{in} \cdot R_n}{n \cdot r_{in}^2}$
ϕ_i	Coeficiente de Preenchimento (a partir de 2α)	$\phi_i = \frac{2}{2 + \left(\tan\left(\frac{2\alpha}{2}\right)\right)^2}$
ϕ_i	Coeficiente de Preenchimento (a partir de A)	Raiz do Polinômio: $A_i^2 \cdot \phi_i^3 - 2 \cdot \phi_i^2 + 4 \cdot \phi_i - 2$
μ_i	Coeficiente de Descarga	$\mu_i = \phi_i \cdot \sqrt{\frac{\phi_i}{2 - \phi_i}}$
$2\alpha_i$	Ângulo de Abertura do Spray	$2\alpha_i = 2 \cdot \text{asin}\left(\frac{2 \cdot \mu_i \cdot A_i}{1 + \sqrt{1 - \phi_i}}\right)$

Fonte: Baseado no desenvolvimento exposto por Bazarov (BAZAROV, YANG e PURI, 2004)

3.1.2 Cálculo do Injetor Real

No equacionamento do injetor real, em que é considerada a viscosidade do fluido, existem dois efeitos de perda com impactos contraditórios. A perda viscosa na entrada e no interior dos canais tangenciais influi na diminuição da vazão do injetor, enquanto a perda de momento angular na cavidade principal do injetor atua aumentando a vazão. Tal cenário cria uma grande diversidade de possibilidades no dimensionamento para as quais é necessária atenção. Na Tabela 6 são descritas as principais equações utilizadas nos algoritmos de dimensionamento desenvolvidos na seção em que é calculado o injetor real, contabilizando o efeito viscoso.

Tabela 6 - Equações para o Cálculo do Injetor Real.

Símbolo	Variável	Equação
λ	Coeficiente de Atrito	$\lambda = 0.3164 \cdot \text{Re}^{-0.25}$
α_{in}	Ângulo de entrada do canal tangencial	$\alpha_{\text{in}} = 90^\circ - \arctan\left(\frac{R_s}{l_{\text{in}}}\right)$
ξ_{in}	Coeficiente de perda hidráulica da entrada dos canais tangenciais	$\xi_{\text{in}} = -\frac{\alpha_{\text{in}}}{150} + 1$
ξ	Coeficiente de perda hidráulica total dos canais tangenciais	$\xi = \xi_{\text{in}} + \lambda \cdot \frac{l_{\text{in}}}{2 \cdot r_{\text{in}}}$
A_r	Parâmetro Geométrico	$A_r = A \cdot K = \frac{R_{\text{in}} \cdot R_n}{n \cdot r_{\text{in}}^2 + \frac{\lambda}{2} \cdot R_{\text{in}} \cdot (R_{\text{in}} - R_n)}$
ϕ_r	Coeficiente de Preenchimento (a partir de A)	Raiz do Polinômio: $A_r^2 \cdot \phi_r^3 - 2 \cdot \phi_r^2 + 4 \cdot \phi_r - 2$
μ_r	Coeficiente de Descarga	$\mu_r = \frac{\phi_r \cdot \sqrt{\frac{\phi_r}{2 - \phi_r}}}{\sqrt{1 + \xi \cdot \mu_{\text{eq}}^2 \cdot \left(\frac{R_{\text{in}}}{C}\right)^2}}$
$2\alpha_r$	Ângulo de Abertura do Spray	$2\alpha_r = 2 \cdot \arcsen\left(\frac{2 \cdot \mu \cdot A}{(1 + \sqrt{1 - \phi}) \cdot (1 - \xi \cdot \mu^2 \cdot \left(\frac{A}{C}\right)^2)}\right)$

Fonte: Baseado em Bazarov (BAZAROV, YANG e PURI, 2004)

Alguns aspectos como a atenção ao valor da perda de momento angular, K, sempre menor que 1 e uma razão de área de entrada e saída do injetor também sempre menor que 1 são aspectos importantes para checar a validade do dimensionamento.

3.1.3 Cálculo do Injetor modulado – Saída Expandida

De acordo com Bazarov (BAZAROV, YANG e PURI, 2004) são possíveis modificações de forma a alterar de maneira independente o coeficiente de descarga do injetor e se ângulo do cone de spray. Para tal, um dos métodos consiste na adição de uma seção divergente após o bocal do injetor onde em princípio o fluido alcançaria sua condição crítica. Portanto, a alteração a jusante do bocal não interfere na vazão do injetor.

No caso do bocal expandido, essa seção divergente não termina em ângulo, mas fecha novamente numa seção cilíndrica com diâmetro superior ao do bocal original. Com este aumento de diâmetro é possível reduzir o ângulo de cone, pois com o maior diâmetro parte da velocidade tangencial é transformada em velocidade axial, efetivamente alterando o ângulo de saída do fluido, portanto, obtendo-se a Equação (3.1) de Meio-Ângulo de abertura do spray modulado.

$$\tan \alpha = \sqrt{\frac{a}{r_1^2 - a}} \quad (3.1)$$

A partir desta é possível adicionar o recurso de ajuste de spray de cone conforme o desejado. O exemplo implementado no dimensionamento deste projeto é apresentado na Figura 20. O dimensionamento do bocal com saída aumentada foi implementado neste trabalho no algoritmo de dimensionamento M1. Neste, um ângulo de interceptação entre estágios é determinado, e o software provê a dimensão necessária para atendê-lo.

Há um limite que deve ser obedecido, no qual a velocidade axial do escoamento na porção expandida não pode superar a velocidade crítica do escoamento no bocal original como mostrado na Equação (3.2) (as equações no formato mostrado utilizam o efeito Skobelkin, portanto só são válidas para bocais expandidos com saída cilíndrica).

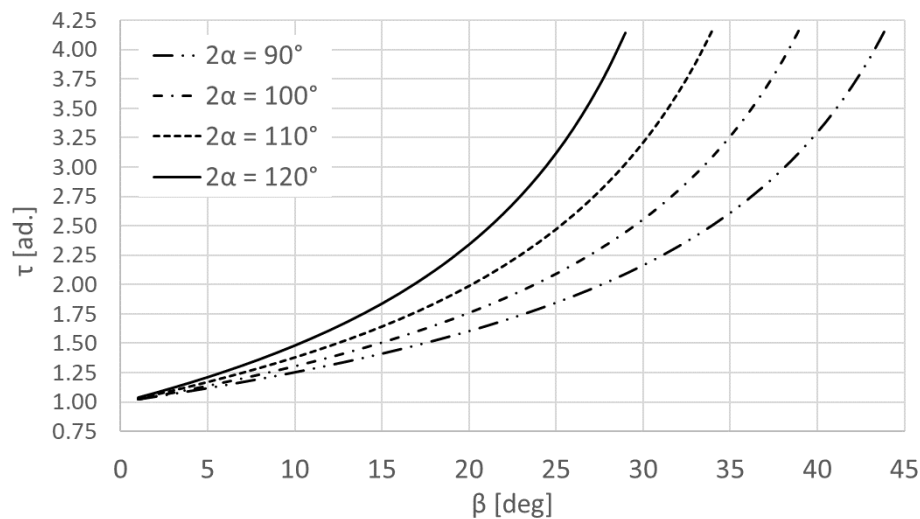
$$\lambda_i = \frac{\text{velocidade axial no bocal modulado}}{\text{velocidade axial no bocal original}} = \frac{U_L}{U_{th}} = \sqrt{3 - 2\phi}; \quad \lambda_{i,\max} = \sqrt{\frac{2 - \phi}{\phi}} \quad (3.2)$$

3.1.4 Cálculo do Injetor modulado – Saída em Corte

A aplicação mais prevalente desta configuração de saída em corte possui o objetivo de direcionar mais combustível para proteção da parede da câmara, direcionando maior quantidade de oxidante para o centro da câmara. Tal configuração é apresentada na Figura 22 e na Figura 13.

O Padrão de alteração da distribuição de vazão mássica do injetor para um corte simples segue as curvas apresentadas na Figura 13, calculadas de acordo com a modelagem de Hinckel (HINCKEL, GOULART e BONTEMPO, 2019).

Figura 13 – Razão de vazão mássica entre as seções de maior e menor vazão do injetor em função de ângulo de corte para valores de ângulo de abertura do spray do injetor original.



Fonte: (GOULART, HINCKEL e BONTEMPO, 2020)

3.1.5 Algoritmos de dimensionamento

Os algoritmos orientados a objeto deste trabalho foram desenvolvidos partindo de um fluxo de cálculo e dimensionamento que trata dos principais desafios no projeto dos injetores, exposto na Figura 14. Inicialmente temos o Algoritmo de Dimensionamento M1, o qual fornece uma geometria inicial para o injetor se baseando apenas em parâmetros básicos do injetor, obtidos a partir das primeiras concepções do propulsor e placa de injeção. Após obter uma geometria inicial, utiliza-se o Algoritmo de Ajustes M2 para realizar pequenas mudanças na geometria gerada, dessa forma adequando os valores para o projeto e fabricação, além de prover a curva de Vazão por Pressão para o injetor em questão.

O algoritmo M3, descrito no fluxograma da Figura 13, não foi implementado, mas faz parte do fluxo de projeto definido inicialmente, pois este seria responsável por determinar um envelope de operação do injetor ($\Delta P \times \dot{m}$) considerando as tolerâncias e dispersões de fabricação para o método escolhido em cada projeto.

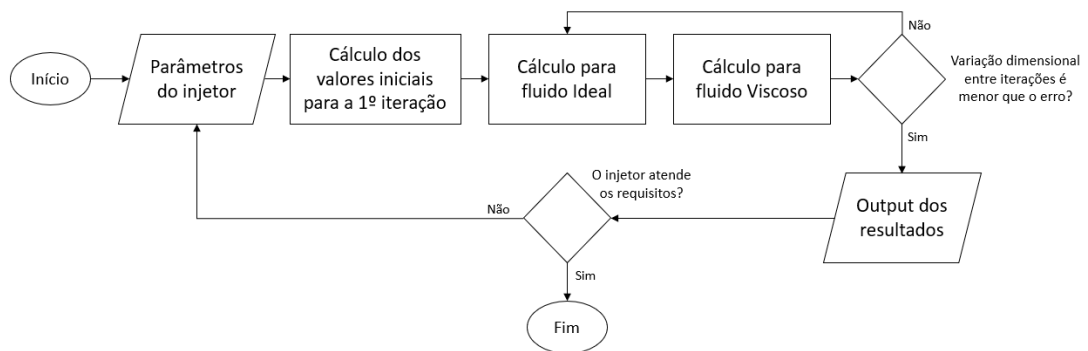
Figura 14 - Fluxograma de Dimensionamento com os métodos desenvolvidos.



Fonte: elaborado pelo autor.

O fluxo de cálculo do Algoritmo M1 é apresentado na Figura 15. Seu critério de parada é baseado num erro geométrico permissível, de forma que, dado que a iteração acrescenta variação geométrica que tendendo a zero, têm-se o cálculo completo.

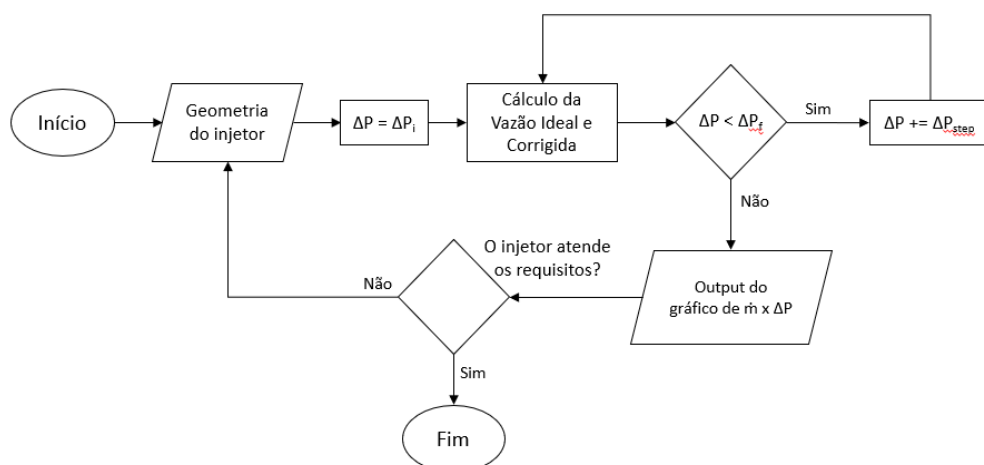
Figura 15 - Fluxograma do algoritmo de dimensionamento M1.



Fonte: elaborado pelo autor.

Para o algoritmo M2, cujo fluxo é demonstrado na Figura 16, o loop de cálculo é acrescido da variação do diferencial de pressão no intervalo de interesse. Dessa forma, para cada valor de diferencial de pressão há um processo iterativo similar ao M1, porém no sentido inverso, calculando a vazão a partir de uma geometria definida.

Figura 16 - Fluxograma do algoritmo de ajustes M2.



Fonte: elaborado pelo autor.

4 RESULTADOS E DISCUSSÃO

Os propulsores utilizados para validação do dimensionamento são referências clássicas no desenvolvimento de injetores centrífugos bipropelente. Na Tabela 7 são apresentados os dados básicos do RD0110, tendo sua principal aplicação nos veículos da família Soyuz e Molniya. Os dados básicos do RD0109 são apresentados na Tabela 8.

Tabela 7 - Parâmetros básicos do propulsor de referência RD0110.

RD0110	
Empuxo (Vácuo)	297.90 kN/ 4 x 74.48 kN
Pressão de Câmara	68.20 bar
Par Propelente	LOx/RG-1
Razão de Mistura	2.22
Impulso Específico	326.0 s
Razão de Expansão	82.20
Fonte: (RUBINSKY, 1994)	

Tabela 8 - Parâmetros básicos do propulsor de referência RD0109.

RD0109	
Empuxo (Vácuo)	54.52 kN
Pressão de Câmara	50.00 bar
Par Propelente	LOx/RG-1
Razão de Mistura	2.14
Impulso Específico	323.5 s
Razão de Expansão	79.40
Fonte: (ALVES, 2008)	

O RD0110 teve um grande papel no estudo das instabilidades de combustão, como descrito por Rubinsky (RUBINSKY, 1994), e o RD0109 foi base para a criação do propulsor brasileiro L75 (ALVES, 2008). O RD0110 possui quatro câmaras de empuxo compondo o propulsor, enquanto o RD0109 apresenta apenas uma câmara, apresentando menor empuxo e sendo utilizado em estágios superiores, no vácuo.

4.1 Resultados do Algoritmo de Dimensionamento M1

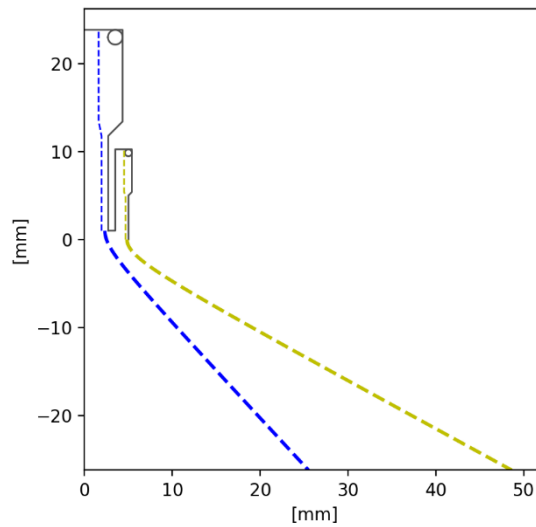
O algoritmo de dimensionamento M1 gera a geometria do injetor a partir dos seus parâmetros básicos como fluido de trabalho, diferencial de pressão e razões de proporção geométrica. No caso de injetores bipropelente também espessura da parede entre estágios e a distância de recessão do bocal do 1º estágio. Como resultado obtêm-se os valores geométricos do injetor e a geometria esperada para o spray de ambos os estágios. Os resultados para os injetores de referência e propostos são expostos a seguir.

4.1.1 Resultados do M1 para o RD0110

A geometria e propriedades do Injetor calculado para o RD0110 utilizando os dados básicos do motor e injeção é apresentado na Figura 17 e seus valores na Tabela 9 para os valores do 1º estágio de Oxidante e na Tabela 10 para o 2º estágio de Combustível.

Em especial se chama a atenção na Tabela 10 sobre a divergência entre o valor declarado de parâmetro geométrico em comparação com o valor calculado a partir das próprias dimensões da mesma referência. Não é claro o porquê desta diferença, portanto são expostos os dois valores.

Figura 17 - Injetor calculado com os dados do propulsor RD0110.



Fonte: elaborado pelo autor.

Uma das limitações do método consiste em não poder calcular valores de Razão de abertura do injetor menores que 1. No entanto, de acordo com Bazarov (BAZAROV, YANG e PURI, 2004), estes podem ser aproximados utilizando um coeficiente de abertura igual a 1, apesar de proverem uma geometria diferente da que é exatamente proposta. Tal divergência é visível na Tabela 9 nos valores do Raio de entrada e do Raio da câmara de vórtice. As dimensões calculadas para o estágio de combustível foram extremamente próximas dos valores de referência, com exceção do valor do Raio de Entrada e Raio da Câmara de vórtice. Estes dois valores diferem, pois é utilizada a aproximação citada que iguala o comportamento de um injetor em que o coeficiente de abertura $C = R_{in}/R_n = 1.00$ com um injetor aberto, $R_s=R_n$.

Uma das variáveis mais sensíveis a variações dimensionais é o raio do canal de entrada, tendo apresentado valor extremamente similar à referência de validação para o cálculo realizado dos dois estágios.

Tabela 9 – Tabela de comparação de geometria e propriedades originais e calculadas para o injetor de oxidante do RD0110.

Geometria e Propriedades		Valores Reais (RUBINSKY, 1994)	Valores Calculados (Calc. pelo autor)	Unidades
Raio do bocal	R_n	2.70	2.695	mm
Raio de entrada	R_{in}	3.50	3.504	mm
Raio da câmara de vórtice	R_s	4.50	4.342	mm
Raio do canal de entrada	r_{in}	0.85	0.838	mm
Número de canais	n_{in}	6	6	un.
Comp. do canal de entrada	l_{in}	1.70	1.675	mm
Comprimento do bocal	L_n	10.55	10.782	mm
Comp. da câmara de vórtice	L_s	11.00	10.420	mm
Ângulo do Spray	2α	80.0	85.98	graus
Vazão mássica	$\dot{m}$	172.9	-	g/s
Diferencial de Pressão	ΔP	4.26	-	bar
Parâm. Geom. Característico	A_i/A_{eq}	2.0 / -	2.2435 / 2.2377	-
Coeficiente de preenchimento	φ_i/φ_{eq}	- / -	0.4771 / 0.4777	-
Coef. de perda hidráulica	ξ	-	0.9670	-
Não-Uniformidade	I	-	2.32%	-

Fonte: elaborado pelo autor.

Tabela 10 - Tabela de comparação de geometria e propriedades originais e calculadas para o injetor de combustível do RD0110.

Geometria e Propriedades		Valores Reais	Valores Calculados	Unidades
		(RUBINSKY, 1994)	(Calc. pelo autor)	
Raio do bocal	R_n	5.00	5.002	mm
Raio de entrada	R_{in}	4.50	5.002	mm
Raio da câmara de vórtice	R_s	5.00	5.397	mm
Raio do canal de entrada	r_{in}	0.35	0.396	mm
Número de canais	n_{in}	6	6	un.
Comp. do canal de entrada	l_{in}	1.05	1.187	mm
Comprimento do bocal	L_n	-	5.002	mm
Comp. da câmara de vórtice	L_s	-	4.857	mm
Ângulo do Spray	2α	135	123.06	graus
Vazão mássica	$\dot{m}$	64.8	-	g/s
Diferencial de Pressão	ΔP	6.96	-	bar
Parâm. Geom. Característico	A_i/A_{eq}	24.5 (30.6) * / -	26.6282 / 26.6282	-
Coeficiente de preenchimento	φ_i/φ_{eq}	- / -	0.1289 / 0.1289	-
Coef. de perda hidráulica	ξ	-	1.0660	-
Não-Uniformidade	I	-	23.06%	-

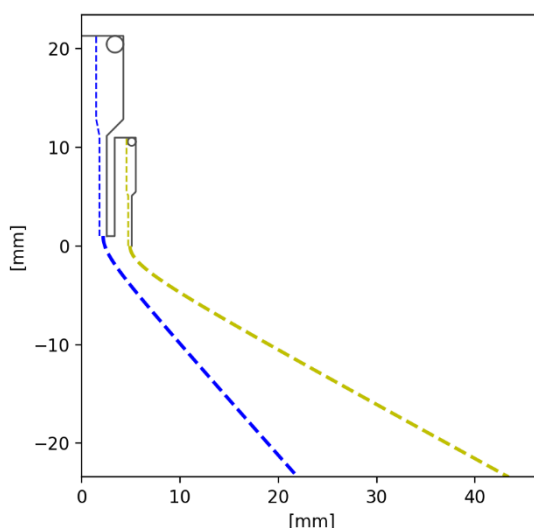
*: Valor fornecido pela referência (Valor calculado pelo autor)

Fonte: elaborado pelo autor.

4.1.2 Resultados do M1 para o RD0109

Assim como realizado para o RD0110, utiliza-se um processo análogo para calcular as dimensões do injetor do RD0109 utilizando o algoritmo M1. Usa-se inputs como propriedades dos propelentes, diferencial de pressão desejado e vazão desejada. Além disso são necessárias razões geométricas adimensionais, espessura da parede e distância de recessão do bocal do 1º estágio. A geometria do injetor e o spray calculados são apresentados na Figura 18.

Figura 18 - Injetor calculado com os dados do propulsor RD0109.



Fonte: elaborado pelo autor.

Para o injetor de oxidante do RD0109, nota-se pelos resultados da Tabela 11 que a divergência entre o valor calculado e o valor de referência foi menor que no caso do injetor de oxidante do RD0110. Para o estágio de combustível do RD0109, o erro do valor calculado para o raio de entrada do canal foi da ordem da tolerância dimensional tipicamente utilizada para este tipo de peça, portanto, novamente apresentando um resultado mais certo para o modelo de cálculo em comparação com a computação do caso do RD0110. Além disso, observa-se na Tabela 12 que outras dimensões importantes como o Raio do Bocal foram praticamente idênticas.

Tabela 11 - Tabela de comparação de geometria e propriedades originais e calculadas para o injetor de oxidante do RD0109.

Geometria e Propriedades		Valores Reais (ALVES, 2008)	Valores Calculados (Calc. pelo autor)	Unidades
Raio do bocal	R_n	2.70	2.542	mm
Raio de entrada	R_{in}	3.60	3.380	mm
Raio da câmara de vórtice	R_s	4.50	4.226	mm
Raio do canal de entrada	r_{in}	0.90	0.846	mm
Número de canais	n_{in}	6	6	un.
Comp. do canal de entrada	l_{in}	5.40	5.074	mm
Comprimento do bocal	L_n	-	10.167	mm
Comp. da câmara de vórtice	L_s	-	8.452	mm
Ângulo do Spray	2α	-	83.62	graus
Vazão mássica	$\dot{m}$	172.9	-	g/s
Diferencial de Pressão	ΔP	4.4	-	bar
Parâm. Geom. Característico	A_i/A_{eq}	2.0 / -	2.0023 / 1.9972	-
Coeficiente de preenchimento	φ_i/φ_{eq}	- / -	0.4998 / 0.5003	-
Coef. de perda hidráulica	ξ	-	0.7884	-
Não-Uniformidade	I	-	2.13%	-

Fonte: elaborado pelo autor.

Tabela 12 - Tabela de comparação de geometria e propriedades originais e calculadas para o injetor de combustível do RD0109.

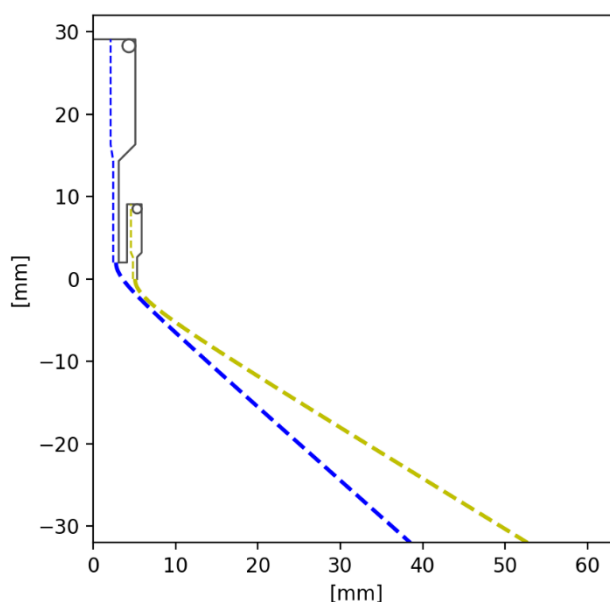
Geometria e Propriedades		Valores Reais (ALVES, 2008)	Valores Calculados (Calc. pelo autor)	Unidades
Raio do bocal	R_n	5.10	5.079	mm
Raio de entrada	R_{in}	4.70	5.079	mm
Raio da câmara de vórtice	R_s	5.10	5.493	mm
Raio do canal de entrada	r_{in}	0.40	0.415	mm
Número de canais	n_{in}	6	6	un.
Comp. do canal de entrada	l_{in}	2.40	2.488	mm
Comprimento do bocal	L_n	-	5.079	mm
Comp. da câmara de vórtice	L_s	-	5.493	mm
Ângulo do Spray	2α	-	122.97	graus
Vazão mássica	$\dot{m}$	72.2	-	g/s
Diferencial de Pressão	ΔP	7.1	-	bar
Parâm. Geom. Característico	A_i/A_{eq}	27.0 / -	25.0076 / 25.0076	-
Coeficiente de preenchimento	ϕ_i/ϕ_{eq}	- / -	0.1339 / 0.1339	-
Coef. de perda hidráulica	ξ	-	1.0334	-
Não-Uniformidade	I	-	21.77%	-

Fonte: elaborado pelo autor.

4.1.3 Resultados do M1 para o Propulsor Proposto – Injetor Central

A partir da Figura 19 fica evidente a disparidade de vazão mássica dada pela alta razão de mistura deste injetor e o desafio de manter a independência hidráulica do estágio 2. O modelo com modulação do spray é apresentado na Figura 20 e Figura 21.

Figura 19 - Injetor central do propulsor proposto.



Fonte: elaborado pelo autor.

Para modular o injetor central, foi utilizado o aumento de diâmetro do bocal do 2º estágio para diminuição do ângulo do spray, garantindo uma melhor mistura. No caso, foi utilizado uma diferença de ângulos de spray conservadora, de apenas 0°, pois um maior diâmetro de saída pode disromper o escoamento de acordo com o limite exposto por Bazarov (BAZAROV, YANG e PURI, 2004).

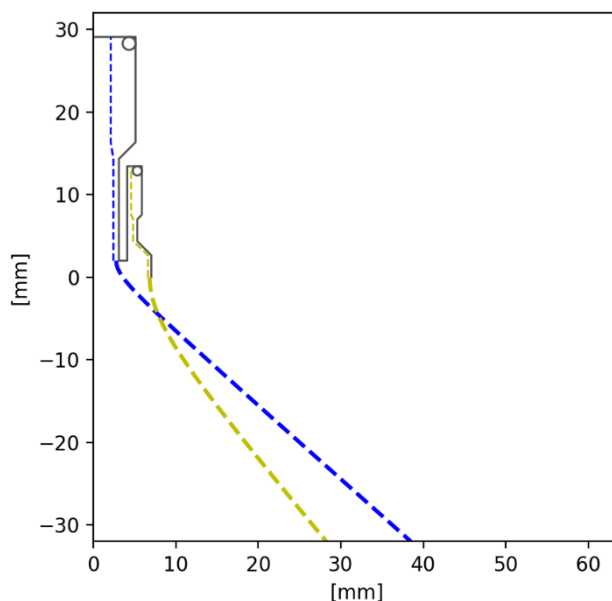
O Injetor com modulação de bocal expandido já é calculado e plotado automaticamente pelo algoritmo de dimensionamento M1. O Resultado modulado obtido pode ser conferido na e todos os valores geométricos e de propriedades na Tabela 13.

Tabela 13 - Geometria e Propriedades calculadas para o injetor central proposto.

Geometria e Propriedades		ST1 - Oxidante	ST2 - Combustível	Unidades
Raio do bocal	R_n	3.081	5.308	mm
Raio de entrada	R_{in}	4.314	5.308	mm
Raio da câmara de vórtice	R_s	5.097	5.865	mm
Raio do canal de entrada	r_{in}	0.784	0.557	mm
Número de canais	n_{in}	6	6	un.
Comp. do canal de entrada	l_{in}	1.567	1.114	mm
Comprimento do bocal	L_n	12.325	2.654	mm
Comp. da câmara de vórtice	L_s	12.743	5.865	mm
Ângulo do Spray	2α	96.97	117.27	graus
Vazão mássica	$\dot{m}$	190.30	95.10	g/s
Diferencial de Pressão	ΔP	6.00	4.00	bar
Parâm. Geom. Característico	A_i/A_r	3.6079 / 3.5303	15.1292 / 15.1292	-
Coeficiente de preenchimento	ϕ_i/ϕ_r	0.3866 / 0.3906	0.1804 / 0.1804	-
Coef. de perda hidráulica	ξ	1.0165	1.0562	-
Não-Uniformidade	I	2.36%	15.68%	-

Fonte: elaborado pelo autor.

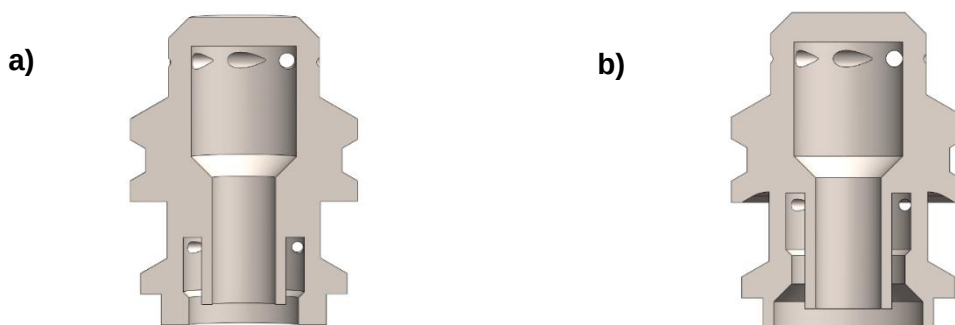
Figura 20 - Injetor central do propulsor proposto com bocal modulado.



Fonte: elaborado pelo autor.

Os injetores modelados em CAD e já ajustados para sua fabricação em manufatura aditiva por fusão a laser em mesa de pó são apresentados na Figura 20. Catálogos técnicos de fornecedores e referências como os trabalhos de Soller et. All (SOLLER, BEHR, *et al.*, 2017), Gradl (GRADL, MIRELES e ANDREWS, 2020) e Kang (KANG, LEE, *et al.*, 2022) foram essenciais para adequar este dimensionamento para impressão 3D.

Figura 21 - Injetor Central a) sem modificação b) com bocal expandido para ajuste do ângulo do spray.



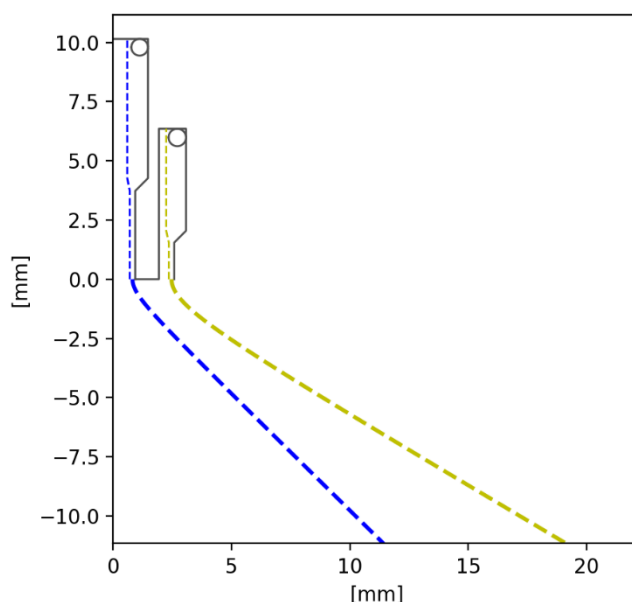
Fonte: elaborado pelo autor.

4.1.4 Dados do M2 para o Propulsor Proposto – Injetor Periférico

O desafio que se apresenta no injetor periférico é o oposto do injetor central. Dada a baixa razão de mistura, deve-se atentar às perdas no estágio 1, que necessita de uma razão de aspecto alongada que pode propiciar uma maior perda de momento angular e dificultar a obtenção da baixa vazão desejada. Também é um desafio para vazões tão baixas quanto as especificadas para este injetor, manter o diâmetro do estágio 2 grande o suficiente para garantir sua independência hidráulica.

O injetor obtido é apresentado na Figura 22 e Tabela 14. Os injetores modelados em CAD são apresentados na Figura 23.

Figura 22 - Injetor periférico do propulsor proposto.



Fonte: elaborado pelo autor.

Para modular o injetor periférico, é proposta uma saída em ângulo no 1º estágio, de oxidante, para direcionar a vazão mássica de oxidante majoritariamente para longe da parede da câmara de combustão. Como demonstrado por Hinckel (GOULART, HINCKEL e BONTEMPO, 2020), considerando um corte com ângulo de 30 graus, têm-se que uma razão de aproximadamente 2 para 1 de vazão de oxidante voltada para o centro da câmara de combustão.

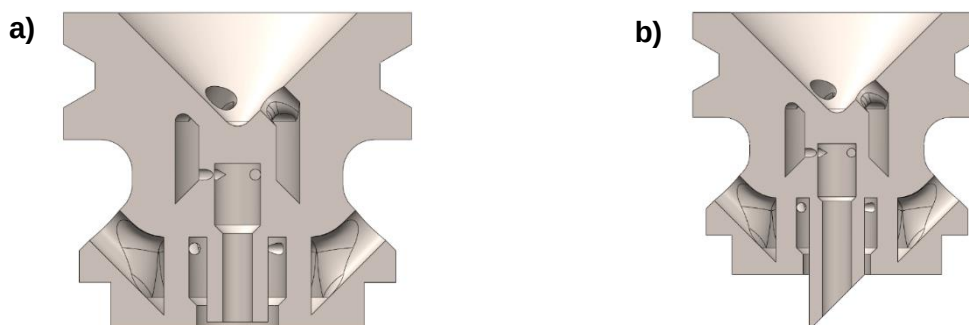
Desta forma há uma razão de mistura média voltada para o centro da câmara de aproximadamente 1.33, enquanto na porção voltada para a parede, a razão de mistura média é de 0.66, implicando numa temperatura de chama de aproximadamente 1350 K ou 1077 °C. Esta configuração é apresentada na Figura 23.

Tabela 14 – Geometria e Propriedades calculadas para o injetor periférico proposto.

Geometria e Propriedades		ST1 - Oxidante	ST2 - Combustível	Unidades
Raio do bocal	R_n	0.932	2.580	mm
Raio de entrada	R_{in}	1.118	2.709	mm
Raio da câmara de vórtice	R_s	1.470	3.077	mm
Raio do canal de entrada	r_{in}	0.352	0.369	mm
Número de canais	n_{in}	3	3	un.
Comp. do canal de entrada	l_{in}	2.813	1.475	mm
Comprimento do bocal	L_n	3.727	1.548	mm
Comp. da câmara de vórtice	L_s	5.879	4.308	mm
Ângulo do Spray	2α	91.18	119.09	graus
Vazão mássica	$\dot{m}$	20.80	20.80	g/s
Diferencial de Pressão	ΔP	6.00	4.00	bar
Parâm. Geom. Característico	A_i/A_{eq}	2.8098 / 2.7813	17.1387 / 16.9145	-
Coefficiente de preenchimento	ϕ_i/ϕ_{eq}	0.4333 / 0.4353	0.1677 / 0.1690	-
Coef. de perda hidráulica	ξ	0.8297	0.9912	-
Não-Uniformidade	I	7.48%	35.54%	-

Fonte: elaborado pelo autor.

Figura 23 - Injetor Periférico a) sem modificação b) com bocal em corte.



Fonte: elaborado pelo autor.

4.2 Resultados do Algoritmo de Ajustes M2

A partir do Método M2, obtemos valores de vazão mássica em função do diferencial de pressão aplicado ao injetor. Desta forma têm-se uma expectativa da característica de funcionamento dos injetores ao longo de uma ampla gama de cenários. Também servindo de referência de comparação com resultados experimentais, ajudando na análise e compreensão de divergências.

É esperado que o valor nominal de operação do injetor encontrado na literatura, se encontre entre as duas curvas calculadas, sendo elas a de vazão com fluido ideal e a de vazão com fluido viscoso. Utilizando ambas as curvas, há uma comparação clara entre a influência das perdas hidráulicas de entrada, que reduzem a vazão e as perdas de momento angular que aumentam a vazão do injetor. Estes dois fatores são as principais distinções entre as duas curvas.

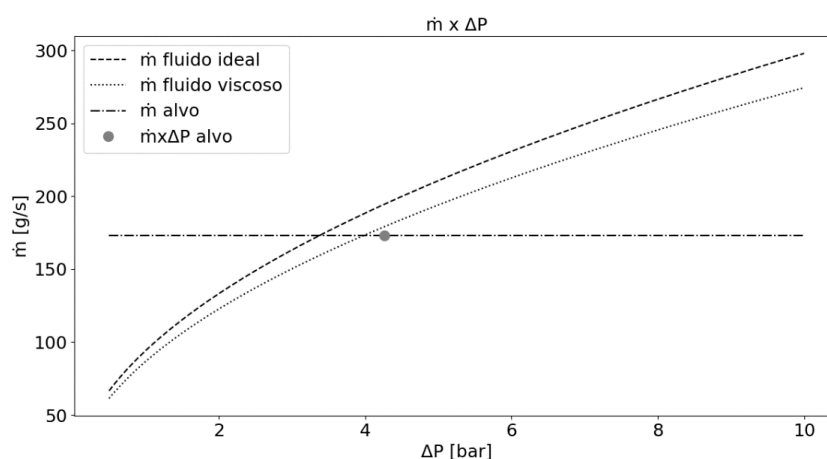
4.2.1 Resultados do M2 para o RD0110

A vazão nominal para o 1º estágio do injetor apresentada por Rubinsky (RUBINSKY, 1994) demonstrou-se na Figura 24 ligeiramente inferior às vazões calculadas. Tal comportamento indica que pode haver um superdimensionamento da perda de momento

angular ou subdimensionamento das perdas nos canais de entrada para as condições geométricas e de operação deste injetor.

Para o 2º estágio, na Figura 25 a vazão nominal do injetor ficou entre as duas curvas, atendendo o comportamento esperado.

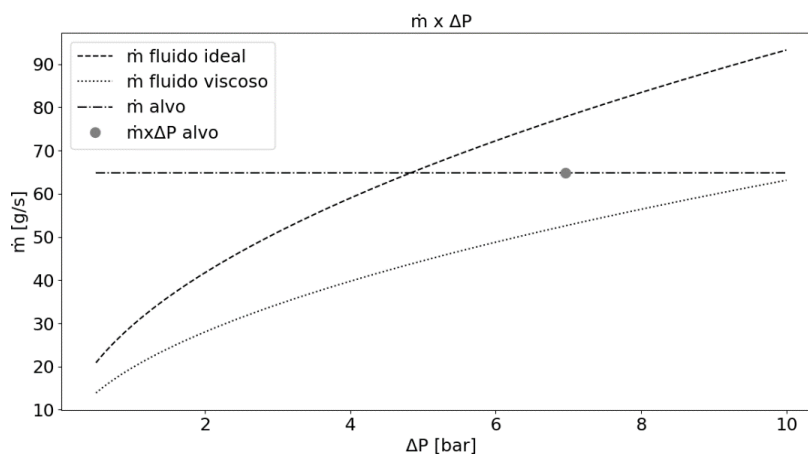
Figura 24 - Curvas de Vazão para o Estágio de Oxidante do Injetor do RD0110.



Fonte: elaborado pelo autor.

Nota-se também que, para os parâmetros do injetor do segundo estágio, há uma grande diferença entre a vazão com e sem efeito viscoso, implicando numa grande incerteza para o método de cálculo utilizado.

Figura 25 - Curvas de Vazão para o Estágio de Combustível do Injetor do RD0110.

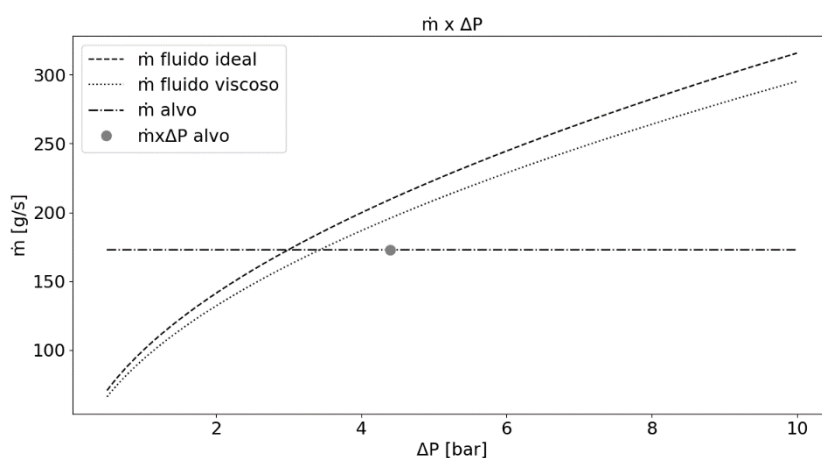


Fonte: elaborado pelo autor.

4.2.2 Resultados do M2 para o RD0109

Os valores calculados e o nominal de vazão de oxidante do RD0109 são expostos na Figura 26. A vazão nominal do 1º estágio do injetor apresentado por Kessaev e acessada pelo trabalho de Alves (ALVES, 2008) se demonstrou significativamente inferior às vazões calculadas.

Figura 26 - Curvas de Vazão para o Estágio de Oxidante do Injetor do RD0109.

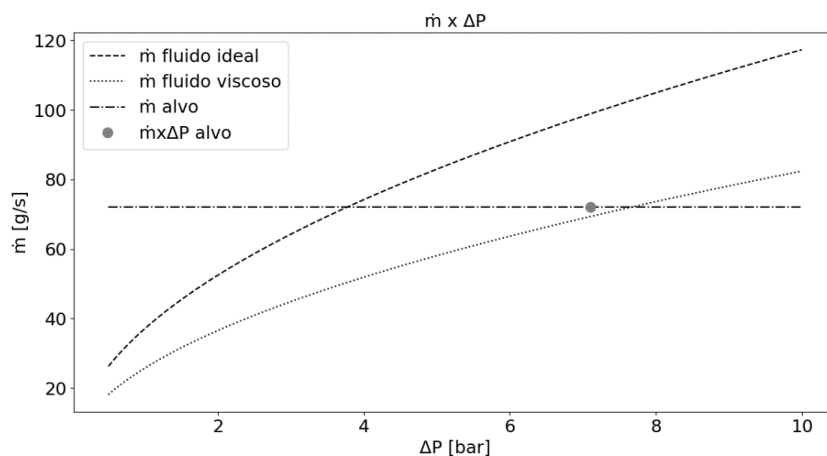


Fonte: elaborado pelo autor.

Tal comportamento da vazão de oxidante indica que pode haver um superdimensionamento da perda de momento angular ou subdimensionamento das perdas nos canais de entrada para as condições geométricas e de operação considerados na simulação deste injetor. O mesmo efeito não é observado para o estágio de combustível, Figura 27, a vazão nominal do injetor ficou entre as duas curvas e próxima a curva que contabiliza os efeitos viscosos, atendendo o comportamento esperado.

Nota-se também que para os parâmetros do injetor de combustível, há uma grande diferença entre a vazão com e sem efeito viscoso, implicando numa grande incerteza para o método de cálculo utilizado.

Figura 27 - Curvas de Vazão para o Estágio de Combustível do Injetor do RD0109.

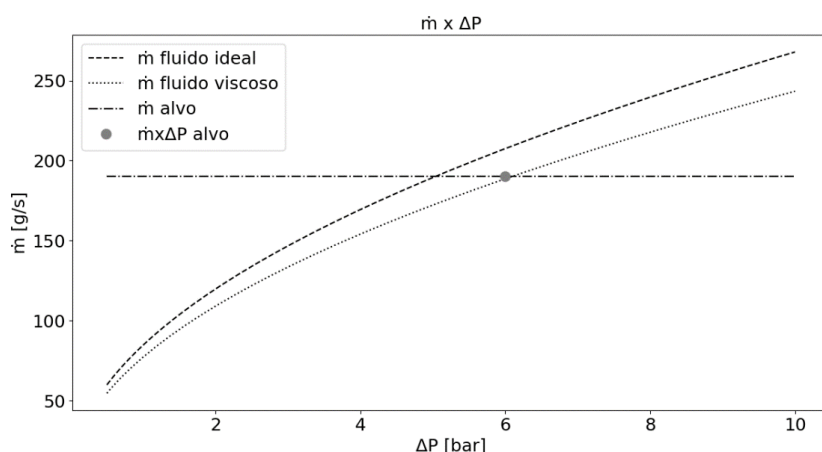


Fonte: elaborado pelo autor.

4.2.3 Resultados do M2 para o Propulsor Proposto – Injetor Central

Para o Injetor de oxidante no 1º estágio, nota-se que na Figura 28, a vazão nominal do injetor ficou entre as duas curvas e próxima à curva que contabiliza os efeitos viscosos, atendendo o comportamento esperado.

Figura 28 - Curvas de Vazão para o Estágio de Oxidante do Injetor Proposto Central.

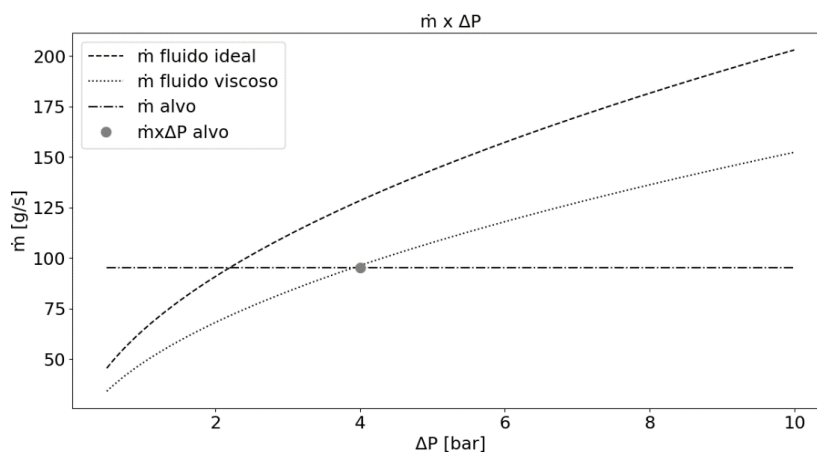


Fonte: elaborado pelo autor.

A vazão nominal do injetor de combustível, apresentada na Figura 29, foi aproximadamente igual à vazão calculada para o caso viscoso. Nota-se também que para os

parâmetros deste injetor, há uma grande diferença entre a vazão com e sem efeito viscoso, implicando numa grande incerteza para o método de cálculo utilizado.

Figura 29 - Curvas de Vazão para o Estágio de Combustível do Injetor Proposto Central.

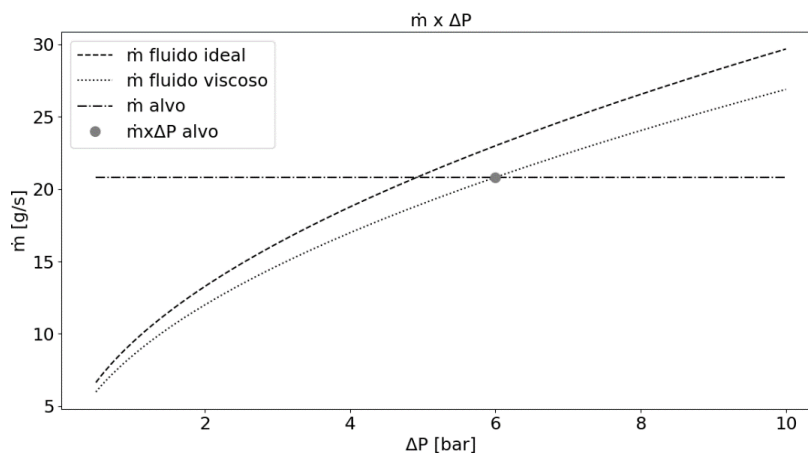


Fonte: elaborado pelo autor.

4.2.4 Resultados do M2 para o Propulsor Proposto – Injetor Periférico

Conforme exposto na Figura 30, a vazão nominal do estágio de oxidante foi aproximadamente igual à vazão calculada para o caso viscoso.

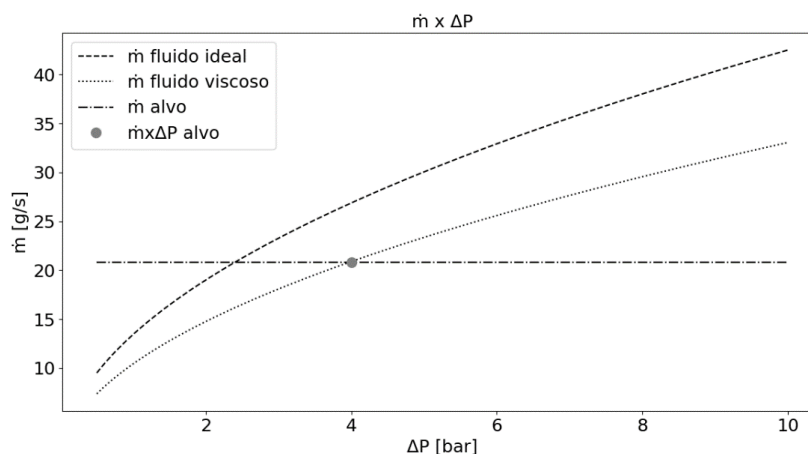
Figura 30 - Curvas de Vazão para o Estágio de Oxidante do Injetor Proposto Periférico.



Fonte: elaborado pelo autor.

No caso do estágio de combustível, cujo resultado é apresentado na Figura 31, a vazão nominal do injetor foi aproximadamente igual à vazão calculada para o caso viscoso. Nota-se também que para os parâmetros deste injetor, há uma grande diferença entre a vazão com e sem efeito viscoso, implicando numa grande incerteza para o método de cálculo utilizado.

Figura 31 – Curvas de Vazão para o Estágio de Combustível do Injetor Proposto Periférico.



Fonte: elaborado pelo autor.

Nota-se que os estágios de combustível, que nos casos avaliados apresentam altos valores do parâmetro geométrico, implicam numa diferença significativa entre as curvas ideais e viscosas, no entanto, os valores experimentais apresentados pelas referências se apresentaram muito próximos aos valores calculados considerando a viscosidade.

4.3 Valores Calculados de Tamanho de Gota

Foram calculados 4 métodos distintos para o SMD dos injetores propostos, sendo que cada um deles apresenta um conjunto diferente de variáveis que o determinam e, portanto, divergem sensivelmente entre si. Os dados calculados são apresentados na Tabela 15.

Tabela 15 – Valores de SMD calculados para os injetores propostos.

Injetor	Estágio	Método	
C – Central	1 – Oxidante	Radcliffe	31.75 μm
		Jasuja	34.56 μm
		Lefebvre	11.22 μm
		Wang & L.	91.30 μm
	2 – Combustível	Radcliffe	40.14 μm
		Jasuja	45.80 μm
		Lefebvre	28.56 μm
		Wang & L.	70.99 μm
P – Periférico	1 – Oxidante	Radcliffe	18.26 μm
		Jasuja	25.29 μm
		Lefebvre	6.45 μm
		Wang & L.	46.07 μm
	2 – Combustível	Radcliffe	27.45 μm
		Jasuja	32.78 μm
		Lefebvre	19.53 μm
		Wang & L.	41.77 μm

Fonte: calculado pelo autor.

Quase todos os valores calculados se encaixaram no intervalo de 10 a 400 μm , descrito por Lefebvre (LEFEBVRE, 1983), com apenas uma divergência apresentando valor inferior ao esperado. Apesar desse resultado ser uma indicação positiva e valores reduzidos de tamanho de gota poderem ter um impacto positivo no processo de combustão, de acordo com Rubinsky, (RUBINSKY, 1994), e Bazarov, (BAZAROV, YANG e PURI, 2004), tamanhos de gota excessivamente pequenos ou excessivamente grandes podem excitar modos de instabilidade a depender da resposta dinâmica dos injetores e da Câmara.

5 CONCLUSÕES E TRABALHOS FUTUROS

Tanto os resultados do dimensionamento dos injetores de validação e propostos, quanto os resultados de tamanho de gota foram satisfatórios na maior parte dos casos e apresentaram valores próximos aos valores de referência. Os resultados do método de dimensionamento M1 condizem com o esperado com divergências diminutas; em geral da ordem de grandeza da tolerância dos processos de fabricação usualmente aplicados. No entanto, foi possível notar divergências significativas para os injetores de oxidante de referência e nos resultados do método de ajustes M2.

Nos resultados do algoritmo de ajustes M2 (utilizando as medidas originais), fica claro que para os estágios de oxidante dos injetores de referência, o método de cálculo superdimensiona a vazão. Analisando a tendência geral das divergências geométricas, nota-se que em baixos valores do parâmetro geométrico (aprox. 2.00) a vazão calculada é maior que o esperado e em altos valores de parâmetro geométrico (de 24 a 27), a vazão é menor que a esperada.

A imprecisão detectada pode partir de uma divergência dos parâmetros do propelente utilizado, o que pode ser influenciado por uma temperatura e/ou pressão (absoluta) de referência diferentes, ou de imprecisões inerentes ao modelo de cálculo. Além disso, caso os injetores em questão não sejam hidráulicamente independentes como os que possuem mistura interna, de fato não é esperada uma aderência precisa aos resultados.

Algumas formas de aprimorar a assertividade do equacionamento para os injetores propostos e os de referência pode ser, por exemplo, por meio de trabalhos futuros com testes hidráulicos dos injetores. A partir destes pode-se investigar as divergências e validar melhorias no modelo de cálculo.

Considerando a modelagem utilizada, um dos fatores que pode causar um impacto significativo no modelo é a diferença na configuração de rugosidade de superfície nos canais tangenciais e na cavidade principal do injetor. O modelo apresentado por Bazarov (BAZAROV,

YANG e PURI, 2004) e Bayvel (BAYVEL e ORZECOWSKI, 1993) utiliza um λ único, no entanto, ao longo do injetor podem haver condições diversas de rugosidade, que podem levar a uma predominância do efeito da perda na entrada ou das perdas de momento angular na cavidade principal, respectivamente acarretando numa vazão menor ou maior que o esperado.

Portanto, propõe-se para trabalhos futuros a análise com λ s diferentes para os canais tangenciais e para o resto do injetor, para ajuste do modelo de cálculo de vazão de acordo com a característica superficial devido ao método de fabricação de cada seção do injetor. Dessa forma, com um coeficiente impactando diretamente as perdas nos canais tangenciais, que diminuem a vazão, e com outro coeficiente impactando na perda de momento angular em toda a cavidade do injetor, é possível ajustar com maior exatidão a vazão esperada.

Outro efeito observado e que poderia ser investigado e quantificado é da aparente maior sensibilidade dos injetores de alto parâmetro geométrico em relação aos efeitos viscosos. Para os injetores de combustível dos segundos estágios, nota-se uma diferença pronunciada entre a curva de vazão ideal e viscosa, tanto em valor absoluto, quanto em inclinação. Dessa forma estipula-se que estes estágios sejam ainda mais sensíveis especialmente quanto aos métodos de fabricação e regimes de número de Reynolds de operação.

Na fabricação de injetores centrífugos existem medidas com tolerância muito estritas, em especial das dimensões dos canais tangenciais e outros parâmetros como concentricidade da câmara de vórtice e o bocal. Tais exigências dificultam significativamente a fabricação e processo de qualidade. Como forma de analisar esses efeitos, é proposto o algoritmo M3 que é uma ferramenta pensada para a análise quantitativa do processo de fabricação.

Propõe-se que futuramente este algoritmo seja desenvolvido e opere partindo da geometria do injetor, as tolerâncias e desvio esperados para o método de fabricação escolhido

e assim calcular um envelope de operação do injetor real fabricado. Este envelope pode ser comparado com as regiões limítrofes de operação do motor, validando ou não o planejamento de manufatura pretendido e servindo de insumo para análises de confiabilidade.

6 REFERÊNCIAS

ALVES, A. **Estudo e Desenvolvimento de Um Sistema de Injeção Centrífugo Bipropelente Utilizado em Motor Foguete a Propelente Líquido**. São José dos Campos. 2008.

BAYVEL, L.; ORZECOWSKI, Z. **Liquid Atomization**. [S.l.]: Taylor & Francis, 1993.

BAZAROV, V.; YANG, V.; PURI, P. Design and Dynamics of Jet and Swirl Injectors. In: YANG, V., et al. **Liquid Rocket Thrust Chambers: Aspects of Modeling, Analysis, and Design**. [S.l.]: AIAA, 2004. p. 19-103.

BETADY, E. et al. Development of a Mobile Rocket Engine Test Stand (MRETS). **Joint Propulsion Conference - AIAA**, 2018.

BRYCE TECH. **Start-Up Space 2022 - Update on Investment in Commercial Space Ventures**. [S.l.]. 2022.

CALABUIG, G.; WILKEN, J. Pre-conceptual staging trade-offs of reusable vehicles. **EUCASS**, 2022.

CALLSEN, S.; STAPPERT, S.; SIPPEL, M. Study on Future European Winged Reusable Launchers. **EUCASS**, 2022.

CNBC. The space industry will be worth nearly \$3 trillion in 30 years, Bank of America predicts, 2017. Disponível em: <<https://www.cnbc.com/2017/10/31/the-space-industry-will-be-worth-nearly-3-trillion-in-30-years-bank-of-america-predicts.html>>.

DANSTAR - TECHNICAL UNIVERSITY OF DENMARK. Danish Student Association for Rocketry. Disponível em: <<https://www.danstar.dk/projects/>>. Acesso em: jun 2023.

DARE - TU DELFT. Cryogenic Rocket Propulsion. Disponível em: <<https://dare.tudelft.nl/projects/cryogenic-rocket-propulsion/>>. Acesso em: jun 2023.

DELPY, R.; OSWALD, J. PERSEUS Project 5kN LOx/Ethanol Rocket Engine Fire Tests. **EUCASS**, 2017.

EUCASS2017. EUCASS 2017 DOI of full papers. Disponível em: <<https://www.eucass.eu/index.php/conferences-and-publications/eucass-2017/eucass-2017-d>>. Acesso em: jun 2023.

EUCASS2019. Flyer for the 8th European Conference for Aerospace Sciences. Disponível em: <http://www.eucass2019.eu/wp-content/uploads/eucass2019_call_for_papers_181029.pdf>. Acesso em: jun 2023.

GONTIJO, M.; FISCHER, G.; COSTA, F. Evaluation of SMD Effects on Characteristic Lengths of Liquid Rocket Engines Using Ethanol/LOx and RP-1/LOx. **ENCIT**, 2020.

GOULART, A. **Testes Hidráulicos dos Injetores do Motor Bipropelente de 400 N da Iteração I**. São José dos Campos. 2019.

GOULART, A.; HINCKEL, J.; BONTEMPO, L. Design and Test of Non-Uniform Spray Swirl Injectors and Its Applications in Injection Plate Design. **IAC**, 2020.

GRADL, P. et al. GRCop-42 Development and Hot-fire Testing Using Additive Manufacturing Powder Bed Fusion for Channel-Cooled Combustion Chambers. **Propulsion and Energy Forum - AIAA**, 2019.

GRADL, P.; MIRELES, O.; ANDREWS, N. **Introductions to Additive Manufacturing for Propulsion and Energy Systems**. [S.l.]. 2020.

HINCKEL, J.; GOULART, A.; BONTEMPO, L. Modulation of the Spray Jet of a Swirl Injector. **EUCASS**, 2019.

HUZEL, D.; HUANG, D. **Modern Engineering for Design of Liquid-Propellant Rocket Engines**. [S.l.]: AIAA, 1992.

INDIANA, C. et al. Combustion of Sprays from triplet injector with green propellants. **EUCASS**, 2017.

JOHNSON, M. \$1.67 Million Gift Fuels Cal Poly Pomona Students' Mission to be First. Disponível em: <<http://polycentric.cpp.edu/2017/05/1-67-million-gift-fuels-cal-poly-pomona-students-mission-t>>. Acesso em: jun 2023.

KAJON, D. et al. Development of the Liquid Oxygen and Methane M10 Rocket Engine for the Vega-E Upper Stage. **EUCASS**, 2019.

KANG, C. et al. Effect of Recess Length on Combustion Characteristics of Additive Manufactured Swirl Coaxial Injector. **EUCASS**, 2022.

KHAVKIN, Y. **The Theory and Practice of Swirl Atomizers**. [S.l.]: Taylor & Francis, 2004.

KIM, D. et al. Effect of Recess on the Spray Characteristics of Liquid-Liquid Swirl Coaxial Injectors. **Journal of Propulsion and Power - AIAA**, 2007.

LEFEBVRE, A. **Gas Turbine Combustion**. [S.l.]: Hemisphere - Taylor & Francis Group, 1983.

LEFEBVRE, A.; MCDONELL, V. **Atomization and Sprays**. [S.l.]: CRC Press - Taylor & Francis Group, 2017.

MARTIN, G. **NewSpace: The Emerging Comercial Space Industry**. [S.l.]. 2015.

PONOMARENKO, A. Rocket Propulsion Analysis, 2023. Disponivel em: <<http://www.rocket-propulsion.com/index.htm>>.

RUBINSKY, V. Combustion Instabiity in the RD-0110 Engine. In: YANG, V.; ANDERSON, W. **Liquid Rocket Engine Combustion Instability**. [S.l.]: AIAA, 1994. p. 89-112.

SDSU ROCKET PROJECT. FAR/Mars Project. Disponivel em: <<https://www.sdsurocketproject.org/far-competition-rocket>>. Acesso em: fev 2019.

SIEDER, J. et al. Development History and Verification of the Flight Model of a 500 N Ethanol/LOx Rocket Engine. **EUCASS**, 2017.

SOLLER, S. et al. Design and Testing of Liquid Propellant Injectors for Additive Manufacturing. **EUCASS**, 2017.

SUTTON, G.; BIBLARZ, O. **Rocket Propulsion Elements, 9th Edition**. Hoboken, New Jersey: John Wiley & Sons, Inc, 2017.

TOKUDOME, S. et al. Experimental Study of a Nitrous Oxide/Ethanol Propulsion System: Technology Demonstration with BBM. **EUCASS**, 2013.

UCSD-SEDS. Colossus Liquid Rocket Engine Static Test Stand Design Proposal. **NASA Marshall Space Flight Center**, 2016.

VASQUES, B.; HAIDN, O. Effect of Pintle Injector Element Geometry on Combustion in a Liquid Oxygen/Liquid Methane Rocket Engine. **EUCASS**, 2017.

7 ANEXO A – CÓDIGOS DO MÉTODO M1

7.1 M1_main.py

```

"""
@author: A. Goulart
"""

#####
folder_name = "rev_B01/v_2"

#####
file_name = "B01_C"
#file_name = "B01_W"
#####

#####
#folder_name = "RD0110/v_2"

#####
#file_name = "RD0110"
#file_name = "RD0110_Ox"
#file_name = "RD0110_F"
#####

#####
#folder_name = "RD0109"

#####
#file_name = "RD0109"
#file_name = "RD0109_Ox"
#file_name = "RD0109_F"
#####

from M1_Method import Method_1

Run_1 = Method_1(folder_name, file_name)

Run_1.set_it_lim(100)
Run_1.set_erro_max(1e-6)

Run_1.set_show_it(0)
Run_1.show_plot_error(0)
Run_1.show_plot_inj(1)

Run_1.set_dpi(200)
Run_1.set_textbox_fontsize(6)
#Run_1.set_textbox_offset(24)
#Run_1.set_textbox_offset(6)
Run_1.set_textbox_offset(14)

Run_1.run_M1()

```

7.2 M1_FileReader.py

```

"""
@author: A. Goulart
"""

class FileReader_1:
    def setFolderName(self, fo_name):
        self.folder_name = fo_name
    def setFileName(self, fi_name):
        self.file_name = fi_name
    def read(self):

        ##### Injector parameters reading #####

        file_loc = str(self.folder_name) + '/' + str(self.file_name)
+ '.txt'

        Input = open(str(file_loc), 'r')

        for i in range(5): Input.readline()

        injector_ID = str(Input.readline())
        print("Identificação do injetor: \t", injector_ID)

        Input.readline()
        self.config = str(Input.readline())
        print("Configuração do injetor: \t", self.config)

        if(self.config == "Mono\n"):

            ##### Beginning of ST1 data if monopropellant #####

            Input.readline()
            self.in_type_1 = str(Input.readline())

            Input.readline()
            self.inlet_radius_1 = float(Input.readline())

            Input.readline()
            self.propellant_1 = str(Input.readline())

            Input.readline()
            self.m_1 = float(Input.readline())

            Input.readline()
            self.delta_p_1 = float(Input.readline())

            Input.readline()
            self.alpha2_1 = float(Input.readline())

            Input.readline()
            self.n_1 = int(Input.readline())

            Input.readline()
            self.opening_coef_1 = float(Input.readline())

```

```

Input.readline()
self.trans_angle_1 = float(Input.readline())

Input.readline()
self.ratio_l_in_1 = float(Input.readline())

Input.readline()
self.ratio_l_s_1 = float(Input.readline())

Input.readline()
self.ratio_l_n_1 = float(Input.readline())

print("ST1: Data read Successful")

elif(self.config == "Bi\n"):

    Input.readline()
    self.t_w = float(Input.readline())

    Input.readline()
    self.delta = float(Input.readline())

    Input.readline()
    self.temp_value = Input.readline()
    if(self.temp_value == "disabled\n"):
        self.angle_dif = "disabled"
    else:
        self.angle_dif = float(self.temp_value)

    Input.readline()
    self.recess = float(Input.readline())

    ##### Beginning of ST1 data if bipropellant #####

    Input.readline()
    self.in_type_1 = str(Input.readline())

    Input.readline()
    self.inlet_radius_1 = float(Input.readline())

    Input.readline()
    self.propellant_1 = str(Input.readline())

    Input.readline()
    self.m_1 = float(Input.readline())

    Input.readline()
    self.delta_p_1 = float(Input.readline())

    Input.readline()
    self.alpha2_1 = float(Input.readline())

    Input.readline()
    self.n_1 = int(Input.readline())

    Input.readline()
    self.opening_coef_1 = float(Input.readline())

    Input.readline()
    self.trans_angle_1 = float(Input.readline())

```

```

Input.readline()
self.ratio_l_in_1 = float(Input.readline())

Input.readline()
self.ratio_l_s_1 = float(Input.readline())

Input.readline()
self.ratio_l_n_1 = float(Input.readline())

print("ST1: Data read Successful")

#### Beginning of ST2 data ####

Input.readline()
self.in_type_2 = str(Input.readline())

Input.readline()
self.inlet_radius_2 = float(Input.readline())

Input.readline()
self.propellant_2 = str(Input.readline())

Input.readline()
self.m_2 = float(Input.readline())

Input.readline()
self.delta_p_2 = float(Input.readline())

Input.readline()
self.alpha2_2 = float(Input.readline())

Input.readline()
self.n_2 = float(Input.readline())

Input.readline()
self.opening_coef_2 = float(Input.readline())

Input.readline()
self.trans_angle_2 = float(Input.readline())

Input.readline()
self.ratio_l_in_2 = float(Input.readline())

Input.readline()
self.ratio_l_s_2 = float(Input.readline())

Input.readline()
self.ratio_l_n_2 = float(Input.readline())

Input.readline()
self.ratio_l_n_out = float(Input.readline())

Input.readline()
self.trans_angle_out = float(Input.readline())

print("ST2: Data read Successful")

else:
    print("ERROR: Configuration type not recognized")

```

```

##### Propellant properties #####

Input = open('prop_data.txt','r')

for i in range(4): Input.readline()

Input.readline()
self.n_propellants = int(Input.readline())

Input.readline()
self.n_properties = int(Input.readline())

self.properties = []

for i in range(self.n_propellants):

    self.properties.append([])

    Input.readline()
    self.properties[i].append(str(Input.readline()))

    for j in range(self.n_properties-1):

        Input.readline()
        self.properties[i].append(float(Input.readline()))

    Input.readline()

prop_ST1 = []
prop_ST2 = []

if(self.config == "Mono\n"):

    ##### properties ST1 #####

    for i in range(self.n_propellants):
        if(self.properties[i][0] == self.propellant_1):
            prop_ST1 = self.properties[i]

    self.rho_1 = prop_ST1[1]

    self.din_visc_1 = prop_ST1[2]

    self.visc_sup_1 = prop_ST1[3]

elif(self.config == "Bi\n"):

    ##### properties ST1 #####

    for i in range(self.n_propellants):
        if(self.properties[i][0] == self.propellant_1):
            prop_ST1 = self.properties[i]

    self.rho_1 = prop_ST1[1]

```

```

self.din_visc_1 = prop_ST1[2]

self.visc_sup_1 = prop_ST1[3]

#### properties ST2 ####

for i in range(self.n_propellants):
    if(self.properties[i][0] == self.propellant_2):
        prop_ST2 = self.properties[i]

self.rho_2 = prop_ST2[1]

self.din_visc_2 = prop_ST2[2]

self.visc_sup_2 = prop_ST2[3]

else:
    print("ERROR: Configuration type not recognized (prop)")

```

7.3 M1_Method.py

```

"""
@author: A. Goulart
"""

import numpy as np

import matplotlib.pyplot as plt

from M1_FileReader import FileReader_1

class Method_1:
    def __init__(self, foldername, filename):
        self.it_lim = 1000
        self.erro_max = 1e-4

        self.fontsize = 10
        self.offset = 26
        self.dpi_value = 800
        self.show_it = 1
        self.fmt = '| | {{: ^{}}s}} | {{: ^{}}s}} |'.format(25, 15) #
widths only

        self.fr = FileReader_1()
        self.fr.setFolderName(foldername)
        self.fr.setFileName(filename)

        plt.isinteractive()

    def set_show_it(self, show_val):
        self.show_it = show_val

    def get_show_it(self):
        return self.show_it

    def set_it_lim(self, n):
        self.it_lim = n

    def get_it_lim(self):
        return self.it_lim

    def set_erro_max(self, n):
        self.erro_max = n

    def get_erro_max(self):
        return self.erro_max

    def set_textbox_fontsize(self, fs):
        self.fontsize = int(fs)

    def get_textbox_fontsize(self):
        return self.fontsize

    def set_textbox_offset(self, off):
        self.offset = off

```

```

def get_textbox_offset(self):
    return self.offset

def show_plot_inj(self, answer):
    self.sh_plot_inj = answer

def show_plot_error(self, answer):
    self.sh_plot_error = answer

def set_dpi(self, n):
    self.dpi_value = n

def get_dpi(self):
    return self.dpi_value

def print_data(self, label, value):
    print(self.fmt.format(str(label), str(value)))

def check_value(self, var_name, min_value, max_value, var_value):
    if(var_value > max_value):
        print("ERROR: %s value out of bounds [%2f, %2f]
(current value: %2f)"(var_name,min_value,max_value,var_value))

        elif(var_value < min_value):
            print("ERROR: %s value out of bounds [%2f, %2f]
(current value: %2f)"(var_name,min_value,max_value,var_value))

##### Métodos - Injetor Ideal #####

def calc_phi(self, alpha2):
    return 2/(2+(np.tan(alpha2/2*np.pi/180))**2)

def calc_geom_char(self, phi):
    return (1-phi)*np.sqrt(2/phi)/phi

def calc_mu(self, phi):
    return phi*np.sqrt(phi/(2-phi))

def calc_a(self, phi):
    return 2*((1-phi)**2)/(2-phi)

def calc_2alpha(self, coef, coef_type):
    if(coef_type == "phi"):
        return 2*180/np.pi*np.arctan(np.sqrt(2*(1-coef)/coef))
    elif(coef_type == "a"):
        return 2*180/np.pi*np.arctan(coef/(1-coef))
    else:
        print("ERROR: input not understood in calc_2alpha")

def calc_2alpha_avg(self, mu, A, phi, ksi, C):
    return 2*(180/np.pi)*np.arcsin(2*mu*A/((1+np.sqrt((1-
phi))*(1-ksi*mu**2*(A/(C))**2))))

##### Métodos - Injetor Real #####

## Calcula o valor do n° de Reynolds nos canais tangenciais do
injetor

def calc_reynolds_in(self, m_in, din_visc, n, r_in_orf, rho):
    return (2*m_in*1e-3)/(din_visc*n*np.pi*(r_in_orf*1e-3))
#return (0.637*m_in)/(np.sqrt(n)*r_in_orf*rho*din_visc)

```

```

    ## calcula o valor do coeficiente de atrito (Lambda)
    def calc_friction_coef(self, reynolds_in):
        #return 10**((25.8/(np.log(reynolds_in))-2) #Bayvel
        return 0.3164*reynolds_in**(-0.25) #Bazarov

    ## Calcula o valor da perda hidráulica no injetor (Ksi)
    def calc_hyd_loss(self, inlet_type, r_s, l_in, inlet_radius,
r_in_orf, friction_coef):
        if(inlet_type == "curved\n"):
            self.alpha_in = 90 - 180/np.pi * np.arctan(r_s/l_in)
#eq(105)
            self.ksi_in = -1*self.alpha_in/150 + 1.1

            elif(inlet_type == "straight\n"):
                self.ksi_in = 0.5*np.exp(-
1.4*(inlet_radius/(2*r_in_orf)))

            else:
                print("Error in calc_hyd_loss:\n Inlet type not
recognized")

                return self.ksi_in + friction_coef*l_in/(2*r_in_orf)

    ## calcula o parâmetro geométrico característico corrigido (A_eq)
[Bazarov, eq(100)]
    def calc_geom_char_eq(self, r_in_pos, r_n, n, r_in_orf,
friction_coef):
        return
(r_in_pos*r_n)/(n*r_in_orf**2+(friction_coef/2)*r_in_pos*(r_in_pos-r_n))

    ## Calcula o Valor do Coeficiente de Preenchimento corrigido
(Phi_eq) [Bazarov, eq(99), eq(100)]
    def calc_phi_eq(self, geom_char_eq):
        self.coefs = [geom_char_eq**2, -2, 4, -2]
        self.roots = np.roots(self.coefs)

        self.count = 0

        self.phi_eq = -1

        for root in self.roots:
            if(0 < root < 1 and root.imag == 0):
                self.count+=1
                self.phi_eq = root.real

        if(self.phi_eq == -1):
            print("Error in Calc_Phi_eq:\n No valid root found.")

        if(self.count > 1):
            print("Error in Calc_Phi_eq:\n Multiple valid values
found")

            print(self.roots)

        return self.phi_eq

    ## Calcula o coeficiente de descarga equivalente [Bazarov,
eq(99)]
    def calc_mu_eq(self, phi_eq):
        return phi_eq**1.5/np.sqrt(2-phi_eq)

```

```

    ## Calcula o coeficiente de descarga real do injetor [Bazarov,
eq(99)]
    def calc_mu_i(self, mu_eq, geom_char, r_in_pos, r_n, ksi):
        return
mu_eq/np.sqrt(1+ksi*mu_eq**2*(geom_char/(r_in_pos/r_n))**2)

    ##### Métodos - Cálculos adicionais #####

    ##Calcula o valor da não-uniformidade percentual esperada do
injetor [Bayvel]
    def calc_non_uniformity(self, r_in_pos, r_n, n, l_s, r_s,
phi_eq):
        return
23.7/((r_in_pos/r_n)**2.7*n**1.34*phi_eq**1.1*(l_s/(2*r_s))**0.15)

    ##### Roda o método de dimensionamento do injetor #####

    def run_M1(self):

        self.fr.read()

        print("Config: ", self.fr.config)

        ##### ##### ##### ##### ##### ##### ##### ##### ##### #####
##### ##### #####
        ##### ##### ##### Parte 1 - Cálculo do ST1 ##### ##### #####
##### ##### #####
        ##### ##### ##### ##### ##### ##### ##### ##### ##### #####
##### ##### #####

        self.phi_1 = self.calc_phi(self.fr.alpha2_1)

        self.A_1 = self.calc_geom_char(self.phi_1)

        self.mu_1 = self.calc_mu(self.phi_1)

        self.r_n_1 = 1e3*0.475*np.sqrt(self.fr.m_1*1e-3 / (self.mu_1
* np.sqrt(self.fr.rho_1*self.fr.delta_p_1*1e5)))

        print("\n\n ST1 Initial Values:")
        self.print_data("A_1", "%.2f"%(self.A_1))
        self.print_data("Phi_1", "%.3f"%(self.phi_1))
        self.print_data("R_n_1", "%.3f mm"%(self.r_n_1))
        self.print_data("2̑", "%.1f deg"%(self.fr.alpha2_1))

        self.inlet_diameter = []
        self.outlet_error = []

        self.erro_r = self.erro_max + 1
        self.contador = 0

        if(self.show_it == 1): print("\n ST1 Iteration data:")
        while self.erro_r >= self.erro_max and self.contador <
self.it_lim:

        ##### Iteração do injetor ideal #####

```

```

        self.r_in_pos_1 = self.fr.opening_coef_1*self.r_n_1

        self.r_in_orf_1 =
np.sqrt(self.r_in_pos_1*self.r_n_1/(self.fr.n_1*self.A_1))

        self.inlet_diameter.append(2*self.r_in_orf_1)

        self.r_s_1 = self.r_in_pos_1+self.r_in_orf_1

        self.l_in_1 = self.fr.ratio_l_in_1*2*self.r_in_orf_1

        self.l_n_1 = self.fr.ratio_l_n_1*2*self.r_n_1

        self.l_s_1 = self.fr.ratio_l_s_1*2*self.r_s_1

        self.a_1 = self.calc_a(self.phi_1)

        self.alpha2_1 = self.calc_2alpha(self.phi_1, "phi")

        self.alpha2_e_1 = self.calc_2alpha(self.a_1, "a")

        ##### Iteração do injetor real #####

        self.rey_in_1 = self.calc_reynolds_in(self.fr.m_1,
self.fr.din_visc_1, self.fr.n_1, self.r_in_orf_1,self.fr.rho_1)

        self.lambda_1 = self.calc_friction_coef(self.rey_in_1)

        self.ksi_1 = self.calc_hyd_loss(self.fr.in_type_1,
self.r_s_1, self.l_in_1, self.fr.inlet_radius_1, self.r_in_orf_1,
self.lambda_1)

        self.check_value("ksi_1", 0, float("inf"), self.ksi_1)

        self.A_eq_1 = self.calc_geom_char_eq(self.r_in_pos_1,
self.r_n_1, self.fr.n_1, self.r_in_orf_1, self.lambda_1)

        self.phi_eq_1 = self.calc_phi_eq(self.A_eq_1)

        self.mu_eq_1 = self.calc_mu_eq(self.phi_eq_1)

        self.mu_i_1 = self.calc_mu_i(self.mu_eq_1, self.A_eq_1,
self.r_in_pos_1, self.r_n_1, self.ksi_1)

        self.a_eq_1 = self.calc_a(self.phi_eq_1)

        self.alpha2_eq_1 = self.calc_2alpha(self.phi_eq_1, "phi")

        self.alpha2_e_eq_1 = self.calc_2alpha(self.a_eq_1, "a")

        ## cálculo do novo raio do bocal

        self.r_n_old = self.r_n_1

        self.r_n_1 = 1e3*0.475*np.sqrt(self.fr.m_1*1e-3 /
(self.mu_i_1 * np.sqrt(self.fr.rho_1*self.fr.delta_p_1*1e5)))

        self.erro_r = abs(self.r_n_old-self.r_n_1)/self.r_n_old

        self.outlet_error.append(self.erro_r)

```

```

        self.A_1 =
self.r_in_pos_1*self.r_n_1/(self.fr.n_1*self.r_in_orf_1**2)

        self.phi_1 = self.calc_phi_eq(self.A_1)

        self.K_1 = self.A_eq_1/self.A_1
        self.check_value("K_1", 0, 1, self.K_1)

        self.r_mn_1 = self.r_n_1*np.sqrt(1-self.phi_eq_1)

        self.r_mk_1 = self.r_mn_1 * np.sqrt(2*(1-
self.phi_eq_1)/(2-self.phi_eq_1))

        self.t_fluid_1 = self.r_n_1-self.r_mn_1

        self.C_1 = self.r_in_pos_1/self.r_n_1

        self.alpha2_avg_1 = self.calc_2alpha_avg(self.mu_i_1,
self.A_eq_1, self.phi_eq_1, self.ksi_1, self.C_1)

    ## print dos dados da iteração
    if(self.show_it == 1):
        print("\nIt. %d"%(self.contador+1))
        self.print_data("A_1", "%.4f"%(self.A_1))
        self.print_data("A_eq_1", "%.4f"%(self.A_eq_1))
        self.print_data("K_1", "%.4f"%(self.K_1))
        self.print_data("Phi_1", "%.4f"%(self.phi_1))
        self.print_data("Phi_eq_1", "%.4f"%(self.phi_eq_1))
        self.print_data("a_1", "%.4f"%(self.a_1))
        self.print_data("a_eq_1", "%.4f"%(self.a_eq_1))
        self.print_data("rey_in_1", "%.4f"%(self.rey_in_1))
        self.print_data("ksi_1", "%.4f"%(self.ksi_1))
        self.print_data("lambda_1", "%.4f"%(self.lambda_1))
        self.print_data("mu_i", "%.4f"%(self.mu_i_1))
        self.print_data("C", "%.4f"%(self.C_1))
        #self.print_data("2Alpha ideal", "%.2f
deg"%(self.alpha2_1))
        #self.print_data("2Alpha real", "%.2f
deg"%(self.alpha2_eq_1))
        #self.print_data("2Alpha ideal jusante", "%.2f
deg"%(self.alpha2_e_1))
        #self.print_data("2Alpha real jusante", "%.2f
deg"%(self.alpha2_e_eq_1))
        self.print_data("2Alpha_avg", "%.2f
deg"%(self.alpha2_avg_1))
        self.print_data("R_n_1", "%.3f mm"%(self.r_n_1))
        self.print_data("r_in_1", "%.3f
mm"%(self.r_in_orf_1))
        self.print_data("Error", "%.2E"%(self.erro_r))

        self.contador += 1

    print("\n\n")

    ##Cálculo da não-uniformidade esperada do injetor
    self.I_1 = self.calc_non_uniformity(self.r_in_pos_1,
self.r_n_1, self.fr.n_1, self.l_s_1, self.r_s_1, self.phi_eq_1)

    if(self.contador >= self.it_lim):

```

```

        print ("\tWarning:\n ST1 reached iteration limit.")

        if(np.pi*self.r_n_1**2 <
self.fr.n_1*np.pi*self.r_in_orf_1**2):
            print("\tWarning:\n Outlet area smaller than total inlet
area (dif = %.2f mm²)"%(self.fr.n_1*np.pi*self.r_in_orf_1**2 -
np.pi*self.r_n_1**2))
            print("\tArea_in = %.2f mm²; \tArea_out = %.2f
mm²"%(self.fr.n_1*np.pi*self.r_in_orf_1**2, np.pi*self.r_n_1**2))

        ##### ST1 design console report #####

        print("\n\nST1 Injector Geometry: \n")
        self.print_data("# of Inlet Channels", "%.1f"%(self.fr.n_1))
        self.print_data("Inlet Channel Radius", "%.3f
\tmm"%(self.r_in_orf_1))
        self.print_data("Inlet Radius", "%.3f
\tmm"%(self.r_in_pos_1))
        self.print_data("Swirl chamber Radius", "%.3f
\tmm"%(self.r_s_1))
        self.print_data("Nozzle Radius", "%.3f \tmm"%(self.r_n_1))

        self.print_data("Nozzle Length", "%.3f \tmm"%(self.l_n_1))
        self.print_data("Swirl Chamber Length", "%.3f
\tmm"%(self.l_s_1))
        self.print_data("Inlet Channel Length", "%.3f
\tmm"%(self.l_in_1))

        print("\n\nST1 Injector Properties: ")
        self.print_data("Propellant", "%s"%(self.fr.propellant_1))
        self.print_data("Delta P", "%.2f bar"%(self.fr.delta_p_1))
        self.print_data("Mass flow target", "%.2f g/s"%(self.fr.m_1))

        self.print_data("A", "%.4f"%(self.A_1))
        self.print_data("A_eq", "%.4f"%(self.A_eq_1))
        self.print_data("K", "%.4f"%(self.K_1))
        self.print_data("Phi", "%.4f"%(self.phi_1))
        self.print_data("Phi_eq", "%.4f"%(self.phi_eq_1))
        self.print_data("a", "%.4f"%(self.a_1))
        self.print_data("a_eq", "%.4f"%(self.a_eq_1))
        self.print_data("Ksi", "%.4f"%(self.ksi_1))
        self.print_data("Lambda", "%.4f"%(self.lambda_1))
        self.print_data("mu_i", "%.4f"%(self.mu_i_1))
        self.print_data("C", "%.4f"%(self.C_1))
        #self.print_data("2Alpha ideal", "%.2f deg"%(self.alpha2_1))
        #self.print_data("2Alpha real", "%.2f
deg"%(self.alpha2_eq_1))
        #self.print_data("2Alpha ideal jusante", "%.2f
deg"%(self.alpha2_e_1))
        #self.print_data("2Alpha real jusante", "%.2f
deg"%(self.alpha2_e_eq_1))
        self.print_data("2Alpha_avg", "%.2f deg"%(self.alpha2_avg_1))

        self.print_data("r_mk_1", "%.2f mm"%(self.r_mk_1))
        self.print_data("r_mn_1", "%.2f mm"%(self.r_mn_1))
        self.print_data("t_fluid_1", "%.2f mm"%(self.t_fluid_1))
        self.print_data("Expected non-uniformity", "%.2f
%%"%(self.I_1))

```

```

##### ST1 - Convergence plot #####

self.step_x = np.ceil(self.contador/10)

if(self.sh_plot_error == 1):
    fig, (ax1, ax2) = plt.subplots(2, dpi = self.dpi_value)
    fig.suptitle("Stage 1 - Solution convergence")

    ax1.plot(range(self.contador),self.inlet_diameter)
    ax1.set_xlabel("Iteration")
    ax1.set_ylabel("d_in [mm]")

ax1.set_xticks(np.arange(0,self.contador+self.step_x,self.step_x))
ax1.grid()

    ax2.plot(range(self.contador),self.outlet_error)
    ax2.set_xlabel("Iteration")
    ax2.set_ylabel("R_n error [ad.]")

ax2.set_xticks(np.arange(0,self.contador+self.step_x,self.step_x))
ax2.grid()

##### ##### ##### ##### ##### ##### ##### ##### ##### #####
##### ##### #####
##### ##### ##### Parte 2 - Cálculo do ST2 ##### ##### #####
##### ##### #####
##### ##### ##### ##### ##### ##### ##### ##### ##### #####
##### ##### #####

if(self.fr.config == "Bi\n"):

    self.phi_2 = self.calc_phi(self.fr.alpha2_2)

    self.A_2 = self.calc_geom_char(self.phi_2)

    self.mu_2 = self.calc_mu(self.phi_2)

    self.r_n_2 = 1e3*0.475*np.sqrt(self.fr.m_2*1e-3 /
(self.mu_2 * np.sqrt(self.fr.rho_2*self.fr.delta_p_2*1e5)))

    print("\n\n\n\n\n ST2 Initial Values:")
    self.print_data("A_2", "%.2f"%(self.A_2))
    self.print_data("Phi_2", "%.3f"%(self.phi_2))
    self.print_data("R_n_2", "%.3f mm"%(self.r_n_2))
    self.print_data("2̑", "%.1f deg"%(self.fr.alpha2_2))

    self.inlet_diameter = []
    self.outlet_error = []

    self.erro_r = 100
    self.contador = 0

    if(self.show_it == 1): print("\n ST2 Iteration data:")
    while self.erro_r >= self.erro_max and self.contador <
self.it_lim:

        ##### Iteração do injetor ideal #####

```

```

        self.r_in_pos_2 = self.fr.opening_coef_2*self.r_n_2

        self.r_in_orf_2 =
np.sqrt((self.r_in_pos_2*self.r_n_2)/(self.fr.n_2*self.A_2))

        self.inlet_diameter.append(2*self.r_in_orf_2)

        self.r_s_2 = self.r_in_pos_2+self.r_in_orf_2

        self.l_in_2 = self.fr.ratio_l_in_2*2*self.r_in_orf_2

        self.l_n_2 = self.fr.ratio_l_n_2*2*self.r_n_2

        self.l_s_2 = self.fr.ratio_l_s_2*2*self.r_s_2

        self.a_2 = self.calc_a(self.phi_2)

        self.alpha2_2 = self.calc_2alpha(self.phi_2, "phi")

        self.alpha2_e_2 = self.calc_2alpha(self.a_2, "a")


        ##### Iteração do injetor real #####

        self.rey_in_2 = self.calc_reynolds_in(self.fr.m_2,
self.fr.din_visc_2, self.fr.n_2, self.r_in_orf_2, self.fr.rho_2)

        self.lambda_2 =
self.calc_friction_coef(self.rey_in_2)

        self.ksi_2 = self.calc_hyd_loss(self.fr.in_type_2,
self.r_s_2, self.l_in_2, self.fr.inlet_radius_2, self.r_in_orf_2,
self.lambda_2)

        self.check_value("ksi_2", 0, float("inf"),
self.ksi_2)

        self.A_eq_2 = self.calc_geom_char_eq(self.r_in_pos_2,
self.r_n_2, self.fr.n_2, self.r_in_orf_2, self.lambda_2)

        self.phi_eq_2 = self.calc_phi_eq(self.A_eq_2)

        self.mu_eq_2 = self.calc_mu(self.phi_eq_2)

        self.mu_i_2 = self.calc_mu_i(self.mu_eq_2, self.A_2,
self.r_in_pos_2, self.r_n_2, self.ksi_2)

        self.a_eq_2 = self.calc_a(self.phi_eq_2)

        self.alpha2_eq_2 = self.calc_2alpha(self.phi_eq_2,
"phi")

        self.alpha2_e_eq_2 = self.calc_2alpha(self.a_eq_2,
"a")


        ## cálculo do novo raio do bocal

        self.r_n_old = self.r_n_2

```

```

        self.r_n_2 = 1e3*0.475*np.sqrt(self.fr.m_2*1e-3 /
(self.mu_i_2 * np.sqrt(self.fr.rho_2*self.fr.delta_p_2*1e5)))

        self.erro_r = abs(self.r_n_old-
self.r_n_2)/self.r_n_old

        self.outlet_error.append(self.erro_r)

        self.A_2 =
self.r_in_pos_2*self.r_n_2/(self.fr.n_2*self.r_in_orf_2**2)

        self.phi_2 = self.calc_phi_eq(self.A_2)

        self.K_2 = self.A_eq_2/self.A_2
        self.check_value("K_2", 0, 1, self.K_2)

        self.r_mn_2 = self.r_n_2*np.sqrt(1-self.phi_eq_2)

        self.r_mk_2 = self.r_mn_2 * np.sqrt(2*(1-
self.phi_eq_2)/(2-self.phi_eq_2))

        self.t_fluid_2 = self.r_n_2-self.r_mn_2

        self.C_2 = self.r_in_pos_2/self.r_n_2

        self.alpha2_avg_2 = self.calc_2alpha_avg(self.mu_i_2,
self.A_eq_2, self.phi_eq_2, self.ksi_2, self.C_2)

    ## print dos dados da iteração
    if(self.show_it == 1):
        print("\nIt. %d"%(self.contador+1))
        self.print_data("A_2", "%.4f"%(self.A_2))
        self.print_data("A_eq_2", "%.4f"%(self.A_eq_2))
        self.print_data("K_2", "%.4f"%(self.K_2))
        self.print_data("Phi_2", "%.4f"%(self.phi_2))
        self.print_data("Phi_eq_2",
"%0.4f"%(self.phi_eq_2))

        self.print_data("a_2", "%.4f"%(self.a_2))
        self.print_data("a_eq_2", "%.4f"%(self.a_eq_2))
        self.print_data("rey_in_2",
"%0.4f"%(self.rey_in_2))

        self.print_data("ksi_2", "%.4f"%(self.ksi_2))
        self.print_data("lambda_2",
"%0.4f"%(self.lambda_2))

        self.print_data("mu_i", "%.4f"%(self.mu_i_2))
        self.print_data("C", "%.4f"%(self.C_2))
        #self.print_data("2Alpha ideal", "%.2f
deg"%(self.alpha2_2))

        #self.print_data("2Alpha real", "%.2f
deg"%(self.alpha2_eq_2))

        #self.print_data("2Alpha ideal jusante", "%.2f
deg"%(self.alpha2_e_2))

        #self.print_data("2Alpha real jusante", "%.2f
deg"%(self.alpha2_e_eq_2))

        self.print_data("2Alpha_avg", "%.2f
deg"%(self.alpha2_avg_2))

        self.print_data("R_n_2", "%.3f mm"%(self.r_n_2))
        self.print_data("r_in_2", "%.3f
mm"%(self.r_in_orf_2))

```

```

        self.print_data("Error", "%.2E"%(self.erro_r))

        self.contador += 1

        print("\n\n")

        ##Cálculo da não-uniformidade esperada do injetor
        self.I_2 = self.calc_non_uniformity(self.r_in_pos_2,
        self.r_n_2, self.fr.n_2, self.l_s_2, self.r_s_2, self.phi_eq_2)

        if(self.contador == self.it_lim):
            print ("\tWarning:\n STL reached iteration limit.")

            if(np.pi*self.r_n_2**2 <
self.fr.n_2*np.pi*self.r_in_orf_2**2):
                print("\tWarning:\n Outlet area smaller than total
inlet area (dif = %.2f mm²)"%(self.fr.n_2*np.pi*self.r_in_orf_2**2 -
np.pi*self.r_n_2**2))
                print("\tArea_in = %.2f mm²; \tArea_out = %.2f
mm²"%(self.fr.n_2*np.pi*self.r_in_orf_2**2, np.pi*self.r_n_2**2))

            ##### ST2 design console report #####

            print("\n\nST2 Injector Geometry: \n")
            self.print_data("# of Inlet Channels",
            "%.1f"%(self.fr.n_2))
            self.print_data("Inlet Channel Radius", "%.3f
\tmm"%(self.r_in_orf_2))
            self.print_data("Inlet Radius", "%.3f
\tmm"%(self.r_in_pos_2))
            self.print_data("Swirl chamber Radius", "%.3f
\tmm"%(self.r_s_2))
            self.print_data("Nozzle Radius", "%.3f
\tmm"%(self.r_n_2))

            self.print_data("Nozzle Length", "%.3f
\tmm"%(self.l_n_2))
            self.print_data("Swirl Chamber Length", "%.3f
\tmm"%(self.l_s_2))
            self.print_data("Inlet Channel Length", "%.3f
\tmm"%(self.l_in_2))

            print("\n\nST2 Injector Properties: ")
            self.print_data("Propellant",
            "%s"%(self.fr.propellant_2))
            self.print_data("Delta P", "%.2f
bar"%(self.fr.delta_p_2))
            self.print_data("Mass flow target", "%.2f
g/s"%(self.fr.m_2))

            self.print_data("A", "%.4f"%(self.A_2))
            self.print_data("A_eq", "%.4f"%(self.A_eq_2))
            self.print_data("K", "%.4f"%(self.K_2))
            self.print_data("Phi", "%.4f"%(self.phi_2))
            self.print_data("Phi_eq", "%.4f"%(self.phi_eq_2))
            self.print_data("a", "%.4f"%(self.a_2))
            self.print_data("a_eq", "%.4f"%(self.a_eq_2))
            self.print_data("Ksi", "%.4f"%(self.ksi_2))
            self.print_data("Lambda", "%.4f"%(self.lambda_2))

```

```

        self.print_data("mu_i", "%.4f"%(self.mu_i_2))
        self.print_data("C", "%.4f"%(self.C_2))
        #self.print_data("2Alpha ideal", "%.2f
deg"%(self.alpha2_2))
        #self.print_data("2Alpha real", "%.2f
deg"%(self.alpha2_eq_2))
        #self.print_data("2Alpha ideal jusante", "%.2f
deg"%(self.alpha2_e_2))
        #self.print_data("2Alpha real jusante", "%.2f
deg"%(self.alpha2_e_eq_2))
        self.print_data("2Alpha_avg", "%.2f
deg"%(self.alpha2_avg_2))

        self.print_data("r_mk_2", "%.2f mm"%(self.r_mk_2))
        self.print_data("r_mn_2", "%.2f mm"%(self.r_mn_2))
        self.print_data("t_fluid_2", "%.2f mm"%(self.t_fluid_2))
        self.print_data("Expected non-uniformity", "%.2f
%%"%(self.I_2))

##### ST2 - Convergence plot #####

self.step_x = np.ceil(self.contador/10)

if(self.sh_plot_error == 1):
    fig, (ax3, ax4) = plt.subplots(2, dpi =
self.dpi_value)

    fig.suptitle("Stage 2 - Solution convergence")

    ax3.plot(range(self.contador),self.inlet_diameter)
    ax3.set_xlabel("Iteration")
    ax3.set_ylabel("d_in [mm]")

    ax3.set_xticks(np.arange(0,self.contador+self.step_x,self.step_x))
    ax3.grid()

    ax4.plot(range(self.contador),self.outlet_error)
    ax4.set_xlabel("Iteration")
    ax4.set_ylabel("R_n error [ad.]")

    ax4.set_xticks(np.arange(0,self.contador+self.step_x,self.step_x))
    ax4.grid()

##### ##### ##### ##### ##### ##### ##### ##### #####
##### ##### #####
##### ##### ##### Parte 3 - Integração dos Estágios #
##### ##### #####
##### ##### ##### ##### ##### ##### ##### ##### #####
##### ##### ##### #####

self.gap = self.r_mk_2 - (self.r_n_1 + self.fr.t_w)

self.fluid_t_2_s = self.r_s_2 - self.r_mk_2

print("\n\nGeometry Check: \t")
self.print_data("D_n_1", "%.3f mm"%(2*self.r_n_1))
self.print_data("D_w_1", "%.3f
mm"%(2*self.r_n_1+2*self.fr.t_w))
self.print_data("D_n_2", "%.3f mm"%(2*self.r_n_2))

```

```

        self.print_data("fluid_t_2_s", "%.3f
mm"%(self.fluid_t_2_s))

        if(self.gap >= self.fr.delta/100*self.fluid_t_2_s):
            print("\n\tHydraulic Independance OK! \n\tDeltaR =
%.3f mm (mín: %.2f mm)"%(self.gap, self.fr.delta/100*self.fluid_t_2_s))

        else:
            print("\n\tERROR: Hydraulic Independance FAIL!
\n\tDeltaR = %.3f mm (mín: %.2f mm)"%(self.gap,
self.fr.delta/100*self.fluid_t_2_s))
            self.check_out = 0

            #####
            # Alteração do ângulo do ST2 #
            #####

        if(self.fr.angle_dif == "disabled"):
            self.check_out = -1

        else:

            self.alpha2_goal = 2*(self.alpha2_e_eq_1/2 -
self.fr.angle_dif)

            if(self.alpha2_goal < self.alpha2_e_eq_2):

                self.r_out =
self.r_n_2*np.sqrt(self.a_2*(1+1/(np.tan(self.alpha2_goal/2*np.pi/180))**2)
)

                self.l_out =
self.fr.ratio_l_n_out*2*self.r_n_2

                self.phi_out = self.phi_2/np.sqrt(3-
2*self.phi_2)

                self.r_mn_out = self.r_out*np.sqrt(1-
self.phi_out)

                self.t_fluid_out = self.r_out-self.r_mn_out

                self.u_tot_2 = (self.fr.m_2*1e-
3)/(self.fr.rho_2*self.fr.n_2*np.pi*(self.r_in_orf_2*1e-3)**2)

                self.u_wave_2 = np.sqrt(1-self.a_2) *
self.u_tot_2

                self.u_a_out = np.sqrt(1-
self.a_2/(self.r_out/self.r_n_2)**2) * self.u_tot_2

                #u_th = u_an = "axial velocity in the nozzle"
                self.u_an_2 = np.sqrt(self.phi_eq_2/(2-
self.phi_eq_2)) * self.u_tot_2

                self.lambda_i = self.u_a_out/self.u_an_2

```

```

self.lambda_i_max = np.sqrt((2-
self.phi_2)/self.phi_2)

        if(self.lambda_i > self.lambda_i_max):
            print("Error, Lambda exceeds maximum
allowable value (Outlet radius too large)!")
            self.print_data("lambda_i",
"%0.3f"%(self.lambda_i))
            self.print_data("lambda_i_max",
"%0.3f"%(self.lambda_i_max))

        if(self.r_out <= self.r_n_2):

            print("\n\tError: Invalid ST2 exit
radius.")

            self.check_out = -1

        else:
            print("\n\nST2 Modified Injector
Geometry: \t")
            self.print_data("2Alpha_2_goal", "%0.1f
deg"%(self.alpha2_goal))
            self.print_data("Phi_out",
"%0.3f"%(self.phi_out))
            self.print_data("R_out", "%0.3f
mm"%(self.r_out))
            self.print_data("L_out", "%0.3f
mm"%(self.l_out))
            self.print_data("R_mn_out", "%0.3f
mm"%(self.r_mn_out))
            self.print_data("t_fluid_out", "%0.3f
mm"%(self.t_fluid_out))
            self.print_data("u_tot_2", "%0.3f
m/s"%(self.u_tot_2))
            self.print_data("u_wave_2", "%0.3f
m/s"%(self.u_wave_2))
            self.print_data("u_a_out", "%0.3f
m/s"%(self.u_a_out))
            self.print_data("u_an_2", "%0.3f
m/s"%(self.u_an_2))
            self.print_data("lambda_i",
"%0.3f"%(self.lambda_i))
            self.print_data("lambda_i_max",
"%0.3f"%(self.lambda_i_max))

            self.check_out = 1

        else:
            print("\n\tWarning: Intercepting angle
greater than input. \n\tInt_angle = %0.1f deg"%(self.alpha2_eq_1/2 -
self.alpha2_eq_2/2))

            self.check_out = -1

```

```

##### ##### ##### ##### #
##### Injector plot #####
##### ##### ##### ##### #

if(self.fr.config == "Mono\n"):

    self.alpha2_plot_1 = self.alpha2_avg_1

    ##### conventional injector plot #####

    ## ST1

    self.st1_n_bottom = 0
    self.st1_n_top = self.st1_n_bottom + self.l_n_1
    self.st1_s_bottom = self.st1_n_top + (self.r_s_1-
self.r_n_1)/np.tan(self.fr.trans_angle_1*np.pi/180)
    self.st1_s_top = self.st1_s_bottom + self.l_s_1

    self.st1_x = [self.r_n_1, self.r_n_1, self.r_s_1,
self.r_s_1, 0]
    self.st1_y = [self.st1_n_bottom, self.st1_n_top,
self.st1_s_bottom, self.st1_s_top, self.st1_s_top]

    ## pontos do orifício tangencial do ST1

    self.st1_in_top = []
    self.st1_in_bottom = []

    self.circ_x_1 = np.arange(self.r_s_1-
2*self.r_in_orf_1,self.r_s_1,0.005)
    for x in self.circ_x_1:
        self.st1_in_top.append((self.st1_s_top-
self.r_in_orf_1) + np.sqrt(abs(self.r_in_orf_1**2-(x-(self.r_s_1-
self.r_in_orf_1))**2)))
        self.st1_in_bottom.append((self.st1_s_top-
self.r_in_orf_1) - np.sqrt(abs(self.r_in_orf_1**2-(x-(self.r_s_1-
self.r_in_orf_1))**2)))

    ## pontos do fluido interno do st1
    self.st1_fluid_n_bottom = 0
    self.st1_fluid_n_top = self.st1_n_bottom + self.l_n_1
    self.st1_fluid_s_bottom = self.st1_n_top + (self.r_s_1-
self.r_n_1)/np.tan(self.fr.trans_angle_1*np.pi/180)
    self.st1_fluid_s_top = self.st1_s_bottom + self.l_s_1

    self.st1_fluid_x = [self.r_mn_1, self.r_mn_1,
self.r_mk_1, self.r_mk_1]
    self.st1_fluid_y = [self.st1_fluid_n_bottom,
self.st1_fluid_n_top, self.st1_fluid_s_bottom, self.st1_fluid_s_top]

    ## pontos do spray do st1

    self.st1_spray_y_i = 0
    self.st1_spray_y_f = -1*self.st1_s_top*1.1
    self.st1_spray_y_step = (self.st1_spray_y_f-
self.st1_spray_y_i)/100
    self.st1_spray_y =
np.arange(self.st1_spray_y_i,self.st1_spray_y_f+self.st1_spray_y_step,self.
st1_spray_y_step)

```

```

self.st1_spray_x = []

self.r_av_1 = (self.r_n_1 + self.r_mn_1)/2

for y in self.st1_spray_y:

self.st1_spray_x.append(self.r_av_1*np.sqrt(1+y**2/(self.r_av_1*np.tan((90-
0.5*self.alpha2_plot_1)*np.pi/180))**2))

    if(self.sh_plot_inj == 1):

        fig, ax5 = plt.subplots(dpi = self.dpi_value)
        ax5.plot(self.st1_x,self.st1_y, linewidth = 2, color
= '0.3', linestyle = '-')
        ax5.plot(self.circ_x_1,self.st1_in_top, linewidth =
1, color = '0.3', linestyle = '-')
        ax5.plot(self.circ_x_1,self.st1_in_bottom, linewidth
= 1, color = '0.3', linestyle = '-')
        ax5.plot(self.st1_fluid_x, self.st1_fluid_y,
linewidth = 1, color = 'b', linestyle = '--')
        ax5.plot(self.st1_spray_x, self.st1_spray_y,
linewidth = 2, color = 'b', linestyle = '--')
        ax5.grid()
        ax5.set_xlabel("[mm]")
        ax5.set_ylabel("[mm]")

        self.x_min = 0
        self.x_max = 2*self.st1_s_top*1.1
        self.y_min = -1*self.st1_s_top*1.1
        self.y_max = self.st1_s_top*1.1

        ax5.set_xlim(self.x_min, self.x_max)
        ax5.set_ylim(self.y_min, self.y_max)
        ax5.set_aspect("equal")

        self.st1_legend = "ST1:\n   R_n: %.2f mm\n   R_in:
%.2f mm\n   R_s: %.2f mm\n   r_in: %.2f mm\n   n_in: %d \n"%(self.r_n_1,
self.r_in_pos_1, self.r_s_1, self.r_in_orf_1, self.fr.n_1)
        self.bbox_props = dict(boxstyle="round", fc="w",
ec="0.5", alpha=0.9)
        ax5.text(self.r_s_1*1.5, 0, self.st1_legend,
fontsize=self.fontsize, ha="left", va="bottom", bbox=self.bbox_props)

##### plots para injetores bipropelente #####

elif(self.fr.config == "Bi\n"):

    self.alpha2_plot_1 = self.alpha2_avg_1
    self.alpha2_plot_2 = self.alpha2_avg_2

    if(self.sh_plot_inj == 1 and (self.check_out == 1 or
self.check_out == -1)):
        ##### conventional injector plot #####

        ## ST1

```

```

        self.stl_n_bottom = self.fr.recess
        self.stl_n_top = self.stl_n_bottom + self.l_n_1
        self.stl_s_bottom = self.stl_n_top + (self.r_s_1-
self.r_n_1)/np.tan(self.fr.trans_angle_1*np.pi/180)
        self.stl_s_top = self.stl_s_bottom + self.l_s_1

        self.stl_x = [self.r_n_1, self.r_n_1, self.r_s_1,
self.r_s_1, 0]
        self.stl_y = [self.stl_n_bottom, self.stl_n_top,
self.stl_s_bottom, self.stl_s_top, self.stl_s_top]

        ## pontos do orifício tangencial do ST1

        self.stl_in_top = []
        self.stl_in_bottom = []

        self.circ_x_1 = np.arange(self.r_s_1-
2*self.r_in_orf_1,self.r_s_1,0.005)
        for x in self.circ_x_1:
            self.stl_in_top.append((self.stl_s_top-
self.r_in_orf_1) + np.sqrt(abs(self.r_in_orf_1**2-(x-(self.r_s_1-
self.r_in_orf_1))**2)))
            self.stl_in_bottom.append((self.stl_s_top-
self.r_in_orf_1) - np.sqrt(abs(self.r_in_orf_1**2-(x-(self.r_s_1-
self.r_in_orf_1))**2)))

        ## pontos do fluido interno do st1
        self.stl_fluid_n_bottom = self.fr.recess
        self.stl_fluid_n_top = self.stl_n_bottom + self.l_n_1
        self.stl_fluid_s_bottom = self.stl_n_top +
(self.r_s_1-self.r_n_1)/np.tan(self.fr.trans_angle_1*np.pi/180)
        self.stl_fluid_s_top = self.stl_s_bottom + self.l_s_1

        self.stl_fluid_x = [self.r_mn_1, self.r_mn_1,
self.r_mk_1, self.r_mk_1]
        self.stl_fluid_y = [self.stl_fluid_n_bottom,
self.stl_fluid_n_top, self.stl_fluid_s_bottom, self.stl_fluid_s_top]

        ## pontos do spray do st1
        self.stl_spray_y_i = 0
        self.stl_spray_y_f = -1*self.stl_s_top*1.1 -
self.fr.recess
        self.stl_spray_y_step = (self.stl_spray_y_f-
self.stl_spray_y_i)/100
        self.stl_spray_y =
np.arange(self.stl_spray_y_i,self.stl_spray_y_f+self.stl_spray_y_step,self.
stl_spray_y_step)

        self.stl_spray_x = []

        self.r_av_1 = (self.r_n_1 + self.r_mn_1)/2

        for y in self.stl_spray_y:
            self.stl_spray_x.append(self.r_av_1*np.sqrt(1+y**2/(self.r_av_1*np.tan((90-
0.5*self.alpha2_plot_1)*np.pi/180))**2))

```

```

        for i in range(len(self.st1_spray_y)):
            self.st1_spray_y[i] = self.st1_spray_y[i] +
self.fr.recess

    ## ST2

    self.st2_n_bottom = 0
    self.st2_n_top = self.st2_n_bottom + self.l_n_2
    self.st2_s_bottom = self.st2_n_top + (self.r_s_2-
self.r_n_2)/np.tan(self.fr.trans_angle_2*np.pi/180)
    self.st2_s_top = self.st2_s_bottom + self.l_s_2

    ## pontos da estrutura do ST2
    self.st2_x = [self.r_n_2, self.r_n_2, self.r_s_2,
self.r_s_2, self.r_n_1+self.fr.t_w, self.r_n_1+self.fr.t_w, self.r_n_1]
    self.st2_y = [self.st2_n_bottom, self.st2_n_top,
self.st2_s_bottom, self.st2_s_top, self.st2_s_top, self.fr.recess,
self.fr.recess]

    ## pontos do orifício tangencial do ST2

    self.st2_in_top = []
    self.st2_in_bottom = []

    self.circ_x_2 = np.arange(self.r_s_2-
2*self.r_in_orf_2,self.r_s_2,0.005)
    for x in self.circ_x_2:
        self.st2_in_top.append((self.st2_s_top-
self.r_in_orf_2) + np.sqrt(abs(self.r_in_orf_2**2-(x-(self.r_s_2-
self.r_in_orf_2))**2)))
        self.st2_in_bottom.append((self.st2_s_top-
self.r_in_orf_2) - np.sqrt(abs(self.r_in_orf_2**2-(x-(self.r_s_2-
self.r_in_orf_2))**2)))

    ## pontos do fluido interno do ST2
    self.st2_fluid_n_bottom = 0
    self.st2_fluid_n_top = self.st2_fluid_n_bottom +
self.l_n_2
    self.st2_fluid_s_bottom = self.st2_fluid_n_top +
(self.r_s_2-self.r_n_2)/np.tan(self.fr.trans_angle_2*np.pi/180)
    self.st2_fluid_s_top = self.st2_fluid_s_bottom +
self.l_s_2

    self.st2_fluid_x = [self.r_mn_2, self.r_mn_2,
self.r_mk_2, self.r_mk_2]
    self.st2_fluid_y = [self.st2_fluid_n_bottom,
self.st2_fluid_n_top, self.st2_fluid_s_bottom, self.st2_fluid_s_top]

    ## pontos do spray do ST2

    self.st2_spray_y_i = 0
    self.st2_spray_y_f = -1*self.st1_s_top*1.1
    self.st2_spray_y_step = (self.st2_spray_y_f-
self.st2_spray_y_i)/100

```

```

        self.st2_spray_y =
np.arange(self.st2_spray_y_i,self.st2_spray_y_f+self.st2_spray_y_step,self.
st2_spray_y_step)

        self.st2_spray_x = []

        self.r_av_2 = (self.r_n_2 + self.r_mn_2)/2

        for y in self.st2_spray_y:

self.st2_spray_x.append(self.r_av_2*np.sqrt(1+y**2/(self.r_av_2*np.tan((90-
0.5*self.alpha2_plot_2)*np.pi/180))**2))


        fig, ax6 = plt.subplots(dpi = self.dpi_value)

        ax6.plot(self.st1_x,self.st1_y, linewidth = 1, color
= '0.3', linestyle = '-')
        ax6.plot(self.circ_x_1,self.st1_in_top, linewidth =
1, color = '0.3', linestyle = '-')
        ax6.plot(self.circ_x_1,self.st1_in_bottom, linewidth
= 1, color = '0.3', linestyle = '-')
        ax6.plot(self.st1_fluid_x, self.st1_fluid_y,
linewidth = 1, color = 'b', linestyle = '--')
        ax6.plot(self.st1_spray_x, self.st1_spray_y,
linewidth = 2, color = 'b', linestyle = '--')

        ax6.plot(self.st2_x,self.st2_y, linewidth = 1, color
= '0.3', linestyle = '-')
        ax6.plot(self.circ_x_2,self.st2_in_top, linewidth =
1, color = '0.3', linestyle = '-')
        ax6.plot(self.circ_x_2,self.st2_in_bottom, linewidth
= 1, color = '0.3', linestyle = '-')
        ax6.plot(self.st2_fluid_x, self.st2_fluid_y,
linewidth = 1, color = 'y', linestyle = '--')
        ax6.plot(self.st2_spray_x, self.st2_spray_y,
linewidth = 2, color = 'y', linestyle = '--')

        ax6.grid()
        ax6.set_xlabel("[mm]")
        ax6.set_ylabel("[mm]")

        self.x_min = 0
        self.x_max = 2*self.st1_s_top*1.1
        self.y_min = -1*self.st1_s_top*1.1
        self.y_max = self.st1_s_top*1.1

        ax6.set_xlim(self.x_min, self.x_max)
        ax6.set_ylim(self.y_min, self.y_max)
        ax6.set_aspect("equal")

        self.st1_legend = "ST1:\n   R_n: %.2f mm\n   R_in:
%.2f mm\n   R_s: %.2f mm\n   r_in: %.2f mm\n   n_in: %d \n   recess: %.2f
mm"%(self.r_n_1, self.r_in_pos_1, self.r_s_1, self.r_in_orf_1, self.fr.n_1,
self.fr.recess)

```

```

        self.st2_legend = "ST2:\n   R_n: %.2f mm\n   R_in: %.2f mm\n   R_s: %.2f mm\n   r_in: %.2f mm\n   n_in: %d \n   int_angle: %.1f°"%(self.r_n_2, self.r_in_pos_2, self.r_s_2, self.r_in_orf_2, self.fr.n_2, self.alpha2_eq_1/2 - self.alpha2_eq_2/2)
        self.bbox_props = dict(boxstyle="round", fc="w", ec="0.5", alpha=0.9)
        ax6.text(max([self.r_s_1,self.r_s_2])*1.5, 0, self.st1_legend, fontsize=self.fontsize, ha="left", va="bottom", bbox=self.bbox_props)

ax6.text(max([self.r_s_1,self.r_s_2])*1.5+self.offset, 0, self.st2_legend, fontsize=self.fontsize, ha="left", va="bottom", bbox=self.bbox_props)


    if(self.check_out == 1):
        ##### modified injector plot #####

        ## ST1

        self.st1_n_bottom = self.fr.recess
        self.st1_n_top = self.st1_n_bottom + self.l_n_1
        self.st1_s_bottom = self.st1_n_top + (self.r_s_1-self.r_n_1)/np.tan(self.fr.trans_angle_1*np.pi/180)
        self.st1_s_top = self.st1_s_bottom + self.l_s_1

        self.st1_x = [self.r_n_1, self.r_n_1, self.r_s_1, self.r_s_1, 0]

        self.st1_y = [self.st1_n_bottom, self.st1_n_top, self.st1_s_bottom, self.st1_s_top, self.st1_s_top]

        ## pontos do orifício tangencial do ST1

        self.st1_in_top = []
        self.st1_in_bottom = []

        self.circ_x_1 = np.arange(self.r_s_1-2*self.r_in_orf_1,self.r_s_1,0.005)
        for x in self.circ_x_1:
            self.st1_in_top.append((self.st1_s_top-self.r_in_orf_1) + np.sqrt(abs(self.r_in_orf_1**2-(x-(self.r_s_1-self.r_in_orf_1))**2)))
            self.st1_in_bottom.append((self.st1_s_top-self.r_in_orf_1) - np.sqrt(abs(self.r_in_orf_1**2-(x-(self.r_s_1-self.r_in_orf_1))**2)))

        ## pontos do fluido interno do st1
        self.st1_fluid_n_bottom = self.fr.recess
        self.st1_fluid_n_top = self.st1_n_bottom + self.l_n_1

        self.st1_fluid_s_bottom = self.st1_n_top + (self.r_s_1-self.r_n_1)/np.tan(self.fr.trans_angle_1*np.pi/180)
        self.st1_fluid_s_top = self.st1_s_bottom + self.l_s_1

        self.st1_fluid_x = [self.r_mn_1, self.r_mn_1, self.r_mk_1, self.r_mk_1]

```

```

        self.st1_fluid_y = [self.st1_fluid_n_bottom,
self.st1_fluid_n_top, self.st1_fluid_s_bottom, self.st1_fluid_s_top]

        ## pontos do spray do st1
        self.st1_spray_y_i = 0
        self.st1_spray_y_f = -1*self.st1_s_top*1.1 -
self.fr.recess
        self.st1_spray_y_step = (self.st1_spray_y_f-
self.st1_spray_y_i)/100
        self.st1_spray_y =
np.arange(self.st1_spray_y_i,self.st1_spray_y_f+self.st1_spray_y_step,self.
st1_spray_y_step)

        self.st1_spray_x = []

        self.r_av_1 = (self.r_n_1 + self.r_mn_1)/2

        for y in self.st1_spray_y:

self.st1_spray_x.append(self.r_av_1*np.sqrt(1+y**2/(self.r_av_1*np.tan((90-
0.5*self.alpha2_plot_1)*np.pi/180))**2))

        for i in range(len(self.st1_spray_y)):
            self.st1_spray_y[i] = self.st1_spray_y[i] +
self.fr.recess

        ## ST2

        self.st2_out_bottom = 0
        self.st2_out_top = self.l_out
        self.st2_n_bottom = self.st2_out_top +
(self.r_out-self.r_n_2)/np.tan(self.fr.trans_angle_out*np.pi/180)
        self.st2_n_top = self.st2_n_bottom + self.l_n_2
        self.st2_s_bottom = self.st2_n_top + (self.r_s_2-
self.r_n_2)/np.tan(self.fr.trans_angle_2*np.pi/180)
        self.st2_s_top = self.st2_s_bottom + self.l_s_2

        ## pontos da estrutura do ST2
        self.st2_x = [self.r_out, self.r_out, self.r_n_2,
self.r_n_2, self.r_s_2, self.r_s_2, self.r_n_1+self.fr.t_w,
self.r_n_1+self.fr.t_w, self.r_n_1]
        self.st2_y = [self.st2_out_bottom,
self.st2_out_top, self.st2_n_bottom, self.st2_n_top, self.st2_s_bottom,
self.st2_s_top, self.st2_s_top, self.fr.recess, self.fr.recess]

        ## pontos do orifício tangencial do ST2

        self.st2_in_top = []
        self.st2_in_bottom = []

        self.circ_x_2 = np.arange(self.r_s_2-
2*self.r_in_orf_2,self.r_s_2,0.005)
        for x in self.circ_x_2:
            self.st2_in_top.append((self.st2_s_top-
self.r_in_orf_2) + np.sqrt(abs(self.r_in_orf_2**2-(x-(self.r_s_2-
self.r_in_orf_2))**2)))

```

```

self.st2_in_bottom.append((self.st2_s_top-
self.r_in_orf_2) - np.sqrt(abs(self.r_in_orf_2**2-(x-(self.r_s_2-
self.r_in_orf_2))**2)))

    ## pontos do fluido interno do ST2
    self.st2_fluid_out_bottom = 0
    self.st2_fluid_out_top = self.l_out
    self.st2_fluid_n_bottom = self.st2_out_top +
(self.r_out-self.r_n_2)/np.tan(self.fr.trans_angle_out*np.pi/180)
    self.st2_fluid_n_top = self.st2_n_bottom +
self.l_n_2
    self.st2_fluid_s_bottom = self.st2_n_top +
(self.r_s_2-self.r_n_2)/np.tan(self.fr.trans_angle_2*np.pi/180)
    self.st2_fluid_s_top = self.st2_s_bottom +
self.l_s_2

    self.st2_fluid_x = [self.r_mn_out, self.r_mn_out,
self.r_mn_2, self.r_mn_2, self.r_mk_2, self.r_mk_2]
    self.st2_fluid_y = [self.st2_fluid_out_bottom,
self.st2_fluid_out_top, self.st2_fluid_n_bottom, self.st2_fluid_n_top,
self.st2_fluid_s_bottom, self.st2_fluid_s_top]

    ## pontos do spray do ST2

    self.st2_spray_y_i = 0
    self.st2_spray_y_f = -1*self.st1_s_top*1.1
    self.st2_spray_y_step = (self.st2_spray_y_f-
self.st2_spray_y_i)/100
    self.st2_spray_y =
np.arange(self.st2_spray_y_i,self.st2_spray_y_f+self.st2_spray_y_step,self.
st2_spray_y_step)

    self.st2_spray_x = []

    self.r_av_2 = (self.r_out + self.r_mn_out)/2

    for y in self.st2_spray_y:

self.st2_spray_x.append(self.r_av_2*np.sqrt(1+y**2/(self.r_av_2*np.tan((90-
0.5*self.alpha2_goal)*np.pi/180))**2))

    fig, ax7 = plt.subplots(dpi = self.dpi_value)

    ax7.plot(self.st1_x,self.st1_y, linewidth = 1,
color = '0.3', linestyle = '-')
    ax7.plot(self.circ_x_1,self.st1_in_top, linewidth
= 1, color = '0.3', linestyle = '-')
    ax7.plot(self.circ_x_1,self.st1_in_bottom,
linewidth = 1, color = '0.3', linestyle = '-')
    ax7.plot(self.st1_fluid_x, self.st1_fluid_y,
linewidth = 1, color = 'b', linestyle = '--')
    ax7.plot(self.st1_spray_x, self.st1_spray_y,
linewidth = 2, color = 'b', linestyle = '--')

    ax7.plot(self.st2_x,self.st2_y, linewidth = 1,
color = '0.3', linestyle = '-')
    ax7.plot(self.circ_x_2,self.st2_in_top, linewidth
= 1, color = '0.3', linestyle = '-')

```

```

        ax7.plot(self.circ_x_2, self.st2_in_bottom,
linewidth = 1, color = '0.3', linestyle = '-')
        ax7.plot(self.st2_fluid_x, self.st2_fluid_y,
linewidth = 1, color = 'y', linestyle = '--')
        ax7.plot(self.st2_spray_x, self.st2_spray_y,
linewidth = 2, color = 'y', linestyle = '--')

        ax7.grid()
        ax7.set_xlabel("[mm]")
        ax7.set_ylabel("[mm]")

        self.x_min = 0
        self.x_max = 2*self.st1_s_top*1.1
        self.y_min = -1*self.st1_s_top*1.1
        self.y_max = self.st1_s_top*1.1

        ax7.set_xlim(self.x_min, self.x_max)
        ax7.set_ylim(self.y_min, self.y_max)
        ax7.set_aspect("equal")

        self.st1_legend = "ST1:\n   R_n: %.2f mm\n
R_in: %.2f mm\n   R_s: %.2f mm\n   r_in: %.2f mm\n   n_in: %d \n   recess:
%.2f mm"%(self.r_n_1, self.r_in_pos_1, self.r_s_1, self.r_in_orf_1,
self.fr.n_1, self.fr.recess)
        self.st2_legend = "ST2:\n   R_n: %.2f mm\n
R_in: %.2f mm\n   R_s: %.2f mm\n   r_in: %.2f mm\n   n_in: %d \n   R_out:
%.2f mm"%(self.r_n_2, self.r_in_pos_2, self.r_s_2, self.r_in_orf_2,
self.fr.n_2, self.r_out)
        self.bbox_props = dict(boxstyle="round", fc="w",
ec="0.5", alpha=0.9)
        ax7.text(max([self.r_s_1, self.r_s_2])*1.5, 0,
self.st1_legend, fontsize=self.fontsize, ha="left", va="bottom",
bbox=self.bbox_props)

        ax7.text(max([self.r_s_1, self.r_s_2])*1.5+self.offset, 0, self.st2_legend,
fontsize=self.fontsize, ha="left", va="bottom", bbox=self.bbox_props)

        elif(self.check_out == -1):
            print("\n\nPlot: Modified injector not
plotted\n\n\t(Angle modification not needed or disabled by user)")

        elif(self.check_out == 0):
            print("\n\nPlot: injector not plotted\n\n\t(Hydraulic
Independance FAIL!)" )

        elif(self.sh_plot_inj == 0):
            print("\n\nPlot: injector not plotted\n\n\t(Disabled by
user)")

        else:
            print("\n\nPlot: injector not plotted\n\n\t(Unknow
Error)")

        elif(self.sh_plot_inj == 1):
            print("Plot: configuration not recognized")

plt.show()

```

7.4 M1 Input File – RD0110: RD0110.txt

```
#####  
#File containing basic injector parameters for geometry calculation#####  
#####  
#####  
#Injector ID  
RD0110  
#Propellant Configuration (Mono or Bi)  
Bi  
#Stage separation wall thickness [mm]  
0.8  
#Interstage clearance [% of ST2 r_mk fluid thickness]  
10  
#Interstage 2Alpha angle difference [deg]  
disabled  
#Recess length [mm]  
1  
#ST1 Type of channel entrance (curved or straight)  
curved  
#ST1 Straight entrance radius [mm] (equal to 0.00 if entrance type is "curved")  
0.00  
#ST1 propellant  
Liquid Oxygen (100 K)  
#ST1 mass flow [g/s]  
172.95  
#ST1 DeltaP [bar]  
4.26  
#ST1 2Alpha start value [deg] (aprox. initial value) (ref: 126.5)  
110.45  
#ST1 number of entrance channels  
6
```

#ST1 Inlet Radius to Nozzle Radius Ratio (aprox. initial value)

1.30

#ST1 Conic Transition angle [deg]

45

#ST1 inlet channel L/D

1.0

#ST1 Swirl Chamber L/D

1.2

#ST1 Nozzle L/D

2.0

#ST2 Type of channel entrance (curved or straight)

curved

#ST2 Straight entrance radius [mm] (equal to 0.00 if entrance type is "curved")

0.00

#ST2 propellant

kerosene (293 K)

#ST2 mass flow [g/s]

64.8

#ST2 DeltaP [bar]

6.96

#ST2 2Alpha iteration initial value [deg]

146.50

#ST2 number of entrance channels

6

#ST2 Inlet Radius to Nozzle Radius Ratio

1.00

#ST2 Conic Transition angle [deg]

45

#ST2 inlet channel L/D

1.5

#ST2 Swirl Chamber L/D

0.45

#ST2 Nozzle L/D

0.50

#ST2 Modified nozzle L/D

0.00

#ST2 Modified nozzle transition angle [deg]

45

7.5 M1 Input File – RD0109: RD0109.txt

```
#####
#File containing basic injector parameters for geometry calculation#####
#####
#####
#Injector ID
RD0109
#Propellant Configuration (Mono or Bi)
Bi
#Stage separation wall thickness [mm]
0.8
#Interstage clearance [% of ST2 r_mk fluid thickness]
10
#Interstage 2Alpha angle difference [deg]
disabled
#Recess length [mm]
1
#ST1 Type of channel entrance (curved or straight)
curved
#ST1 Straight entrance radius [mm] (equal to 0.00 if entrance type is "curved")
0.00
#ST1 propellant
Liquid Oxygen (100 K)
#ST1 mass flow [g/s]
172.9
#ST1 DeltaP [bar]
4.40
#ST1 2Alpha start value [deg] (aprox. initial value) (ref: 126.5)
108.4
#ST1 number of entrance channels
6
```

#ST1 Inlet Radius to Nozzle Radius Ratio (aprox. initial value)

1.33

#ST1 Conic Transition angle [deg]

45

#ST1 inlet channel L/D

3.0

#ST1 Swirl Chamber L/D

1.00

#ST1 Nozzle L/D

2.00

#ST2 Type of channel entrance (curved or straigth)

curved

#ST2 Straigth entrance radius [mm] (equal to 0.00 if entrance type is "curved")

0.00

#ST2 propellant

kerosene (293 K)

#ST2 mass flow [g/s]

72.2

#ST2 DeltaP [bar]

7.10

#ST2 2Alpha iteration initial value [deg]

145.90

#ST2 number of entrance channels

6

#ST2 Inlet Radius to Nozzle Radius Ratio

1.00

#ST2 Conic Transition angle [deg]

45

#ST2 inlet channel L/D

3.0

#ST2 Swirl Chamber L/D

0.50

#ST2 Nozzle L/D

0.50

#ST2 Modified nozzle L/D

0.00

#ST2 Modified nozzle transition angle [deg]

45

7.6 M1 Input File – Injetor Proposto Central: B01_C.txt

```
#####
#File containing basic injector parameters for geometry calculation#####
#####
#####
#Injector ID
B01_C
#Propellant Configuration (Mono or Bi)
Bi
#Stage separation wall thickness [mm]
1.0
#Interstage clearance [% of ST2 r_mk fluid thickness]
30
#Interstage 2Alpha angle difference [deg]
0.0
#Recess length [mm]
2.00
#ST1 Type of channel entrance (curved or
straigh)#####
curved
#ST1 Straigh entrance radius [mm] (equal to 0.00 if entrance type is "curved")
0.00
#ST1 propellants
Liquid Oxygen (90 K)
#ST1 mass flow [g/s]
190.30
#ST1 DeltaP [bar]
6.00
#ST1 2Alpha start value [deg] (aprox. initial value) (ref: 126.5)
120
#ST1 number of entrance channels
6
```

#ST1 Inlet Radius to Nozzle Radius Ratio (aprox. initial value)

1.4

#ST1 Conic Transition angle [deg]

45

#ST1 inlet channel L/D

1.0

#ST1 Swirl Chamber L/D

1.25

#ST1 Nozzle L/D

2.00

#ST2 Type of channel entrance (curved or
straighth)#####

curved

#ST2 Straighth entrance radius [mm] (equal to 0.00 if entrance type is "curved")

0.00

#ST2 propellant

Ethanol 95% (300 K)

#ST2 mass flow [g/s]

95.1

#ST2 DeltaP [bar]

4.00

#ST2 2Alpha iteration initial value [deg]

140

#ST1 number of entrance channels

6

#ST2 Entrance Radius to Nozzle Radius Ratio

1.0

#ST1 Conic Transition angle [deg]

45

#ST2 inlet channel L/D

1.0

#ST2 Swirl Chamber L/D

0.5

#ST2 Nozzle L/D

0.25

#ST2 Modified nozzle L/D

0.25

#ST2 Modified nozzle transition angle [deg]

45

7.7 M1 Input File – Injetor Proposto Periférico: B01_W.txt

```
#####
#File containing basic injector parameters for geometry calculation#####
#####
#####
#Injector ID
B01_W
#Propellant Configuration (Mono or Bi)
Bi
#Stage separation wall thickness [mm]
1.0
#Interstage clearance [% of ST2 r_mk fluid thickness]
20
#Interstage 2Alpha angle difference [deg or disabled]
disabled
#Recess length [mm]
0.00
#ST1 Type of channel entrance (curved or
straigh)#####
curved
#ST1 Straigh entrance radius [mm] (equal to 0.00 if entrance type is "curved")
0.00
#ST1 propellants
Liquid Oxygen (90 K)
#ST1 mass flow [g/s]
20.80
#ST1 DeltaP [bar]
6.00
#ST1 2Alpha start value [deg] (aprox. initial value) (ref: 126.5)
115
#ST1 number of entrance channels
3
```

#ST1 Inlet Radius to Nozzle Radius Ratio (aprox. initial value)

1.20

#ST1 Conic Transition angle [deg]

45

#ST1 inlet channel L/D

4.00

#ST1 Swirl Chamber L/D

2.00

#ST1 Nozzle L/D

2.00

#ST2 Type of channel entrance (curved or
straigh)#####

curved

#ST2 Straigh entrance radius [mm] (equal to 0.00 if entrance type is "curved")

0.00

#ST2 propellant

Ethanol 95% (300 K)

#ST2 mass flow [g/s]

20.80

#ST2 DeltaP [bar]

4.00

#ST2 2Alpha iteration initial value [deg]

142

#ST1 number of entrance channels

3

#ST2 Entrance Radius to Nozzle Radius Ratio

1.05

#ST1 Conic Transition angle [deg]

45

#ST2 inlet channel L/D

2.00

#ST2 Swirl Chamber L/D

0.70

#ST2 Nozzle L/D

0.30

#ST2 Modified nozzle L/D

0.00

#ST2 Modified nozzle transition angle [deg]

45

8 ANEXO B – CÓDIGOS DO MÉTODO M2

8.1 M2_main.py

```
# -*- coding: utf-8 -*-
"""
@author: A. Goulart
"""

#####
#### Data File Selection ####
#####

#####
Foldername = "rev_B01/v_3"

##

Filename = "B01_C_Ox"
#Filename = "B01_C_F"

#Filename = "B01_W_Ox"
#Filename = "B01_W_F"
#####

##### OK
#Foldername = "RD0110/v_2"

##

#Filename = "RD0110_Ox"
#Filename = "RD0110_F"
#####

#####
#Foldername = "RD0109/v_2"

##

#Filename = "RD0109_Ox"
#Filename = "RD0109_F"
#####

from M2_Method import Method_2

Run_1 = Method_2(Foldername, Filename)

Run_1.set_it_lim(100)
Run_1.set_erro_max(1e-6)

Run_1.set_delta_p_i(0.50)
Run_1.set_delta_p_f(10.00)
Run_1.set_delta_p_step(0.05)

Run_1.set_lambda_type("Bazarov")
#Run_1.set_lambda_type("Bayvel")
```

```
Run_1.run_M2()
```

8.2 M2_FileReader.py

```
# -*- coding: utf-8 -*-
"""
    @author: A. Goulart
"""

class FileReader_2:
    def setFolderName(self, ffname):
        self.Foldername = ffname
    def setFileName(self, fname):
        self.Filename = fname
    def read(self):

        #####
        ##### Injector parameters reading #####
        #####

        print("\033[1m+"\n Filereader initialization...
\n"+" \033[0m")

        self.file_loc = str(self.Foldername) + '/' +
str(self.Filename) + '.txt'

        print("\nFile location: ", self.file_loc, "\n")

        self.f_len = sum(1 for line in open(str(self.file_loc)))

        self.Input = open(str(self.file_loc), 'r')

        for i in range(5): self.Input.readline()

        self.Injector_ID = str(self.Input.readline())
        print("Identificação do injetor: \t", self.Injector_ID)

        print("File Length: ", self.f_len)

        self.DataStream = []

        for i in range(self.f_len):
            self.Input.readline()
            self.DataStream.append(self.Input.readline())

        ##### Beginning of ST1 data #####

        self.inlet_type = str(self.DataStream[0])

        self.propellant = str(self.DataStream[1])

        self.m_0 = float(self.DataStream[2])

        self.delta_p0 = float(self.DataStream[3])
```

```

self.n = int(self.DataStream[4])

self.r_in_orf = float(self.DataStream[5])

self.l_in = float(self.DataStream[6])

self.inlet_radius = float(self.DataStream[7])

self.r_s = float(self.DataStream[8])

self.l_s = float(self.DataStream[9])

self.r_n = float(self.DataStream[10])

self.l_n = float(self.DataStream[11])

self.r_in_pos = float(self.DataStream[12])

#####
##### Propellant properties #####
#####

self.Input = open('prop_data.txt', 'r')

for i in range(4): self.Input.readline()

self.Input.readline()
self.n_propellants = int(self.Input.readline())

self.Input.readline()
self.n_properties = int(self.Input.readline())

self.Properties = []

for i in range(self.n_propellants):

    self.Properties.append([])

    self.Input.readline()
    self.Properties[i].append(str(self.Input.readline()))

    for j in range(self.n_properties-1):

        self.Input.readline()

self.Properties[i].append(float(self.Input.readline()))

    self.Input.readline()

self.prop_ST1 = []

##### properties ST1 #####

for i in range(self.n_propellants):
    if(self.Properties[i][0] == self.propellant):
        self.prop_ST1 = self.Properties[i]

self.rho = self.prop_ST1[1]

self.din_visc = self.prop_ST1[2]

```

```

self.visc_sup = self.prop_ST1[3]

print("\033[1m"+" \n File reading complete!  \n\n"+" \033[0m")

```

8.3 M2_Method.py

```

"""
@author: A. Goulart
"""

import numpy as np

import matplotlib.pyplot as plt

from M2_FileReader import FileReader_2

class Method_2:
    def __init__(self, foldername, filename):
        self.it_lim = 1000
        self.erro_max = 1e-6

        self.delta_p_i = 1.00 #bar
        self.delta_p_f = 10.0 #bar
        self.delta_p_step = 0.01
        self.lambdaType = "Bazarov"

        self.fr = FileReader_2()
        self.fr.setFolderName(foldername)
        self.fr.setFileName(filename)

        self.fmt = '| | {{: ^{s}}} | {{: ^{s}}} |'.format(25, 15) #
widths only

    def set_lambda_type(self, typeName):
        if (typeName == "Bazarov"):
            self.lambdaType = typeName
        elif (typeName == "Bayvel"):
            self.lambdaType = typeName
        else:
            print("ERROR: lambda type not understood")

    def set_delta_p_i(self, n):
        if (n == 0):
            self.delta_p_i = 0.01
        else:
            self.delta_p_i = n

    def get_delta_p_i(self):
        print("delta_p_i = ", self.delta_p_i)

    def set_delta_p_f(self, n):
        self.delta_p_f = n

```

```

def get_delta_p_f(self):
    print("delta_p_f = ", self.delta_p_f)

def set_delta_p_step(self,n):
    self.delta_p_step = n

def get_delta_p_step(self):
    print("delta_p_step = ", self.delta_p_step)

def set_it_lim(self,n):
    self.it_lim = n

def get_it_lim(self):
    print("it_lim = ", self.it_lim)

def set_erro_max(self,n):
    self.erro_max = n

def get_erro_max(self):
    print("erro_max = ", self.erro_max)

def print_data(self, label, value):
    print(self.fmt.format(str(label), str(value)))

#####      #####      #####      #####      #####      #####
##### Métodos para cálculo do injetore #####
#####      #####      #####      #####      #####      #####

def calc_phi(self, geom_char):
    self.coefs = [geom_char**2, -2, 4, -2]
    self.roots = np.roots(self.coefs)

    self.count = 0

    self.phi_eq = -1

    for root in self.roots:
        if(0 < root < 1 and root.imag == 0):
            self.count+=1
            self.phi = root.real

    if(self.phi == -1):
        print("Error in calc_phi:\n No valid root found.")

    if(self.count > 1):
        print("Error in calc_phi:\n Multiple valid values found")
        print(self.roots)

    return self.phi

def calc_geom_char(self,r_in_pos, r_n, n, r_in_orf,
friction_coef):
    return r_in_pos*1e-3*r_n*1e-3/(n*(r_in_orf*1e-
3)**2+friction_coef/2*r_in_pos*1e-3*(r_in_pos*1e-3-r_n*1e-3))

def calc_2alpha(self, phi):
    return 2*180/np.pi*np.arctan(np.sqrt(2*(1-phi)/phi))

```

```

def calc_reynolds_in(self,m_in,din_visc,n,r_in_orf):
    return (2*m_in*1e-3)/(din_visc*n*np.pi*(r_in_orf*1e-3))

def calc_friction_coef(self, reynolds_in):
    if(self.lambdaType == "Bazarov"):
        return 0.3164*reynolds_in**(-0.25) #Bazarov
    elif(self.lambdaType == "Bayvel"):
        return 10**(25.8/(np.log(reynolds_in)**2.58)-2)
#Bayvel
    else:
        print("ERROR: lambda type not understood")

    ## Calcula o valor da perda hidráulica no injetor (Ksi)
    def calc_hyd_loss(self, inlet_type, r_s, l_in, inlet_radius,
r_in_orf, friction_coef):
        if(inlet_type == "curved\n"):
            self.alpha_in = 90 - 180/np.pi * np.arctan(r_s/l_in)
            self.ksi_in = -1*self.alpha_in/150 + 1.1

        elif(inlet_type == "straighth\n"):
            self.ksi_in = 0.5*np.exp(-
1.4*(inlet_radius/(2*r_in_orf)))

        else:
            print("Error in calc_hyd_loss:\n Inlet type not
recognized")

            return self.ksi_in + friction_coef*l_in/(2*r_in_orf)

    ## calcula o parâmetro geométrico característico corrigido (A_eq
[Bazarov, eq(100)]
    def calc_geom_char_eq(self, r_in_pos, r_n, n, r_in_orf,
friction_coef):
        return
r_in_pos*r_n/(n*r_in_orf**2+friction_coef/2*r_in_pos*(r_in_pos-r_n))

    ## Calcula o coeficiente de descarga equivalente [Bazarov,
eq(99)]
    def calc_mu_eq(self, phi_eq):
        return phi_eq**1.5/np.sqrt(2-phi_eq)

    ## Calcula o coeficiente de descarga real do injetor [Bazarov,
eq(99)]
    def calc_mu_i(self, mu_eq, geom_char, r_in_pos, r_n, ksi):
        return
mu_eq/np.sqrt(1+ksi*mu_eq**2*(geom_char/(r_in_pos/r_n))**2)

    ##### Métodos - Cálculos adicionais #####

    ##Calcula o valor da não-uniformidade percentual esperada do
injetor [Bayvel]
    def calc_non_uniformity(self, r_in_pos, r_n, n, l_s, r_s,
phi_eq):
        return
23.7/((r_in_pos/r_n)**2.7*n**1.34*phi_eq**1.1*(l_s/(2*r_s))**0.15)

```

```

def calc_2alpha_avg(self, mu, A, phi, ksi, C):
    return 2*(180/np.pi)*np.arcsin(2*mu*A/((1+np.sqrt((1-
phi))*(1-ksi*mu**2*(A/(C))**2))))

##### Cálculo do injetor #####

def run_M2(self):

    self.fr.read()

    #SIG - Swirl Injector from Geometry
    print("\n\n Method M2 \n")

    ##### ST1 Calculation #####

    print("m_target: \t%.2f \tg/s\n"%(np.round(self.fr.m_0,2)))

    print("\n ST1 - Geometria do injetor:")

    self.print_data("n", "%.1f"%(self.fr.n))

    self.print_data("r_in_orf", "%.2f mm"%(self.fr.r_in_orf))

    self.print_data("l_in", "%.2f mm"%(self.fr.l_in))

    self.print_data("r_s", "%.2f mm"%(self.fr.r_s))

    self.print_data("l_s", "%.2f mm"%(self.fr.l_s))

    self.print_data("r_n", "%.2f mm"%(self.fr.r_n))

    self.print_data("r_in_pos", "%.2f mm"%(self.fr.r_in_pos))

    self.n_conv = []
    self.m_ideal = []
    self.m_real = []
    self.alpha2_ideal = []
    self.alpha2_real_eq = []
    self.alpha2_real_i = []

    self.pressure_range = np.arange(self.delta_p_i,
self.delta_p_f+self.delta_p_step, self.delta_p_step)

    for self.delta_p in self.pressure_range:

        self.erro_m = self.erro_max + 1
        self.m_prev = self.erro_max + 1
        self.contador = 0

        while self.erro_m >= self.erro_max and self.contador <
self.it_lim:

            ##### ST1 - Cálculo do Estágio Ideal #####

```

```

    ### Ideal Injector Parameters    ###

    self.A = self.calc_geom_char(self.fr.r_in_pos,
self.fr.r_n, self.fr.n, self.fr.r_in_orf, 0)

    self.phi = self.calc_phi(self.A)

    #Coeficiente de Descarga ideal
    self.mu = self.phi*np.sqrt(self.phi/(2-self.phi))

    #raio da camada líquida ideal
    self.r_mn = self.fr.r_n*np.sqrt(1-self.phi)

    #espessura da camada líquida ideal
    self.t_mn = self.fr.r_n-self.r_mn

    #Vazão mássica ideal
    self.m = self.mu*(np.pi*self.fr.r_n**2)*1e-
6*np.sqrt(2*self.delta_p*1e5*self.fr.rho)*1e3 #vazão mássica ideal [g/s]

    #Coeficiente de redução de área do estágio
    self.C = self.fr.r_in_pos/(self.fr.r_n)

    ##### Cálculo do ângulo de spray ideal #####

    self.alpha2 = self.calc_2alpha(self.phi)

    ##### ST1 - Cálculo do Estágio Real #####

    #Número de Reynolds nos canais tangenciais
    if(self.contador == 0):
        self.rey_in =
self.calc_reynolds_in(self.m,self.fr.din_visc,self.fr.n,self.fr.r_in_orf)
    else:
        self.rey_in =
self.calc_reynolds_in(self.m_prev,self.fr.din_visc,self.fr.n,self.fr.r_in_o
rf)

    #print("\nrey_in = ", self.rey_in)

    #Coeficiente de resistência de superfície
    self.friction_coef =
self.calc_friction_coef(self.rey_in)

    #Parâmetro Geométrico equivalente
    self.A_eq = self.calc_geom_char(self.fr.r_in_pos,
self.fr.r_n, self.fr.n, self.fr.r_in_orf, self.friction_coef)

    ##### Cálculo do Coeficiente de área livre real
    self.phi_eq = self.calc_phi(self.A_eq)

```

```

#raio da camada líquida real
self.r_mn_eq = self.fr.r_n*np.sqrt(1-self.phi_eq)

#Espessura de camada líquida real
self.t_mn_eq = self.fr.r_n*(1-np.sqrt(1-self.phi_eq))

#mm

#Coeficiente de perda de momento angular
self.K = self.A_eq/self.A

##### Cálculo da perda nos canais tangenciais #####

self.ksi = self.calc_hyd_loss(self.fr.inlet_type,
self.fr.r_s, self.fr.l_in, self.fr.inlet_radius, self.fr.r_in_orf,
self.friction_coef)

#Coeficiente de Descarga
self.mu_eq = self.calc_mu_eq(self.phi_eq)

self.mu_i = self.calc_mu_i(self.mu_eq, self.A_eq,
self.fr.r_in_pos, self.fr.r_n, self.ksi)

#Vazão mássica real
self.m_i = np.pi*(self.fr.r_n*1e-
3)**2*np.sqrt(2*self.fr.rho*self.delta_p*1e5)/np.sqrt((2-
self.phi_eq)/self.phi_eq**3+self.ksi*self.A**2/(self.fr.r_in_pos/self.fr.r_n)**2)*1e3

#print("m_i = ", self.m_i, " g/s")

self.erro_m = abs((self.m_i-self.m_prev)/self.m_prev)
#print("erro_m = ", self.erro_m)

self.m_prev = self.m_i

self.C = self.fr.r_in_pos/self.fr.r_n

##### Cálculo do ângulo de spray real #####
self.alpha2_eq = self.calc_2alpha(self.phi_eq)
self.alpha2_i = self.calc_2alpha_avg(self.mu_i,
self.A_eq, self.phi_eq, self.ksi, self.C)

##### Cálculo da não-uniformidade #####
self.I = self.calc_non_uniformity(self.fr.r_in_pos,
self.fr.r_n, self.fr.n, self.fr.l_s, self.fr.r_s, self.phi_eq)

self.contador +=1

self.n_conv.append(self.contador)

self.m_ideal.append(self.m)
self.m_real.append(self.m_i)

self.alpha2_ideal.append(self.alpha2)
self.alpha2_real_eq.append(self.alpha2_eq)
self.alpha2_real_i.append(self.alpha2_i)

```

```

##### Tests #####
if(self.t_mn <= 0): print("\nERROR: invalid t_mn value
(delta_p = %.2f bar)"%(self.delta_p))
elif(self.t_mn <= 0.1): print("\nCaution! Low value of
t_mn (delta_p = %.2f bar)"%(self.delta_p))

if(self.t_mn_eq <= 0): print("\nERROR: invalid t_mn_eq
value (delta_p = %.2f bar)"%(self.delta_p))
elif(self.t_mn_eq <= 0.1): print("\nCaution! Low value of
t_mn_eq (delta_p = %.2f bar)"%(self.delta_p))

if(self.rey_in < 1e3): print("\nERROR: Low Reynolds
number (%.2E < 1e4) (delta_p = %.2f bar)"%(self.rey_in, self.delta_p))
elif(self.rey_in > 1e7): print("\nCaution: High Reynolds
number (%.2E > 1e6) (delta_p = %.2f bar)"%(self.rey_in, self.delta_p))

if(self.contador >= self.it_lim):
    print ("\n\tWarning:\n iteration limit reached
(delta_p = %.2f bar)"%(self.delta_p))
    print ("\terror = ", self.erro_m)

##### #####
##### #####
##### #####
##### ##### Plot Vazão x DeltaP ### #####
##### #####
##### #####
##### #####
##### #####

fig, ax11 = plt.subplots(figsize=(14, 7))
ax11.grid()
ax11.set_title("ṁ x ΔP")
ax11.plot(self.pressure_range, self.m_ideal, 'k--',
markersize=1, label = "ṁ fluido ideal", color = '0.5')
ax11.plot(self.pressure_range, self.m_real, 'k:',
markersize=3, label = "ṁ fluido viscoso")
ax11.plot(self.pressure_range,
[self.fr.m_0]*len(self.pressure_range), 'k-.', markersize=1, label = "ṁ
alvo")
ax11.plot(self.fr.delta_p0, self.fr.m_0, '.', markersize=20,
label = "ṁxΔP alvo", color = '0.5')
ax11.legend()
ax11.set_ylabel("ṁ [g/s]")
ax11.set_xlabel("ΔP [bar]")
ax11.tick_params(axis='x', labels=8)
ax11.tick_params(axis='y', labels=8)

##### #####
##### #####

```

```

#####
##### #####
##### ##### Plot Alpha2 x DeltaP ##
##### #####
##### #####
##### #####
##### #####

fig, ax11 = plt.subplots(figsize=(14, 7))
ax11.grid()
ax11.set_title("2α x ΔP")
ax11.plot(self.pressure_range, self.alpha2_real_i, 'k-.',
markersize=3, label = "2α fluido viscoso")
ax11.legend()
ax11.set_ylabel("2α [deg]")
ax11.set_xlabel("ΔP [bar]")

ax11.tick_params(axis='x', labelsz=8)
ax11.tick_params(axis='y', labelsz=8)

```

8.4 M2 Input File – RD0110

8.4.1 Estágio 1 – Oxidante: RD0110_Ox.txt

```
#####  
#File containing basic injector parameters for geometry calculation#####  
#####  
#####  
#####  
#####Injector ID  
RD0110 - Ox  
#00#ST1 Type of channel entrance (curved or straight)  
curved  
#01#ST1 propellants  
Liquid Oxygen (100 K)  
#02#ST1 mass flow reference [g/s]  
172.9  
#03#ST1 DeltaP [bar]  
4.26  
#04#ST1 number of entrance channels  
6  
#05#ST1 Entrance Channels Radius [mm]  
0.85  
#06#ST1 Entrance Channels Length [mm]  
3.40  
#07#ST1 Entrance Channel Chamfer [mm] (Equal to 0.0 if entrance type equal to "curved")  
0.0  
#08#ST1 Swirl Chamber Radius [mm]  
4.50  
#09#ST1 Swirl Chamber Length [mm]  
10.55  
#10#ST1 Nozzle Radius [mm]
```

2.70

#11#ST1 Nozzle Length [mm]

11.00

#12#ST1 Entrance Radius [mm]

3.50

8.4.2 Estágio 2 – Combustível: RD0110_F.txt

```
#####  
#File containing basic injector parameters for geometry calculation#####  
#####
```

#####Injector ID

RD0110 - F

#00#ST1 Type of channel entrance (curved or straight)

curved

#01#ST1 propellants

kerosene (293 K)

#02#ST1 mass flow reference [g/s]

64.8

#03#ST1 DeltaP [bar]

6.96

#04#ST1 number of entrance channels

6

#05#ST1 Entrance Channels Radius [mm]

0.35

#06#ST1 Entrance Channels Length [mm]

1.20

#07#ST1 Entrance Channel Chamfer [mm] (Equal to 0.0 if entrance type equal to "curved")

0.0

#08#ST1 Swirl Chamber Radius [mm]

5.00

#09#ST1 Swirl Chamber Length [mm]

5.00

#10#ST1 Nozzle Radius [mm]

5.00

#11#ST1 Nozzle Length [mm]

5.00

#12#ST1 Entrance Radius [mm]

4.50

8.5 M2 Input File – RD0109

8.5.1 Estágio 1 – Oxidante: RD0109_Ox.txt

```
#####  
#File containing basic injector parameters for geometry calculation#####  
#####  
#####  
#####  
#####Injector ID  
RD0110 - Ox  
#00#ST1 Type of channel entrance (curved or straight)  
curved  
#01#ST1 propellants  
Liquid Oxygen (100 K)  
#02#ST1 mass flow reference [g/s]  
172.9  
#03#ST1 DeltaP [bar]  
4.40  
#04#ST1 number of entrance channels  
6  
#05#ST1 Entrance Channels Radius [mm]  
0.90  
#06#ST1 Entrance Channels Length [mm]  
5.40  
#07#ST1 Entrance Channel Chamfer [mm] (Equal to 0.0 if entrance type equal to "curved")  
0.0  
#08#ST1 Swirl Chamber Radius [mm]  
4.50  
#09#ST1 Swirl Chamber Length [mm]  
18.00  
#10#ST1 Nozzle Radius [mm]
```

2.70

#11#ST1 Nozzle Length [mm]

3.74

#12#ST1 Entrance Radius [mm]

3.60

8.5.2 Estágio 2 – Combustível: RD0109_F.txt

```
#####  
#File containing basic injector parameters for geometry calculation#####  
#####
```

#####Injector ID

RD0109 - F

#00#ST1 Type of channel entrance (curved or straight)

curved

#01#ST1 propellants

kerosene (293 K)

#02#ST1 mass flow reference [g/s]

72.2

#03#ST1 DeltaP [bar]

7.10

#04#ST1 number of entrance channels

6

#05#ST1 Inlet Channels Radius [mm]

0.40

#06#ST1 Inlet Channels Length [mm]

2.40

#07#ST1 Inlet Channel Chamfer [mm] (Equal to 0.0 if entrance type equal to "curved")

0.0

#08#ST1 Swirl Chamber Radius [mm]

5.10

#09#ST1 Swirl Chamber Length [mm]

3.10

#10#ST1 Nozzle Radius [mm]

5.10

#11#ST1 Nozzle Length [mm]

3.10

#12#ST1 Entrance Radius [mm]

4.70

8.6 M2 Input File – Injetor Proposto Central

8.6.1 Estágio 1 – Oxidante: B01_C_Ox.txt

```
#####  
#File containing basic injector parameters for geometry calculation#####  
#####  
#####  
#####  
#####Injector ID  
Rev-B01_W Oxidizer  
#00#ST1 Type of channel entrance (curved or straight)  
curved  
#01#ST1 propellants  
Liquid Oxygen (90 K)  
#02#ST1 mass flow reference [g/s]  
190.30  
#03#ST1 DeltaP [bar]  
6.00  
#04#ST1 number of entrance channels  
6  
#05#ST1 Entrance Channels Radius [mm]  
0.78  
#06#ST1 Entrance Channels Length [mm]  
1.57  
#07#ST1 Entrance Channel Chamfer [mm] (Equal to 0.0 if entrance type equal to "curved")  
0.0  
#08#ST1 Swirl Chamber Radius [mm]  
5.10  
#09#ST1 Swirl Chamber Length [mm]  
12.74  
#10#ST1 Nozzle Radius [mm]
```

3.08

#11#ST1 Nozzle Length [mm]

12.33

#12#ST1 Entrance Radius [mm]

4.31

8.6.2 Estágio 2 – Combustível: B01_C_F.txt

```
#####  
#File containing basic injector parameters for geometry calculation#####  
#####
```

#####Injector ID

Rev-B01_C Fuel

#00#ST1 Type of channel entrance (curved or straight)

curved

#01#ST1 propellants

Ethanol 95% (300 K)

#02#ST1 mass flow reference [g/s]

95.10

#03#ST1 DeltaP [bar]

4.00

#04#ST1 number of entrance channels

6

#05#ST1 Entrance Channels Radius [mm]

0.56

#06#ST1 Entrance Channels Length [mm]

1.11

#07#ST1 Entrance Channel Chamfer [mm] (Equal to 0.0 if entrance type equal to "curved")

0.0

#08#ST1 Swirl Chamber Radius [mm]

5.87

#09#ST1 Swirl Chamber Length [mm]

5.87

#10#ST1 Nozzle Radius [mm]

5.31

#11#ST1 Nozzle Length [mm]

2.654

#12#ST1 Entrance Radius [mm]

5.31

8.7 M2 Input File – Injetor Proposto Periférico

8.7.1 Estágio 1 – Oxidante: B01_W_Ox.txt

```
#####  
#File containing basic injector parameters for geometry calculation#####  
#####  
#####  
#####  
#####Injector ID  
Rev-B01_W Oxidizer  
#00#ST1 Type of channel entrance (curved or straight)  
curved  
#01#ST1 propellants  
Liquid Oxygen (90 K)  
#02#ST1 mass flow reference [g/s]  
20.80  
#03#ST1 DeltaP [bar]  
6.00  
#04#ST1 number of entrance channels  
3  
#05#ST1 Entrance Channels Radius [mm]  
0.352  
#06#ST1 Entrance Channels Length [mm]  
2.813  
#07#ST1 Entrance Channel Chamfer [mm] (Equal to 0.0 if entrance type equal to "curved")  
0.0  
#08#ST1 Swirl Chamber Radius [mm]  
1.470  
#09#ST1 Swirl Chamber Length [mm]  
5.879  
#10#ST1 Nozzle Radius [mm]
```

0.93

#11#ST1 Nozzle Length [mm]

3.727

#12#ST1 Entrance Radius [mm]

1.118

8.7.2 Estágio 2 – Combustível: B01_W_F.txt

```
#####  
#File containing basic injector parameters for geometry calculation#####  
#####  
#####
```

#####Injector ID

Rev-B01_W Fuel

#00#ST1 Type of channel entrance (curved or straight)

curved

#01#ST1 propellants

Ethanol 95% (300 K)

#02#ST1 mass flow reference [g/s]

20.8

#03#ST1 DeltaP [bar]

4.00

#04#ST1 number of entrance channels

3

#05#ST1 Entrance Channels Radius [mm]

0.369

#06#ST1 Entrance Channels Length [mm]

1.475

#07#ST1 Entrance Channel Chamfer [mm] (Equal to 0.0 if entrance type equal to "curved")

0.0

#08#ST1 Swirl Chamber Radius [mm]

3.077

#09#ST1 Swirl Chamber Length [mm]

4.308

#10#ST1 Nozzle Radius [mm]

2.580

#11#ST1 Nozzle Length [mm]

1.548

#12#ST1 Entrance Radius [mm]

2.709